

Оглавление

ВВЕДЕНИЕ.....	5
1 Постановка задачи.....	8
1.1 Актуальность работы.....	8
1.2 Системный анализ существующих программных решений автоматизации мониторинга.....	8
1.3 Постановка задачи.....	17
1.4 Функциональные требования.....	18
1.4.1 Роли пользователей.....	18
1.4.2 Таблица функциональных требований (FR).....	19
1.5 Нефункциональные требования.....	19
Выводы по разделу.....	21
2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ.....	22
2.1. Архитектура системы и обоснование технологического стека.....	22
2.1.1. Назначение и общая идея.....	22
2.1.2. Уровень представления (клиент).....	23
2.1.3. Уровень безопасности и идентификации.....	24
2.1.4. Уровень приложения и оркестрации (ядро системы).....	24
2.1.5 Уровень доменных микросервисов.....	26
2.1.6. Уровень данных (логическое разделение хранилищ).....	27
2.1.7 Физическая архитектура (Docker Compose).....	28
2.2 Обоснование выбора технологий.....	29
2.2.1 Слой бэкенда и асинхронных задач.....	29
2.2.2 Слой хранения данных и кэширования.....	29
2.2.3 Фронтенд и клиентский слой.....	30
2.2.4 Безопасность и доставка (Deployment).....	30
2.3 Роли пользователей и права доступа.....	31
2.4 Диаграммы вариантов использования (Use Case).....	33
2.4.1 Общая диаграмма прецедентов.....	33

2.5.2 Таблица прецедентов.....	34
2.6 Потоки данных.....	35
2.6.1 Сквозной поток: онбординг контейнера.....	35
2.6.2 Поток аутентификации.....	37
2.6.3 Фоновый поток Celery.....	38
2.7 Проектирование базы данных PostgreSQL.....	39
2.7.1 Концептуальная модель.....	39
2.7.2 Логическая модель (таблицы и связи).....	40
2.7.3 Физическая модель (типы и индексы).....	42
2.8 Схема данных в Redis.....	43
2.8.1 Разделение logical databases.....	43
2.8.2 Ключи приложения (DB 0).....	43
2.9 Схема данных в MinIO (S3).....	44
2.9.1 Общие параметры.....	44
2.9.2 Иерархия объектов.....	45
2.9.3 Таблица объектов.....	46
2.10 Бизнес-логика.....	47
2.10.1 Классификация контейнеров.....	47
2.10.2 Генерация мониторинга.....	47
2.10.3 Модель IDEF0 (контекст).....	48
2.11 Пользовательский интерфейс.....	49
2.11.1 Карта экранов.....	49
2.11.2 Защита маршрутов.....	52
Выводы по разделу 2.....	52
3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ.....	53
3.1 Структура репозитория и средства развёртывания.....	53
3.1.1 Организация исходного кода.....	53
3.1.2 Docker Compose (реализация физического стенда).....	54

3.2 API Aggregator - детальная реализация.....	54
3.3 Users API - реализация.....	56
3.4 Docker API - реализация.....	56
3.5 Docker Classification - реализация алгоритма.....	56
3.5.1 WeightedDiscovery.....	56
3.5.2 Отличие от signatures.yml корня.....	58
3.6 Prometheus Generation - реализация.....	58
3.6.1 Poyter generate.....	58
3.6.2 PrometheusConfigGenerator.generate_config (логика).....	59
3.6.3 MinioService (generation).....	60
3.7 Prometheus Manager и Grafana.....	61
3.8 Frontend - реализация клиента.....	61
3.9 Nginx.....	62
3.10 Файл signatures.yml - реализация конфигурации платформы.....	62
Выводы по разделу 3.....	63
4. ТЕСТИРОВАНИЕ И АПРОБАЦИЯ СИСТЕМЫ.....	65
4.1 Цели и стратегия тестирования.....	65
4.2 Средства и окружение тестирования.....	65
4.3 Автоматизированное тестирование.....	65
4.4 Ручная апробация и интеграционные сценарии.....	66
4.5 Проверка нефункциональных требований.....	66
4.6 Ограничения тестирования.....	66
Выводы по разделу 4.....	67
5. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	68
5.1 Назначение и область применения.....	68
5.2 Требования к рабочему месту пользователя.....	68
5.2.1 Программные требования.	68
5.2.2 Предварительные условия на стороне инфраструктуры.	69
5.3 Обзор интерфейса и навигация.....	69
5.4 Вход в систему и завершение сеанса.....	70
5.4.1 Первый вход (учётная запись maintainer).....	70

5.5 Работа с хостами (раздел Hosts).....	70
5.5.1 Просмотр списка хостов.....	70
5.5.2 Добавление хоста.....	71
5.6 Работа с контейнерами (раздел Containers).....	71
5.6.1 Панель фильтрации и обновления.....	72
5.6.2 Столбцы таблицы контейнеров.....	72
5.6.3. Меню Actions (операции над контейнером).....	73
5.6.5 Карточка контейнера (детали).....	74
5.7 Сквозной сценарий онбординга («мониторинг по кнопке»).....	74
5.7.1 Вариант А: «Start all» (одна команда).....	75
5.7.2 Вариант В: пошаговый онбординг.....	75
5.8 Раздел Prometheus.....	77
5.8.1 Блок Prometheus Manager.....	78
5.8.2 Main Prometheus Config.....	78
5.8.3 Таблица Services.....	78
5.9 Раздел Grafana.....	79
5.9.1 Grafana Manager.....	79
5.9.2 Available Containers.....	79
5.9.3 Working Dashboards.....	79
5.10 Раздел Configuration.....	80
5.10.2 Prometheus manager settings.....	81
5.11 Управление пользователями (только maintainer).....	81
Выводы по разделу 5.....	81
ЗАКЛЮЧЕНИЕ.....	83
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	86
ПРИЛОЖЕНИЕ А.....	89
ПРИЛОЖЕНИЕ Б.....	123

ВВЕДЕНИЕ

Современная индустрия разработки программного обеспечения характеризуется повсеместным переходом от монолитных архитектурных решений к микросервисным распределенным системам, функционирующим в изолированных контейнеризированных средах на базе платформы Docker. Рост масштабируемости, связности и динамичности подобных ИТ-инфраструктур выдвигает повышенные требования к обеспечению непрерывного контроля их работоспособности. В данных условиях классические подходы к мониторингу уступают место комплексной концепции наблюдаемости (observability), базирующейся на сквозном сборе и анализе трех фундаментальных типов данных: метрик, журналов событий (логов) и трассировок запросов.

Сложность обеспечения наблюдаемости в высокодинамичных Docker-средах обусловлена высокой волатильностью и коротким жизненным циклом объектов наблюдения. Контейнеры могут автоматически масштабироваться, перезапускаться и перемещаться между узлами распределенной системы, из-за чего время их существования нередко исчисляется минутами. Традиционные методы статического ручного конфигурирования систем мониторинга (таких как Prometheus) в этих условиях оказываются неэффективными и приводят к возникновению критической операционной нагрузки на инженерный персонал (инженеров эксплуатации и SRE). Специалисты сталкиваются с необходимостью непрерывного ручного поиска новых контейнеров, определения их функционального назначения (технологического стека), развертывания специализированных агентов сбора метрик (экспортеров) и рутинного обновления конфигурационных файлов центрального сервера сбора данных.

Согласно методологии обеспечения надежности систем (Site Reliability Engineering, SRE), объем рутинных неавтоматизируемых операций (toil) в общей структуре трудозатрат инженера не должен превышать 50%. Однако на практике в крупных контейнерных кластерах задачи сопровождения контура мониторинга могут поглощать до 70–80% рабочего времени, существенно повышая вероятность возникновения ошибок, связанных с человеческим фактором. Любая задержка или ошибка при ручном обновлении конфигурации ведет к потере видимости инфраструктуры («слепым зонам») и, как следствие, к увеличению среднего времени обнаружения инцидентов (Time to Detect, TTD), что влечет за собой прямые финансовые и репутационные потери для бизнеса.

Таким образом, существует явное техническое противоречие между высокой динамичностью и гетерогенностью современных контейнеризированных ИТ-

инфраструктур и статическим характером традиционных инструментов обеспечения их наблюдаемости. Необходимость разрешения данного противоречия определяет актуальность разработки автоматизированного комплекса, реализующего замкнутый цикл обнаружения, классификации и конфигурационного сопровождения распределенного контура мониторинга. Вопросам автоматизации сбора метрик в распределенных вычислительных системах посвящено множество современных исследований и коммерческих разработок. На рынке представлены такие решения, как VictoriaMetrics (компонент vmagent), Grafana Alloy, Netdata и Checkmk. Проведенный в рамках данной работы системный анализ показал, что существующие программные средства автоматизации либо накладывают жесткие требования к экспертной квалификации персонала для ручного написания сложных декларативных правил фильтрации и преобразования данных, либо жестко ограничены рамками локального вычислительного узла и не обеспечивают комплексного сквозного управления жизненным циклом конфигураций как централизованных артефактов в масштабе всей распределенной инфраструктуры.

Объект исследования: процессы обеспечения наблюдаемости (observability) программных компонентов в распределенных контейнеризированных вычислительных средах.

Предмет исследования: методы, алгоритмы и программные средства автоматического обнаружения, идентификации и конфигурационного сопровождения компонентов систем мониторинга и визуализации ИТ-инфраструктуры.

Цель работы - повышение эксплуатационной надежности распределенных контейнеризированных ИТ-инфраструктур и сокращение среднего времени обнаружения аварийных инцидентов (Time to Detect) за счет снижения операционной рутинной нагрузки на инженерный персонал путем автоматизации процессов сквозного конфигурирования и развертывания компонентов контура наблюдаемости.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести системный анализ существующих программных решений в области автоматизации мониторинга, выявить их архитектурные ограничения и формализовать требования к разрабатываемому комплексу.
2. Разработать алгоритм и гибкую систему правил взвешенной сигнатурной классификации технологического стека Docker-контейнеров на основе интеллектуального анализа их

метаданных (образов, открытых сетевых портов, переменных окружения и системных меток).

3. Спроектировать событийно-ориентированную микросервисную архитектуру автоматизированного комплекса, обеспечивающую изоляцию компонентов, отказоустойчивость и горизонтальное масштабирование подсистем сбора данных.
4. Разработать программный комплекс *Auto_Observability*, реализующий автоматический сбор инвентаря удаленных хостов, динамическую генерацию конфигурационных файлов СУБД Prometheus, автоматизированную оркестрацию sidecar-контейнеров экспортеров и импорт параметризованных аналитических панелей (дашбордов) в Grafana.
5. Провести экспериментальную апробацию разработанного решения на испытательном стенде, выполнить оценку его производительности, масштабируемости и рассчитать показатели снижения временных затрат на онбординг новой инфраструктуры в сравнении с традиционными подходами.

Методы исследования. Для решения поставленных задач использовались положения системного анализа, теории вычислительных систем и сетей, методы объектно-ориентированного и микросервисного проектирования, принципы контейнеризации и автоматизации инфраструктуры как кода (Infrastructure as Code, IaC), а также методы сигнатурного анализа и математического моделирования операционной нагрузки.

1 Постановка задачи

1.1 Актуальность работы

В условиях цифровой трансформации бизнеса надежность серверных приложений напрямую коррелирует с финансовыми показателями компании. Одним из главных препятствий для обеспечения этой надежности в современных Docker-средах является «конфигурационный ад» (YAML hell) - избыточная сложность ручного управления настройками мониторинга.

Согласно методологии Site Reliability Engineering (SRE), разработанной в Google, количество рутинных операций (toil) в работе инженера не должно превышать 50%. Однако на практике в динамических окружениях значительная часть времени тратится на:

- поиск вновь запущенных контейнеров и определение их типа;
- ручное написание и репликацию scrape_configs для Prometheus;
- развертывание и настройку соответствующих sidecar-контейнеров (экспортеров);
- поддержание актуальности реестра хостов и их доступности.

Если общее время эксплуатации системы T_{ops} представить как функцию от количества контейнеров n , то при ручном управлении трудозатраты растут линейно. Математически эффективность автоматизации можно выразить через снижение операционной нагрузки:

$$T_{ops} = T_{dev} + \sum_{i=1}^n (T_{manual,i} \times (1 - \alpha))$$

,где α – коэффициент автоматизации. Разработка системы, способной автоматически проводить онбординг контейнеров в мониторинг, является актуальной задачей, так как позволяет радикально снизить риск человеческой ошибки и ускорить время обнаружения инцидентов (Time to Detect).

1.2 Системный анализ существующих программных решений автоматизации мониторинга

Для формализации требований к разрабатываемому программному комплексу необходимо провести детальный системный анализ существующих программных средств автоматизации сбора телеметрии, представленных на современном рынке, выявив их архитектурные особенности, достоинства, недостатки и эксплуатационные ограничения в контексте динамических контейнеризированных сред. Рассмотрим четыре наиболее характерных решения.

1. VictoriaMetrics (компонент vmagent)

VictoriaMetrics представляет собой высокопроизводительную специализированную систему управления базами данных временных рядов (Time Series Database, TSDB). В составе данной экосистемы поставляется облегченный агент сбора данных vmagent, предназначенный для фильтрации, агрегации и маршрутизации метрик. Для автоматизации обнаружения объектов наблюдения в динамических средах агент поддерживает нативный механизм `docker_sd_config`. Данный компонент осуществляет непрерывный опрос локального или удаленного сокета Docker API, извлекая актуальный список идентификаторов контейнеров, их сетевых адресов и внутренних метаданных.

- **Достоинства:** Исключительно высокая эффективность сжатия данных в хранилище (в 7–10 раз превосходящая стандартные показатели Prometheus) и минимальное потребление оперативной памяти и ресурсов процессора агентом сбора. Механизм `docker_sd_config` избавляет администратора от необходимости статического описания IP-адресов каждого контейнера.
- **Архитектурные ограничения:** Главным недостатком является полное отсутствие встроенных интеллектуальных механизмов классификации типов прикладной нагрузки. Компонент vmagent извлекает из Docker API лишь «сырые» метаданные. Без предварительной разметки самих контейнеров системными метками (labels) на этапе их сборки или запуска, система мониторинга не способна самостоятельно определить функциональное назначение контейнера (например, отличить экземпляр СУБД PostgreSQL от брокера сообщений Redis). Инженер эксплуатации вынужден вручную проектировать сложные и громоздкие правила переопределения меток (relabeling) с использованием регулярных выражений в конфигурационных файлах. Таким образом, автоматизация сбора метрик для гетерогенного стека приложений без жестко стандартизированной предварительной разметки инфраструктуры остается нереализованной.

Кроме того, специфика декларативного описания конфигураций для данного агента накладывает на администратора высокую операционную нагрузку. Необходимость явного сопоставления строковых шаблонов, фильтрации системных параметров контейнеров и принудительного переопределения целевых портов сбора телеметрии приводит к существенному усложнению инфраструктурного кода. Практический пример реализации подобного конфигурационного файла, содержащего избыточный набор правил фильтрации и регулярных выражений для СУБД временных рядов, представлен на рисунке 1.

```
1 scrape_configs:
2   - job_name: 'docker_apps_vmagent_relabeling'
3     docker_sd_configs:
4       - host: unix:///var/run/docker.sock
5         refresh_interval: 30s
6     relabel_configs:
7       # 1. Отсекаем контейнеры, у которых нет метки автоматического сбора
8       - source_labels: [__meta_docker_container_label_monitor_enable]
9         regex: 'true'
10        action: keep
11       # 2. Выделяем тип приложения из образа и пишем в метку 'app_type'
12       - source_labels: [__meta_docker_container_image]
13         regex: '.*(postgres|postgis).*'
14         target_label: app_type
15         replacement: 'postgresql'
16         action: replace
17       # 3. Сложное регулярное выражение для извлечения имени проекта и окружения
18       - source_labels: [__meta_docker_container_name]
19         regex: '^/([a-zA-Z0-9_-]+)-(prod|stage|dev)-([a-zA-Z0-9_-]+)$'
20         target_label: project_env
21         replacement: '$1_$2'
22         action: replace
23       # 4. Переопределение порта для сбора метрик на основе внутренних метаданных
24       - source_labels: [__address__, __meta_docker_container_label_monitor_port]
25         regex: '(.+):(?:\d+);(\d+)'
26         target_label: __address__
27         replacement: '$1:$2'
28         action: replace
```

Рисунок 1 – Пример декларативного описания правил переопределения меток (relabeling) в конфигурационном файле СУБД временных рядов

2. Grafana Alloy

Grafana Alloy представляет собой современный универсальный коллектор телеметрических данных (метрики, логи, трассировки, профилирование), спроектированный как эволюционное развитие агентов Grafana Agent и Flow. Программный продукт ориентирован на построение гибких событийно-ориентированных конвейеров (пайплайнов) обработки данных. Взаимодействие с контейнерной инфраструктурой реализуется через специализированный декларативный блок компонентов discovery.docker.

- **Достоинства:** Высокая степень унификации (один агент заменяет собой стек из нескольких утилитарных сборщиков) и реактивное отслеживание событий жизненного цикла контейнеров в реальном времени. Компонент discovery.docker формирует

динамические внутренние массивы целей (targets), передавая их по цепочке смежным модулям обработки.

- **Архитектурные ограничения:** Чрезвычайно высокий порог вхождения и сложность сопровождения конфигурационного кода. Grafana Alloy использует собственный специализированный декларативный язык конфигурации River, синтаксис которого базируется на блоках, атрибутах и выражениях связывания. Несмотря на автоматическое обнаружение контейнеров, логика извлечения метрик, подбора портов, авторизации и сопоставления с шаблонами для каждого уникального программного стека должна быть явно запрограммирована инженером в конфигурации River. Это не решает проблему высокой операционной нагрузки (toil), а лишь смещает её из плоскости поддержки статических YAML-файлов в плоскость написания и отладки сложного кода на специфичном конфигурационном языке.

Принципиальной особенностью построения конвейеров в Grafana Alloy является организация направленных графов, где выходные значения одного модуля жестко связываются с входными параметрами следующего. Данный подход требует от инженера детального проектирования промежуточных этапов агрегации. Архитектурная топология указанных связей, отражающая последовательную передачу динамических массивов целей от стадии обнаружения до стадии отправки в хранилище на специализированном конфигурационном языке River, приведена на рисунке 2.

```
1 // Блок обнаружения: опрашивает локальный Docker API
2 discovery.docker "local_containers" {
3   host = "unix:///var/run/docker.sock"
4   refresh_interval = "15s"
5 }
6
7 // Блок фильтрации: выбирает только production-контейнеры
8 discovery.relabel "filter_production" {
9   targets = discovery.docker.local_containers.targets
10
11   rule {
12     source_labels = ["__meta_docker_container_label_environment"]
13     regex         = "production"
14     action        = "keep"
15   }
16 }
17
18 // Блок сбора метрик: отправляет цели на парсинг в Prometheus
19 prometheus.scrape "docker_metrics" {
20   targets = discovery.relabel "filter_production".output
21   forward_to = [prometheus.remote_write.local_storage.receiver]
22 }
```

Рисунок 2 – Структура связывания компонентов discovery и scrape на специализированном конфигурационном языке River

3. Netdata

В отличие от двух предыдущих решений, требующих активного участия инженера в настройке, Netdata позиционируется как распределенная система мониторинга вычислительных систем в реальном времени, функционирующая в соответствии с методологией «нулевой конфигурации» (Zero-Configuration). В архитектуру Netdata заложены механизмы автоматического сканирования операционной системы и контейнерного окружения с секундным интервалом дискретизации метрик.

- **Достоинства:** Высокая скорость развертывания и мгновенная готовность к работе. Встроенная подсистема автоматического обнаружения (автодискавери) включает в себя более 800 предустановленных коллекторов, которые самостоятельно идентифицируют запущенные в Docker процессы, сканируют открытые локальные сетевые порты и инициализируют сбор метрик с генерацией динамических визуальных панелей.
- **Архитектурные ограничения:** Выраженная хост-центричная архитектура (host-centric). Netdata проектировалась как локальный высокодетализированный монитор конкретного физической или виртуальной машины. Данное решение принципиально не имеет встроенных оркестрационных механизмов для централизованного декларативного управления конфигурациями распределенного парка серверов Prometheus, не способно выступать в роли координатора внешних sidecar-экспортеров и не обеспечивает сохранение конфигурационных артефактов как отчуждаемых элементов ИТ-инфраструктуры предприятия.

Автономность функционирования локального демона Netdata накладывает строгие ограничения на возможности масштабирования системы сбора телеметрии. В условиях распределенной микросервисной архитектуры локальные базы данных и контуры отображения оказываются изолированы внутри конкретной вычислительной единицы, что делает невозможным сквозной мониторинг. Архитектурная декомпозиция, иллюстрирующая структурное взаимодействие компонентов и изоляцию логических контуров обработки, представлена на рисунке 3.

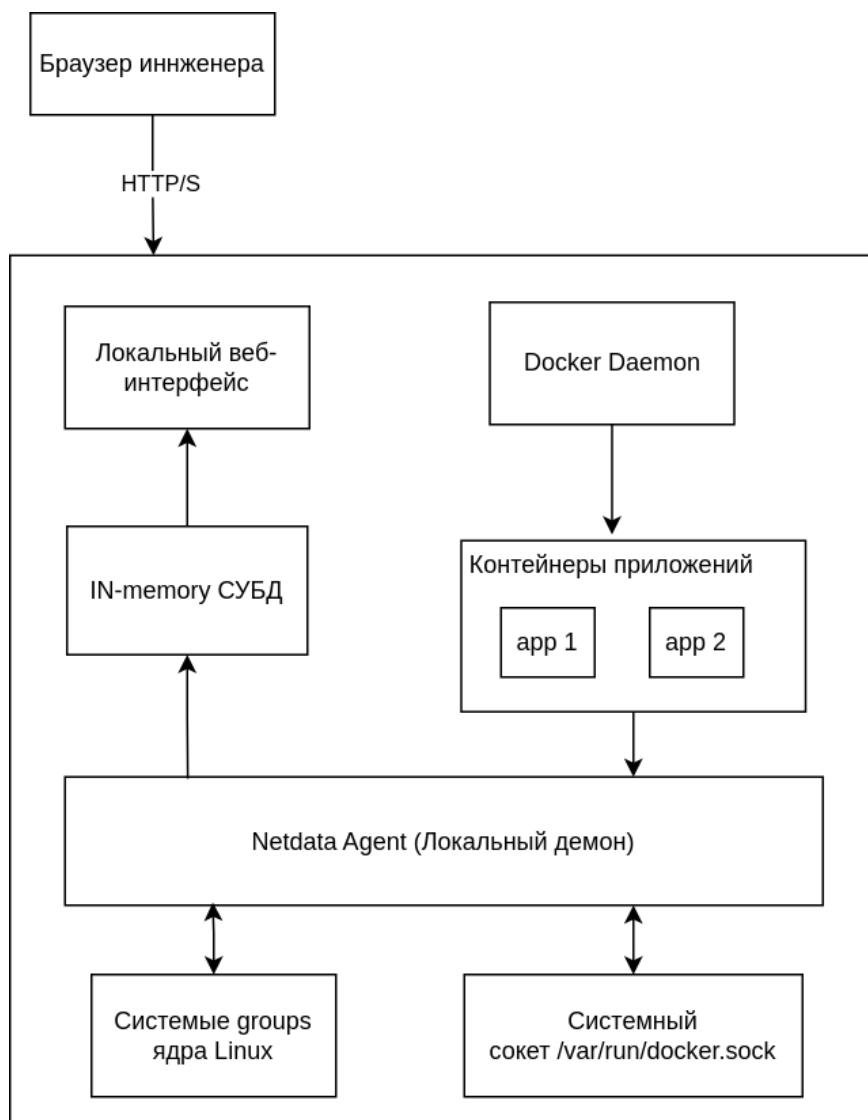


Рисунок 3 – Схема хост-центричной архитектуры локального агента мониторинга с изолированным контуром визуализации

4. Checkmk

Checkmk является комплексной программной платформой мониторинга ИТ-инфраструктуры корпоративного уровня (Enterprise), построенной на базе оптимизированного ядра сбора данных и поддерживающей глубокую автоматизацию через архитектурный модуль «фабрики агентов» (Agent Bakery).

Суть механизма «фабрики агентов» (Agent Bakery) заключается в централизованной динамической сборке и компиляции кастомизированных исполняемых пакетов агентов мониторинга (в форматах .deb, .rpm, .msi) под конкретные целевые хосты на основе правил, заданных в веб-интерфейсе платформы. Фабрика автоматически

внедряет в тело агента необходимые зависимости и скрипты расширения (плагины), избавляя от необходимости вручную настраивать конфигурации на сотнях серверов.

Для мониторинга Docker-инфраструктуры Checkmk использует специализированный плагин `mk_docker.py`. Этот скрипт, написанный на языке Python, внедряется фабрикой на целевой хост и выполняет следующие низкоуровневые операции:

1. Устанавливает прямое соединение с локальным сокетом `/var/run/docker.sock` для обхода сетевых ограничений.
 2. Выполняет глубокую инспекцию состояния (строковые и числовые параметры) каждого контейнера через вызов команд Docker Inspect API.
 3. Извлекает не только базовые системные метрики (потребление CPU, RAM, I/O sgroups), но и анализирует внутренние переменные окружения (ENV), состояние здоровья приложения (Healthcheck), количество перезапусков и топологию виртуальных сетей Docker.
 4. Агрегирует полученные данные в структурированный текстовый формат CheckmkAgent и передает их на центральный сервер мониторинга.
- **Достоинства:** Мощный функционал сквозной автоматизации и глубокая инспекция контейнерной нагрузки на низком системном уровне без необходимости предварительной ручной настройки экспортеров на целевых машинах.
 - **Архитектурные ограничения:** Чрезвычайно высокая избыточная сложность, монолитность и ресурсоемкость системы (heavyweight-платформа). Полноценное функционирование Checkmk требует развертывания обособленной масштабной инфраструктуры управления, что технически и экономически нецелесообразно для небольших или средних распределенных Docker-кластеров, а также тестовых и учебных контуров. Кроме того, Checkmk использует собственные проприетарные и закрытые стандарты представления метрик и внутренние интерфейсы визуализации (WATO), что делает систему полностью несовместимой с современной общепринятой открытой декларативной экосистемой Prometheus/Grafana и концепцией «Инфраструктура как код» (Infrastructure as Code, IaC).

Реализация сквозного мониторинга в рамках данной платформы жестко разделена на два изолированных контура. Первый контур отвечает за высокоуровневую оркестрацию, компиляцию зависимостей и генерацию исполняемых бинарных манифестов, в то время как второй осуществляет непосредственное низкоуровневое

сканирование Docker API на удаленных серверах. Алгоритм поэтапной сборки, трансфера и последующего сбора телеметрических данных с использованием специализированных модулей и сокетов ядра представлен на рисунке 4.

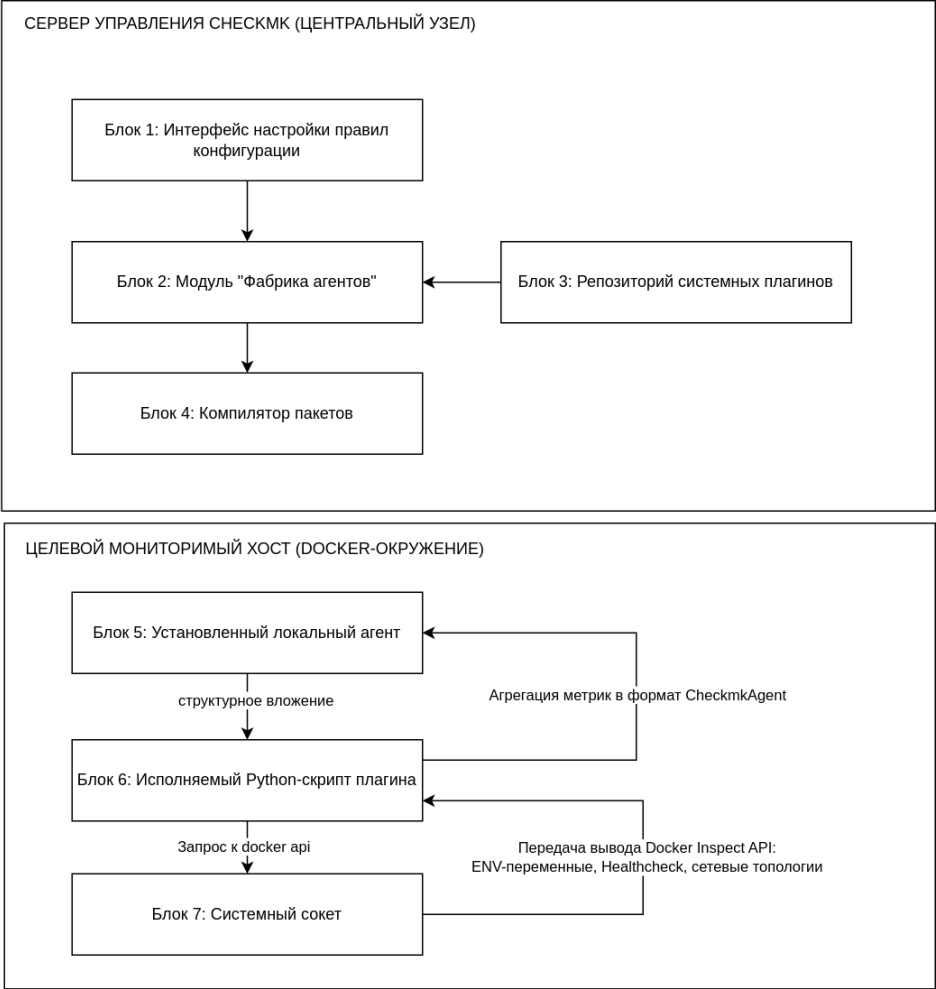


Рисунок 4 – Схема сквозного функционирования механизма «фабрики агентов» и низкоуровневого плагина инспекции контейнеров

Подводя итог аналитическому обзору, для наглядного сопоставления характеристик исследованных систем и выявления существующего технологического разрыва результаты системного анализа были структурированы и сведены в таблицу 1.

Таблица 1 – Сравнительный анализ существующих систем автоматизации мониторинга

Критерий сравнения	VictoriaMetrics	Grafana Alloy	Netdata	Checkmk	Разрабатываемый комплекс
Динамическое обнаружение	+	+	+	+	+

объектов					
Автоматическая классификация стека нагрузки без ручной разметки	-	-	+	+	+
Централизованное управление конфигурационными артефактами	-	-	-	+	+
Автоматизированное развертывание сопутствующих экспортеров	-	-	-	+	+
Автоматическая инициализация аналитических панелей	-	-	+	+	+
Сложность внедрения и сопровождения (порог вхождения)	средняя	высокая	низкая	Очень высокая	низкая

Как следует из таблицы, ни одна из рассмотренных систем в полной мере не решает задачу автоматизации полного цикла мониторинга в динамической Docker-среде при сохранении совместимости с экосистемой Prometheus/Grafana. Именно этот выявленный разрыв и определяет необходимость разработки собственного решения.

1.3 Постановка задачи

На основании проведенного системного анализа можно сформулировать цель работы. Цель проекта - разработка комплекса Auto_Observability для автоматизации контура наблюдаемости (observability) Docker-инфраструктуры: от обнаружения контейнеров на удалённых хостах до генерации конфигурации Prometheus, развёртывания экспортёров метрик и импорта дашбордов Grafana с минимальным ручным вмешательством оператора.

В современных микросервисных и контейнерных средах ручная настройка scrape_configs, target-файлов, sidecar-экспортёров и дашбордов для каждого нового сервиса отнимает значительное время у инженеров эксплуатации (SRE/DevOps). Существующие платформы мониторинга (Prometheus, Grafana) мощны, но не решают задачу онбординга нового контейнера «из коробки»: оператору всё равно нужно знать тип стека (PostgreSQL, Redis, Kafka и т.д.), подобрать экспортёр, написать YAML и связать его с дашбордом. Разрабатываемая система устраняет этот разрыв за счёт цепочки: обнаружение контейнеров → сигнатурная классификация стека → генерация артефактов мониторинга → применение конфигурации → визуализация в Grafana.

Для достижения поставленной цели необходимо решить следующие инженерные задачи:

1. **Провести системный анализ** предметной области, исследовать архитектурные особенности и ограничения существующих программных аналогов автоматизации мониторинга и формализовать структурированный перечень функциональных и нефункциональных требований к проектируемому комплексу.
2. **Разработать многофакторный алгоритм взвешенной сигнатурной классификации** технологического стека запущенных Docker-контейнеров на основе интеллектуального анализа комплекса их метаданных (структуры имени образа, открытых сетевых портов, системных меток и внутренних переменных окружения).
3. **Спроектировать событийно-ориентированную микросервисную архитектуру** автоматизированного комплекса, обеспечивающую изоляцию функциональных модулей, отказоустойчивость, слабую связность компонентов и горизонтальное масштабирование подсистем асинхронной обработки данных.
4. **Разработать централизованный программный интерфейс (API Aggregator)** и сопутствующую подсистему асинхронных фоновых задач на базе Celery и Redis для распределенного сбора инвентаря удаленных хостов в режиме реального времени.

5. **Реализовать программные модули «Фабрики конфигураций» и «Оркестратора экспортеров»**, обеспечивающие динамическую генерацию валидных YAML-артефактов для СУБД Prometheus с их версионизируемым хранением в объектном S3-хранилище MinIO, а также автоматизированный sidecar-запуск зависимых агентов сбора метрик в сетевых пространствах целевых приложений.
6. **Разработать интеграционный модуль визуализации** для автоматического импорта и параметризации аналитических панелей (дашбордов) в Grafana через HTTP API на основе динамических шаблонов.
7. **Разработать пользовательский графический интерфейс (Web-UI)** на базе фреймворка Vue 3, поддерживающий защищенное взаимодействие по протоколу JWT и реализующий ролевую модель разграничения прав доступа (RBAC) к инфраструктурным командам.
8. **Провести экспериментальную апробацию** разработанного программного комплекса на испытательном стенде, выполнить оценку производительности разработанных алгоритмов классификации и рассчитать показатели эффективности снижения временных затрат на постановку инфраструктуры под контроль в сравнении с традиционными подходами.

1.4 Функциональные требования

Перейдем к формализации требований к системе. Сначала определим роли пользователей, затем — конкретные функциональные возможности.

1.4.1 Роли пользователей

Система поддерживает четыре роли пользователей, которые представлены в таблице 2.

Таблица 2 – Таблица ролей пользователей

Роль	Описание	Основные права
maintainer	Первичный администратор (создается при старте)	Полный доступ к API агрегатора + управление пользователями
admin	Администратор мониторинга	Все операции агрегатора , без управления пользователями
dev	Разработчик	Просмотр + частично изменения; запрещены критичные операции (генерация конфигов, старт/стоп контейнеров, импорт дашбордов и т.д.)
user	Наблюдатель	Просмотр

1.4.2 Таблица функциональных требований (FR)

В таблице 3 представлены функциональные требования

Таблица 3 – Таблица функциональных требований (FR)

ID	Требование
FR-01	Вход в систему по логину и паролю, получение JWT
FR-02	Управление учётными записями (только maintainer)
FR-03	Регистрация, просмотр, изменение и удаление целевых хостов
FR-04	Периодическая и ручная синхронизация состояния хостов
FR-05	Получение списка контейнеров и обновление инвентаря
FR-06	Старт, остановка, удаление контейнера; очистка образов/томов
FR-07	Классификация контейнера по сигнатурам
FR-08	Генерация конфигурации Prometheus для контейнера
FR-09	Запуск sidecar-экспортёра метрик
FR-10	Управление signatures.yml (чтение/запись)
FR-11	Просмотр всех конфигураций и файлов из MinIO
FR-12	Управление главным prometheus.yml и target-файлами
FR-13	Управление контейнером Prometheus на хосте
FR-14	Импорт дашбордов Grafana по шаблону стека
FR-15	Учёт и удаление импортированных дашбордов
FR-16	Управление контейнером Grafana на хосте
FR-17	Фоновое обновление контейнеров (каждую минуту)
FR-18	Веб-интерфейс с маршрутизацией и проверкой роли
FR-19	RBAC на уровне API агрегатора

1.5 Нефункциональные требования

Наряду с функциональными требованиями, необходимо определить требования к качественным характеристикам системы. Они сгруппированы по характеристикам качества ISO/IEC 25010 и представлены в таблицах 4-9.

Таблица 4 – Таблица Платформа и развёртывание (NFR-01 – NFR-03)

ID	Требование
NFR-01	Воспроизводимый стенд на Docker Compose: PostgreSQL 15, Redis 7, MinIO, 10+ микросервисов
NFR-02	Python 3.11+, Node.js 18+ для локальной разработки
NFR-03	Healthcheck для критичных зависимостей (postgres, redis, minio, основные API)

Таблица 5 – Таблица Производительность и масштабируемость (NFR-04 – NFR-06)

ID	Требование
NFR-04	Кэширование инвентаря контейнеров и хостов в Redis для быстрого отклика UI
NFR-05	Горизонтальное масштабирование воркеров Celery (api_aggregator_celery_worker_1/2)
NFR-06	Целевое время ответа типовых GET-запросов агрегатора – порядка 1–3 с при доступных зависимостях (уточняется в разделе апробации)

Таблица 6 – Таблица Надёжность и сопровождаемость (NFR-07 – NFR-09)

ID	Требование
NFR-07	Крупные YAML-артефакты хранятся в MinIO, а не в BLOB-полях PostgreSQL
NFR-08	Микросервисная декомпозиция: изменение классификатора не затрагивает UI
NFR-09	Единый HTTP-клиент APIGateway для вызовов между сервисами

Таблица 7 – Таблица Безопасность (NFR-10 – NFR-14)

ID	Требование	Статус в прототипе
NFR-10	JWT с ограниченным сроком жизни (JWT_EXPIRE_MINUTES, по умолчанию 480 мин)	Реализовано
NFR-11	Хеширование паролей (bcrypt) в app_users	Реализовано
NFR-12	RBAC по ролям на каждый HTTP-метод и путь	Реализовано
NFR-13	Секрет JWT_SECRET общий для Users API и Aggregator	Реализовано
NFR-14	CORS allow_origins=["*"] на агрегаторе	Технический долг – для production нужен whitelist origin

Таблица 8 – Таблица Удобство использования (NFR-15)

ID	Требование
NFR-15	Сценарий «выбрал контейнер → сгенерировал конфиг → запустил экспортер → импортировал дашборд» доступен из UI без SSH на хост

Таблица 9 – Таблица Тестируемость (NFR-16)

ID	Требование
NFR-16	Автотесты pytest в каталоге tests/ (роутеры, Redis, MinIO, JWT, Celery,

	классификация)
--	----------------

Выводы по разделу

В данном разделе обоснована актуальность создания системы автоматизации мониторинга как средства борьбы с конфигурационной нагрузкой в динамических Docker-средах. Выявлено, что ручное управление мониторингом ведет к росту операционной рутины (toil), что противоречит принципам SRE.

Проведенный анализ аналогов (VictoriaMetrics, Grafana Alloy, Netdata, Checkmk) показал, что существующие инструменты либо требуют сложной предварительной настройки, либо не обеспечивают полного цикла автоматизации для централизованных систем мониторинга на базе Prometheus. На основании выявленного технологического разрыва сформулирована цель работы и определены ключевые задачи исследования и разработки. Завершает раздел формализованный перечень функциональных и нефункциональных требований, который станет основой для проектирования архитектуры разрабатываемого комплекса.

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1. Архитектура системы и обоснование технологического стека

На рисунке 5 продемонстрирована схема, отражающая логическую архитектуру проекта и взаимосвязи микросервисов

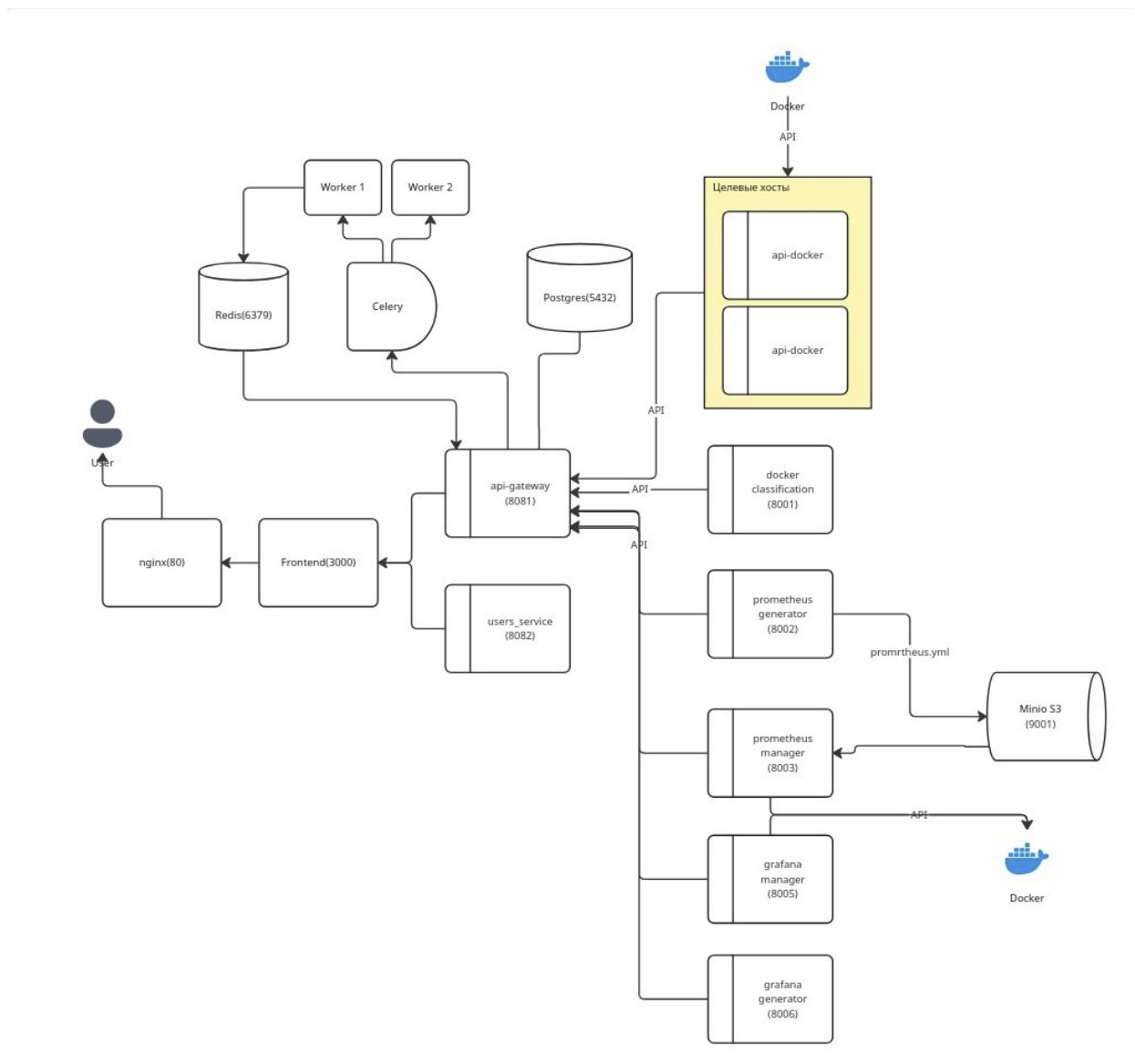


Рисунок 5 – Логическая архитектура

Подробно каждый компонент логической архитектуры описывается в разделах 2.1.2 — 2.1.7

2.1.1. Назначение и общая идея

Система решает задачу автоматизированного онбординга Docker-контейнеров в контур мониторинга. Оператор не пишет вручную `scrape_configs` для Prometheus и не

подбирает дашборды Grafana для каждого нового сервиса: платформа сама обнаруживает контейнеры, определяет технологический стек, генерирует конфигурационные артефакты, сохраняет их в хранилище и при необходимости применяет на хосте.

Логически архитектура строится вокруг трёх принципов:

1. Единая точка входа для клиента – все бизнес-операции идут через один публичный API (API Gateway), а не через десяток разрозненных URL микросервисов.
2. Разделение ответственности – каждый микросервис отвечает за узкую задачу (Docker, классификация, генерация YAML, управление Prometheus/Grafana).
3. Разделение данных по роли – оперативный снимок инфраструктуры в кэше, долговременные метаданные в СУБД, крупные YAML-файлы в объектном хранилище.

2.1.2. Уровень представления (клиент)

Клиентская часть – одностраничное приложение (SPA) на Vue 3 с TypeScript. Оно не содержит бизнес-логики доступа к Docker, MinIO или Prometheus: интерфейс только отображает данные и отправляет HTTP-запросы к backend.

Логически UI разбит на функциональные зоны, соответствующие предметной области:

- аутентификация – экран входа, хранение JWT в браузере;
- хосты – регистрация машин, с которых снимается инвентарь контейнеров;
- контейнеры – список, фильтрация, детали, действия по жизненному циклу (в пределах прав роли);
- Prometheus – просмотр и создание конфигураций, работа с менеджером;
- Grafana – шаблоны дашбордов, импорт, учёт импортированных панелей;
- Конфигурация – редактирование единого файла правил signatures.yml (для ролей с правами записи);
- пользователи – только для роли maintainer.

Навигация построена так, чтобы отражать типичный сценарий работы оператора: сначала хосты → затем контейнеры → затем настройка мониторинга для выбранного контейнера.

В production-сборке перед SPA стоит Nginx, который выполняет не только раздачу статики, но и логическое разделение API:

- запросы к /api/v1/auth/* и /api/v1/users направляются в сервис Users API;
- все остальные запросы с префиксом /api – в API Aggregator.

Для браузера это выглядит как один origin (один хост и порт), хотя за кулисами работают два независимых backend-приложения. Так упрощается конфигурация CORS и не нужно «размазывать» логику аутентификации по всем микросервисам.

2.1.3. Уровень безопасности и идентификации

Users API – отдельный логический компонент, отвечающий только за идентичность пользователя:

- проверка логина и пароля;
- выдача JWT (access token) с полями sub (имя пользователя) и role;
- CRUD учётных записей (доступен роли maintainer);
- эндпоинт «текущий пользователь» (/auth/me) для UI.

Пароли хранятся в PostgreSQL в таблице app_users в виде bcrypt-хеша, не в открытом виде. При первом запуске системы создаётся учётная запись maintainer из переменных окружения – это «корневой» администратор платформы.

Users API не знает о контейнерах, Prometheus или Grafana. Его зона ответственности ограничена аутентификацией и управлением пользователями.

API Aggregator не выдаёт токены, но проверяет каждый входящий запрос к бизнес-роутерам:

1. Извлекает JWT из заголовка Authorization: Bearer
2. Проверяет подпись тем же секретом JWT_SECRET, что и Users API.
3. Сопоставляет роль из токена с матрицей RBAC (maintainer, admin, dev, user).

Исключения из проверки: служебные пути (/health, /docs, корень API) и режим тестов с SKIP_JWT_AUTH.

Таким образом, логически безопасность разделена на аутентификацию (кто вы) – Users API, и авторизацию бизнес-операций (что вам можно) – Aggregator. Это классический паттерн: один сервис «логинит», другой «пускает к ресурсам».

2.1.4. Уровень приложения и оркестрации (ядро системы)

API Gateway – логическое ядро всей платформы. Именно он:

- реализует REST API, описанный в OpenAPI (/api/v1/hosts, /containers, /prometheus, /grafana);
- содержит бизнес-правила согласования данных между хранилищами;
- оркестрирует вызовы специализированных микросервисов;
- запускает фоновые задачи через Celery (описание ниже).

- Агрегатор не дублирует низкоуровневую работу с Docker daemon, не парсит PDF и не строит YAML для Prometheus самостоятельно - он делегирует это узким сервисам и собирает результат в единый ответ для клиента.

Внутри агрегатор структурирован по доменным роутерам как представлено в таблице 10.

Таблица 10 – Таблица структуры агрегатора

Роутер	Предметная область
hosts	Учёт целевых хостов, синхронизация их доступности
containers	Инвентарь контейнеров, операции Docker, обогащение данными мониторинга
prometheus	Генерация конфигов, экспортёры, main config, Prometheus Manager
grafana	Шаблоны, импорт дашбордов, Grafana Manager

Каждый роутер опирается на сервисный слой (работа с Redis, PostgreSQL, MinIO) и на шлюз HTTP-вызовов к микросервисам.

Для обращения к внутренним микросервисам используется единый класс APIGateway – тонкая обёртка над HTTP-клиентом с базовым URL сервиса. Логически это анти-коррупционный слой: роутеры агрегатора не зависят от деталей URL и формата ответов каждого микросервиса; все вызовы проходят через единый метод `make_request(method, path, ...)`. Такой подход упрощает сопровождение: при смене адреса сервиса меняется только переменная окружения (`DOCKER_API_URL`, `PROMETHEUS_GENERATION_URL` и т.д.), а не десятки мест в коде.

Логически выделяется асинхронный контур обновления данных (Celery), не привязанный к действиям пользователя в UI. Задачи, которые решает Celery представлены в таблице 11.

Таблица 11 – Таблица асинхронных задач Celery

Задача	Периодичность	Смысл
update_hosts	каждые 15 секунд	Прочитать хосты из PostgreSQL, проверить доступность Docker API, записать статус в Redis
update_containers	каждую минуту	Для каждого хоста: discover контейнеров → классификация → обновление Redis; удаление устаревших ключей

Celery использует Redis в двух ролях: логическая БД 1 – брокер очереди задач, логическая БД 2 – backend результатов. Это отделено от логической БД 0, где лежит кэш хостов и контейнеров для UI.

Фоновые воркеры – это тот же образ приложения, что и API Gateway, но с другой командой запуска (celery worker). Логически они – «второе лицо» ядра системы, работающее по расписанию.

2.1.5 Уровень доменных микросервисов

Ниже отображены специализированные компоненты, которые не видны браузеру напрямую (кроме Swagger при отладке). Агрегатор вызывает их по внутренней сети.

Первый компонент — Docker API, который выступает единственной точкой взаимодействия с Docker daemon на целевом хосте. Его функции включают discover (получение списка контейнеров и детальной информации inspect), manage (запуск, остановка и удаление контейнеров), а также pull and run (запуск sidcar-экспортёра в сети целевого контейнера). Логически Docker API не хранит состояние между запросами: каждый вызов обращается к актуальному состоянию daemon, а долговременный снимок инвентаря формирует агрегатор в Redis. Для удалённых хостов агрегатор строит URL вида `http://{host}:{port}`, где порт и адрес берутся из записи хоста в PostgreSQL.

Следующий компонент — Docker Classification, который по метаданным контейнера (labels, env, image, ports) определяет тип технологического стека (postgresql, redis, kafka и т.д.). Механизм работы основан на правилах из файла signatures.yml с весовыми коэффициентами: суммируются совпадения, возвращается ранжированный список технологий с оценками. Этот сервис является stateless: на вход принимает JSON с описанием контейнера, на выход отдаёт результат классификации. Он не обращается к базе данных и не знает о мониторинге — только определяет, «что это за софт». Результат классификации используется агрегатором при записи в Redis и при выборе шаблона экспортёра в Prometheus Generation.

Третий компонент — Prometheus Generation, который выполняет роль фабрики конфигураций. По типу стека и параметрам контейнера он генерирует YAML для Prometheus и сохраняет его в MinIO. Выходные артефакты включают scrape_config.yml (фрагмент job для Prometheus), {job_name}.yaml (список targets), а при работе с «главным» конфигом — mainConfig/prometheus.yml и target-файлы в каталоге mainConfig/targets/. Сервис читает правила экспортёров из того же файла signatures.yml (образ экспортёра, порт, переменные окружения, суффикс job).

Четвёртый компонент — Prometheus Manager, который управляет жизненным циклом Docker-контейнера Prometheus на машине оператора и синхронизирует конфиги из MinIO на локальную файловую систему, откуда Prometheus читает prometheus.yml и

targets/*.yaml. Логически это мост между объектным хранилищем (источник правды для YAML) и работающим процессом Prometheus (потребитель файлов на диске). Доступные операции: start, stop, restart, status, settings, а также config/update (скачать из MinIO, записать на хост, перезагрузить Prometheus).

Пятый компонент — Grafana Generation, который отвечает за импорт дашбордов. Он загружает шаблон дашборда (с grafana.com по grafana_dashboard_id из signatures.yml или по явному ID), подставляет datasource Prometheus и импортирует его через HTTP API в экземпляр Grafana оператора. Сервис работает с внешней Grafana как с отдельной системой; в MinIO дашборды не хранятся (кэш JSON может находиться локально на диске сервиса). Учёт успешного импорта (uid, url, связь с конфигом Prometheus) ведёт агрегатор в PostgreSQL в таблицу grafana_imported_dashboards.

Шестой компонент — Grafana Manager, который является аналогом Prometheus Manager, но для Docker-контейнера Grafana. Он выполняет операции start, stop, restart, status, но не импортирует дашборды — только поднимает и останавливает инфраструктуру визуализации.

2.1.6. Уровень данных (логическое разделение хранилищ)

Система использует три класса хранилищ с разными ролями.

Первое хранилище — PostgreSQL, которое служит источником структурированной правды о том, что было настроено для мониторинга и кто имеет доступ к системе. Логические области данных представлены в таблице 12.

Таблица 12 – Таблица логических областей данных

Область	Таблицы	Смысл
Инфраструктура	hosts	Зарегистрированные целевые хосты
Наблюдаемые объекты	containers	Контейнеры, для которых уже запускался сценарий генерации (с FK на конфиги)
Мониторинг Prometheus	prometheus_configs	Метаданные job: экспортёр, target, ссылка на MinIO
Визуализация	grafana_imported_dashboards	Учёт импортированных дашбордов
Безопасность	app_users	Пользователи и роли (ведёт Users API)

Связи с каскадным удалением: при удалении контейнера удаляются его конфигурации Prometheus; дашборд может быть привязан к конфигу опционально.

PostgreSQL не используется как кэш для быстрого списка всех контейнеров на всех хостах в реальном времени – для этого Redis.

Второе хранилище — Redis, которое обеспечивает быстрый снимок текущего состояния инфраструктуры для UI и фона. В логической БД 0 (кэш приложения) хранятся:

- `host:{id}` – JSON с именем, адресом, портом и статусом доступности (результат health-check);
- `container:{host_name}:{container_id}` – JSON с Docker inspect, результатом классификации, `host_name`.

TTL на ключи не ставится: актуальность обеспечивается периодическим перезаписыванием и diff-удалением исчезнувших контейнеров. Логические БД 1 и 2 отведены под инфраструктуру Celery (очередь и результаты задач) и не предназначены для чтения UI. Логически Redis здесь играет роль «оперативной модели» инвентаря, а PostgreSQL – «модели настроенного мониторинга».

Третье хранилище — MinIO (S3), которое выполняет роль репозитория конфигурационных артефактов для хранения крупных текстовых YAML, неудобных для размещения в строках PostgreSQL. Логическая структура (один bucket prometheus) включает:

- `configs/{container_id}/` – per-container scrape и targets;
- `mainConfig/` – общий prometheus.yml и каталог targets/ для агрегированных job.

Связь с PostgreSQL: в таблице prometheus_configs хранятся minio_bucket и minio_file_path – указатель на объекты, а не сами файлы. Важно отметить, что файл signatures.yml (правила классификации и шаблоны экспортёров) в MinIO не лежит – он монтируется в контейнеры как файл на диске; это конфигурация платформы, а не артефакт мониторинга конкретного контейнера.

2.1.7 Физическая архитектура (Docker Compose)

На этапе апробации все компоненты разворачиваются в одной bridge-сети app_network. Исключение: docker_api и менеджеры монтируют /var/run/docker.sock для управления контейнерами на хосте. Как в таблице 14.

Таблица 14 – Таблица развертки контейнеров

Контейнер	Порт на хосте	Назначение
frontend	3000	SPA + reverse proxy
api_aggregator	8081	Центральный API
users_api	8082	JWT и пользователи
docker_classification	8001	Классификация стека docker контейнеров
prometheus_generation	8002	Генерация YAML Prometheus
prometheus_manager	8003	Жизненный цикл Prometheus
docker_api	8004	Docker SDK
grafana_manager	8005	Жизненный цикл Grafana
grafana_generation	18010	Импорт дашбордов
minio	9002 / 9003	S3 API / Console
postgres, redis	internal	БД и кэш

2.2 Обоснование выбора технологий

2.2.1 Слой бэкенда и асинхронных задач

Выбор Python 3.11+ в качестве основного языка разработки продиктован его доминирующим положением в сфере автоматизации инфраструктуры и DevOps. Язык обладает зрелой экосистемой библиотек, включая *Docker SDK* для управления контейнерами, *boto3* для работы с объектными хранилищами и *PuYAML* для разбора конфигураций. Использование единого языка на всех микросервисах упрощает переиспользование кода и поддержку системы.

В качестве веб-фреймворка выбран FastAPI. Он обеспечивает высокую производительность за счет асинхронности (*async/await*) и встроенной валидации данных с помощью *Pydantic*, что минимизирует ошибки на стыке сервисов. Автоматическая генерация документации OpenAPI (*/docs*) значительно ускоряет интеграцию бэкенда с фронтендом.

Для выноса «тяжелых» и долгоиграющих операций (таких как периодический опрос (polling) Docker-хостов и контейнеров) из основного потока API используется Celery 5. Это гарантирует, что интерфейс останется отзывчивым, пока система выполняет фоновые задачи по сбору телеметрии

2.2.2 Слой хранения данных и кэширования

Реляционная СУБД PostgreSQL 15 выбрана в качестве главного хранилища метаданных благодаря строгому соблюдению принципов ACID, поддержке внешних ключей (FK) и каскадных удалений, что критично для поддержания целостности связей

между хостами и контейнерами. Важным фактором стала развитая поддержка типа JSONB: это позволяет гибко сохранять сырые данные инспекции контейнеров (`docker_info`) и динамические результаты классификации сервисов (`classification_score`) без постоянного изменения схемы таблиц. Для обеспечения высокой скорости работы и снижения нагрузки на базу данных развернут Redis 7. Он выполняет двойную роль: служит in-memory хранилищем для быстрого доступа к текущему инвентарю инфраструктуры, а также выступает в качестве отказоустойчивого брокера сообщений для Celery. Разделение этих функций изолировано на уровне логических баз данных (Logical DB).

Для работы с артефактами конфигураций мониторинга и резервными копиями применяется MinIO - легковесное S3-совместимое объектное хранилище. Оно позволяет развернуть полноценный аналог облачного хранилища локально на тестовом стенде, используя для взаимодействия стандартный протокол S3 и библиотеку *boto3*.

2.2.3 Фронтенд и клиентский слой

Интерфейс системы проектируется как одностраничное приложение (SPA) на связке Vue 3 и TypeScript. Vue 3 предоставляет легковесный и реактивный UI благодаря Composition API, а строгое использование TypeScript гарантирует типобезопасность на стороне клиента при обработке структуры данных, приходящих из REST API. В качестве инструмента сборки выбран Vite, который кардинально ускоряет процесс разработки за счет сверхбыстрого горячего перезапуска модулей (HMR), избавляя инженера от долгих ожиданий при изменении интерфейса.

2.2.4 Безопасность и доставка (Deployment)

Для разграничения доступа к API применяется механизм JWT-токенов с алгоритмом шифрования HS256. Это stateless-подход: агрегаторы и микросервисы могут самостоятельно проверять валидность подписи токена без выполнения постоянных запросов в Redis или базу данных для проверки сессий, что повышает общую производительность.

В production-сборке для маршрутизации трафика используется веб-сервер Nginx. Он работает в режиме Reverse proxy, раздавая статические файлы фронтенда и проксируя запросы на модули аутентификации (auth) и бизнес-логики (business API).

Вся инфраструктура проекта упакована в Docker и оркеструется через Docker Compose. Это обеспечивает полную воспроизводимость стенда: проект гарантированно

запустится с одинаковыми версиями окружения как на локальной машине разработчика, так и на демонстрационном или учебном сервере. Завершает стек связка Prometheus и Grafana - индустриальный стандарт для сбора метрик и визуализации состояния разработанного сервиса. Краткое обоснование и назначение выбранного стека представлено в таблице 15.

Таблица 15 – Таблица обоснования стека

Технология	Назначение	Обоснование выбора
Python 3.11+	Backend всех микросервисов	Единый язык, богатая экосистема для DevOps (Docker SDK, boto3, PyYAML)
FastAPI	REST API	Высокая производительность, автогенерация OpenAPI (/docs), валидация Pydantic
Vue 3 + TypeScript	SPA	Реактивный UI, типобезопасность при работе с REST
Vite	Сборка frontend	Быстрый HMR при разработке
PostgreSQL 15	Метаданные	ACID, JSON-поля для docker_info и classification_score, FK и каскады
Redis 7	Кэш + Celery	In-memory скорость для инвентаря; отдельные logical DB для broker/backend
MinIO	S3-совместимое хранилище	Локальный запуск без облака; протокол S3 для boto3
Celery 5	Фоновые задачи	Периодический poll хостов и контейнеров
Docker / Compose	Среда исполнения	Воспроизводимый стенд
Prometheus + Grafana	Стек observability	De-facto стандарт метрик и визуализации
JWT (HS256)	Аутентификация	Stateless-проверка на агрегаторе без сессий в Redis
Nginx	Reverse proxy в production-сборке frontend	Маршрутизация auth и business API

2.3 Роли пользователей и права доступа

В системе выделены четыре основные роли, каждая из которых обладает строго ограниченным набором привилегий:

- Maintainer (Главный администратор / Сопровождающий) - обладает максимальным уровнем доступа. Это единственная роль, имеющая право выполнять любые операции

(полный CRUD) в рамках Users API для управления пользователями, а также отправлять любые запросы (Все методы) к API aggregator для настройки глобальных параметров системы.

- Admin (Администратор) - сфокусирован на управлении инфраструктурой. Данная роль имеет неограниченный доступ ко всем методам и эндпоинтам API aggregator, что позволяет настраивать таргеты, управлять конфигурациями и просматривать логи, однако не имеет прав на изменение состава пользователей в Users API.
- Dev (Разработчик) - обладает промежуточным уровнем прав, необходимым для отладки и подключения новых сервисов. Роли доступны все безопасные запросы (GET), а также строго ограниченный набор деструктивных или модифицирующих методов (Ограниченные POST/PUT) на API aggregator.
- User (Пользователь / Наблюдатель) - наименее привилегированная роль, предназначенная для рядового мониторинга. Пользователю разрешены только безопасные запросы на чтение данных (Только GET) через API aggregator, что полностью исключает возможность изменения конфигурации или удаления сущностей.

Блок схема разграничения ролей и прав доступа представлена на рисунке 6.

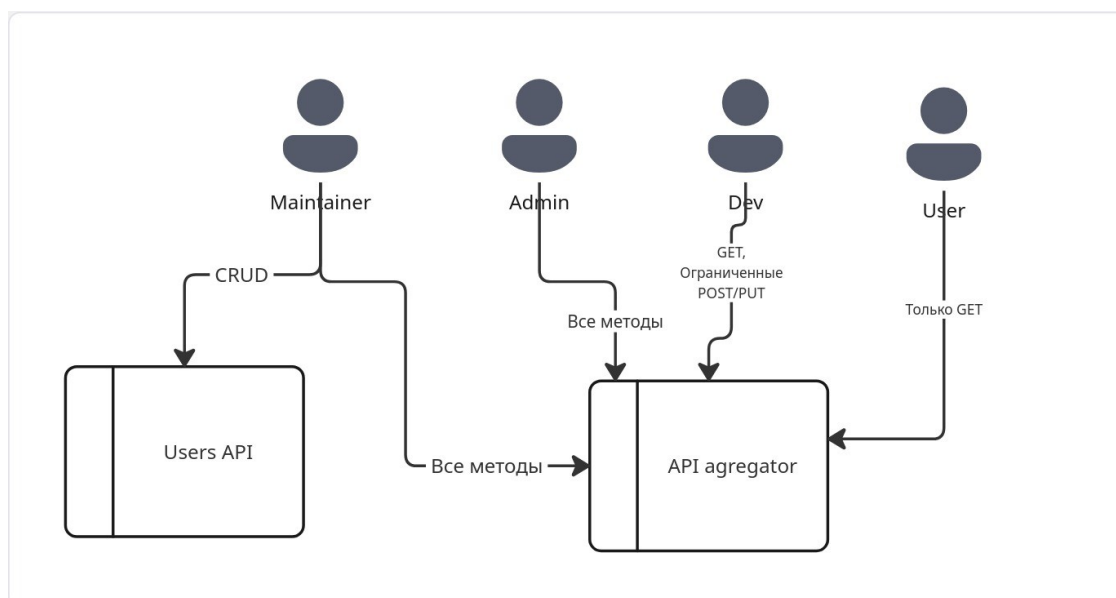


Рисунок 6 – Разграничение ролей и прав доступа

Детализация роли dev (запрещённые операции):

- PATCH /containers/update_containers;

- POST/DELETE управление контейнерами;
- POST /prometheus/generate_config, up_exporter, update_signature;
- все main_config и manager/* для Prometheus;
- PUT /grafana/templates.yml, import_dashboard, manager/*, DELETE imported_dashboards/*.

Frontend дублирует ограничения для UX (usePermissions.ts), но авторитетная проверка - на сервере.

2.4 Диаграммы вариантов использования (Use Case)

2.4.1 Общая диаграмма прецедентов

Для детального анализа функциональных требований к системе и определения границ взаимодействия пользователей с программным комплексом разработаны диаграммы вариантов использования (прецедентов). Данные модели позволяют наглядно отобразить полный спектр задач, которые решает каждая группа пользователей в процессе администрирования, настройки и мониторинга инфраструктуры.

Роль суперадминистратора (Maintainer) обладает максимально полным, сквозным функционалом в рамках всей системы. Данный субъект отвечает не только за управление ключевыми процессами автоматизации мониторинга (такими как генерация и применение конфигураций Prometheus, импорт готовых аналитических дашбордов в Grafana, управление хостами, контейнерами и запуск необходимых экспортеров), но и за глобальное администрирование самой платформы. Только этой роли доступны прецеденты управления учетными записями пользователей (регистрация, изменение прав, удаление), базовая аутентификация, тонкая настройка сигнатур классификации приложений в файле signatures.yml, а также запуск, остановка и управление жизненным циклом самих серверов Prometheus и Grafana. Полный состав функций суперадминистратора и его сценарии взаимодействия с системой представлены на рисунке 7.

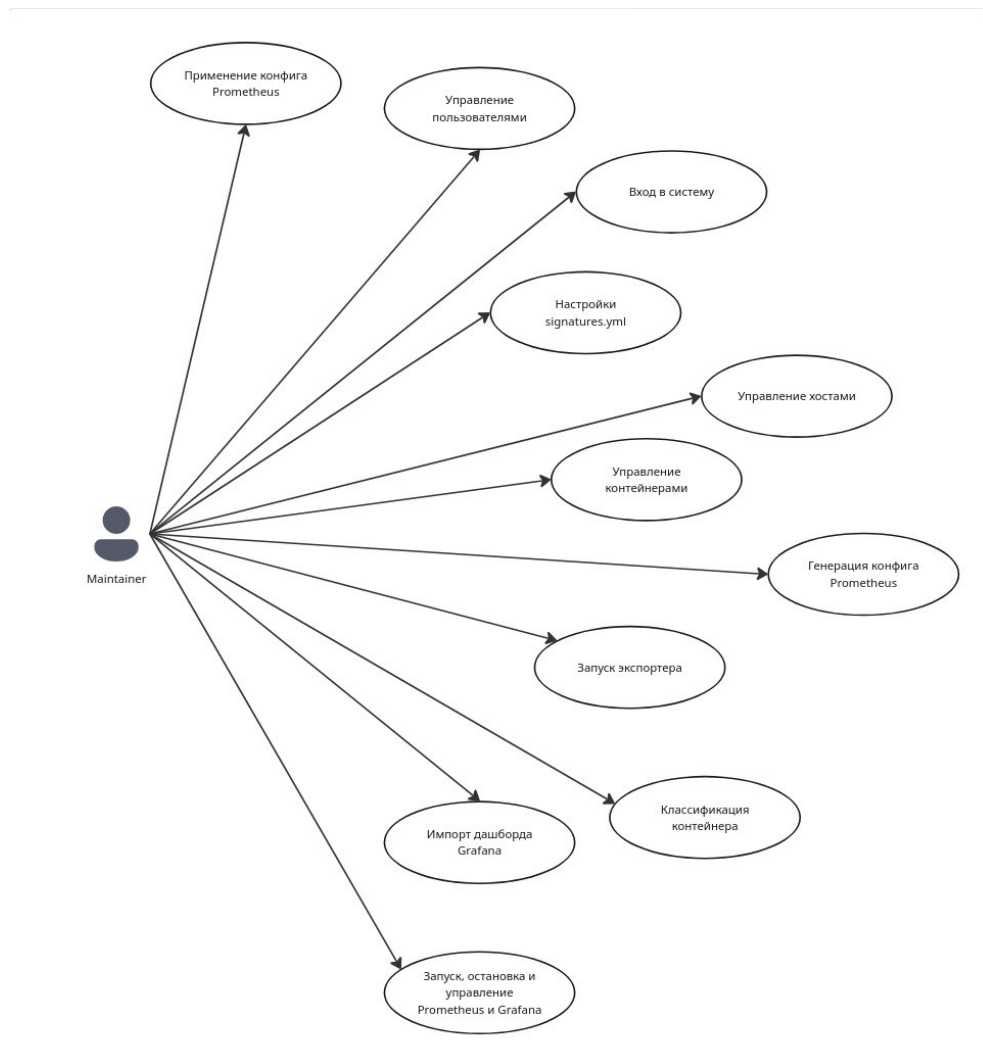


Рисунок 7 – Диаграмма прецедентов для роли Maintainer

2.5.2 Таблица прецедентов

Для детальной формализации требований к разрабатываемому программному комплексу и точного определения логики работы каждого компонента системы выполняется спецификация прецедентов (Use Cases). Каждому функциональному сценарию присваивается уникальный идентификатор, определяются иницилирующие его акторы, а также задаются жесткие рамки в виде предусловий (состояний системы, необходимых для успешного начала выполнения процесса) и конечного ожидаемого результата (изменений в базах данных, хранилищах или сессии пользователя). Спецификация основных вариантов использования, покрывающая процессы аутентификации, инвентаризации инфраструктуры, генерации конфигураций и оркестрации observability-стека, приведена в таблице 16.

Таблица 16 – Таблица прецедентов

ID	Прецедент	Актор	Предусловие	Результат
UC-01	Аутентификация	Все	Учётная запись в app_users	JWT в localStorage
UC-02	Зарегистрировать хост	admin, maintainer,	JWT	Запись в hosts + Redis host: {id}
UC-03	Обновить инвентарь контейнеров	admin, maintainer	Хост доступен	Ключи container:{host}:{id} в Redis
UC-04	Просмотреть контейнеры	Все (GET)	–	Список из Redis
UC-05	Сгенерировать мониторинг	admin, maintainer, dev	Контейнер классифицирован	MinIO + prometheus_configs
UC-06	Импортировать дашборд	admin, maintainer, dev	Есть grafana_dashboard_id в signatures	Запись в grafana_imported_dashboards
UC-07	Управлять пользователями	maintainer	–	

2.6 Потоки данных

2.6.1 Сквозной поток: онбординг контейнера

Онбординг контейнера в мониторинг – это сквозной сценарий, в котором участвуют UI, API Aggregator, Redis, два микросервиса (Docker и классификация), генератор Prometheus, объектное хранилище MinIO, PostgreSQL, Prometheus Manager и сам Docker Engine.

Первый этап – актуализация инвентаря. Оператор в интерфейсе инициирует обновление списка контейнеров (например, запрос PATCH .../update_containers). Агрегатор для каждого зарегистрированного хоста обращается к сервису docker_api методом discover: тот на стороне целевой машины выполняет аналог docker ps с последующим inspect для каждого контейнера. В ответ агрегатор получает сырой набор метаданных (образ, состояние, сеть, переменные окружения, метки и т.д.).

Второй этап: классификация в цикле по контейнерам. Для каждого контейнера агрегатор передаёт собранное описание в docker_classification. Сервис сопоставляет признаки с правилами из signatures.yml и возвращает стек (например, postgresql) и оценки

(scores) по кандидатам. Агрегатор объединяет это с идентификатором хоста и записывает результат в Redis логической БД 0 по ключу вида

`container:{host_name}:{container_id}` – так формируется оперативный снимок, из которого потом UI и генерация берут данные без повторного опроса Docker.

Третий этап: генерация конфигурации мониторинга. По команде оператора (POST `.../generate_config`) агрегатор читает из Redis сведения о нужном контейнере (стек, сеть, порты и пр.) и вызывает `prometheus_generation`. Тот строит YAML фрагменты для Prometheus и выкладывает их в MinIO (операции уровня S3 – условно «PUT» объектов под префиксом `configs/{container_id}/`). В ответ агрегатор получает имя бакета и пути к файлам. После успешной загрузки в MinIO агрегатор фиксирует результат в PostgreSQL: создаёт или обновляет записи таблиц `containers` и `prometheus_configs`, связывая метаданные конфигурации с идентификатором контейнера и ссылками на объекты в хранилище.

Четвёртый этап: запуск экспортера. Оператор запускает подъём метрик (POST `.../up_exporter`). Агрегатор снова обращается к `docker_api`, который выполняет `pull_and_run` образа экспортера и подключает контейнер к той же Docker-сети, что и целевой сервис, чтобы Prometheus мог достигать до `endpoint`'а метрик.

Пятый этап – применение конфигурации на хосте. Запрос к Prometheus Manager (POST `.../manager/config/update`) инициирует синхронизацию из MinIO на файловую систему хоста: менеджер читает объекты из бакета и раскладывает их в каталог, откуда работает контейнер Prometheus, после чего перезагружает (или перезапускает) Prometheus, чтобы он подхватил новые `scrape_config` и `targets`.

В сумме поток превращает «контейнер в Docker» в наблюдаемый объект: инвентарь и классификация в Redis, долговременный учёт настроек в PostgreSQL, файлы сбора метрик в MinIO и работающий экспортер плюс актуальный Prometheus на хосте. Потоки данных, демонстрирующие все эти процессы отображены на рисунке 10.

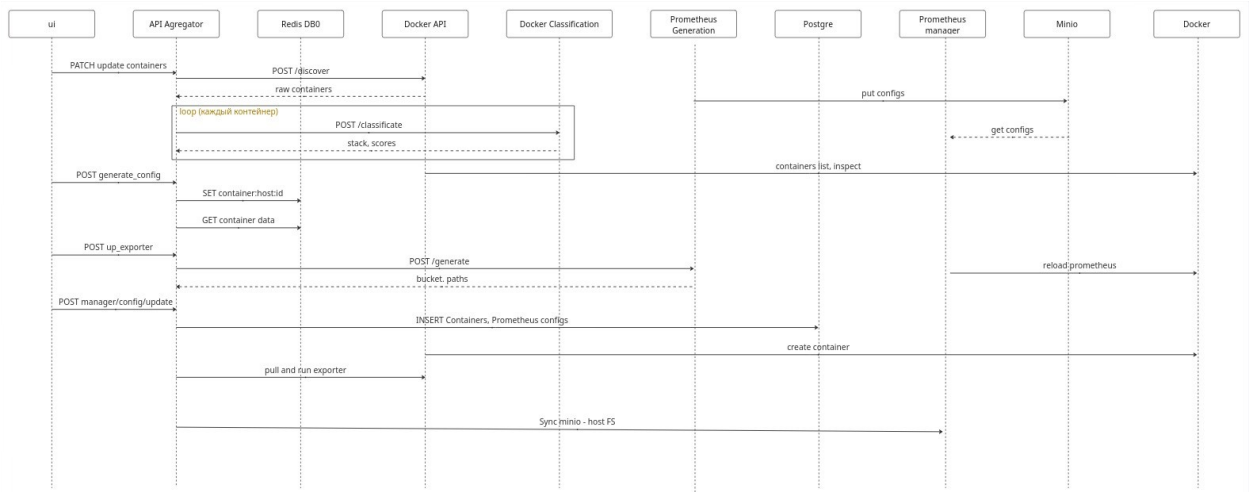


Рисунок 8 – Онбординг контейнера

2.6.2 Поток аутентификации

Аутентификация отделена от бизнес-API: браузер сначала обращается к Users API, а уже затем к API Aggregator с тем же идентификатором сессии в виде JWT.

Пользователь вводит логин и пароль; Frontend отправляет POST /api/v1/auth/login с телом {username, password}. Запрос по конфигурации прокси попадает в Users API. Сервис выполняет запрос к PostgreSQL, таблица app_users: проверяет наличие записи, активность учётной записи и корректность пароля (сравнение с хешем, обычно bcrypt). При успехе формируется access_token (JWT) с полями, достаточными для идентификации пользователя и его роли, задаётся время жизни токена.

Ответ {access_token, role, ...} приходит во Frontend; токен сохраняется, как правило, в localStorage (или аналоге), чтобы последующие запросы к агрегатору могли автоматически добавлять заголовок Authorization: Bearer <token>.

Дальнейшие обращения, например GET /api/v1/containers, идут уже на API Aggregator. Middleware извлекает JWT, проверяет подпись тем же секретом, что и при выдаче токена, достаёт роль и пропускает запрос к обработчику только если токен валиден и не истёк. Дополнительно срабатывает RBAC: для каждого HTTP-метода и пути решается, разрешена ли операция данной роли (maintainer, admin, dev, user). При успешной проверке агрегатор возвращает 200 с JSON-телом; при отказе – код ошибки доступа без раскрытия деталей проверки пароля.

Итог: Users API отвечает за «кто вы» и выдачу учётных данных в форме JWT; API Aggregator отвечает за «что вам можно» на каждом бизнес-запросе; PostgreSQL хранит только то, что нужно для проверки пароля и роли при входе. Подробно данный поток отображен на рисунке 11.

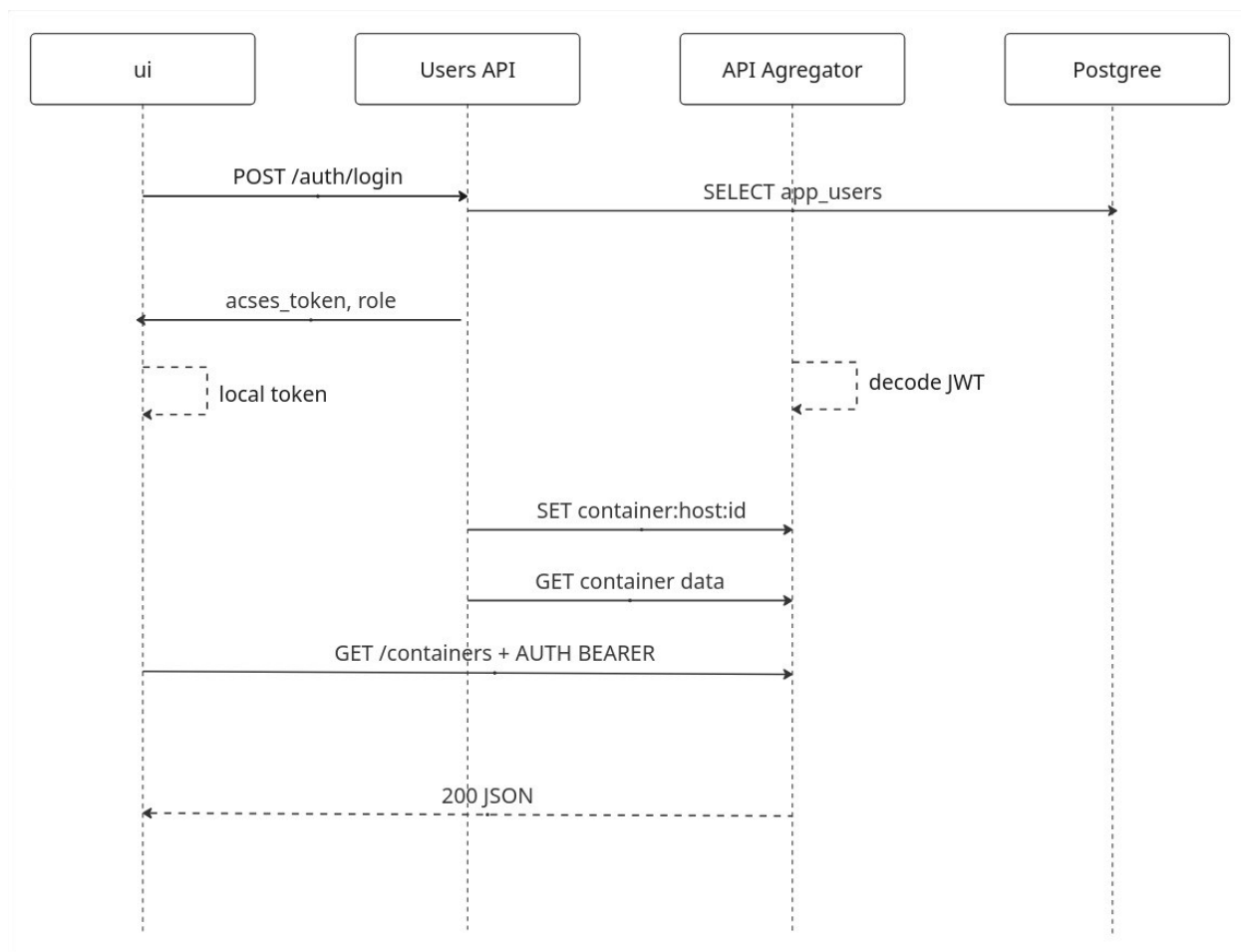


Рисунок 9 – Поток аутентификации

2.6.3 Фоновый поток Celery

Помимо действий оператора система поддерживает автоматическое обновление состояния инфраструктуры с помощью Celery Beat (планировщик) и воркеров Celery.

Задача обновления хостов запускается каждые 15 секунд, обновление контейнеров – каждую минуту. Очередь и результаты задач обслуживаются через отдельные логические базы Redis (брокер и backend), не смешиваясь с ключами host:* и container:* в БД 0.

В задаче update_hosts воркер читает из PostgreSQL таблицу hosts – список целевых машин и параметров подключения к Docker API. Для каждого хоста выполняется проверка доступности (health docker_api). Результат агрегируется и записывается в Redis по ключам host:{host_id}: помимо адреса и порта сохраняется статус (успех, таймаут, «хост недоступен» и т.п.). Благодаря этому UI может показывать актуальную «живость» хостов без ручного обновления.

В задаче update_containers воркер снова опирается на список хостов, для каждого вызывает docker_api (discover), получает список контейнеров с inspect, затем для

контейнеров выполняет классификацию через `docker_classification`. Итог по каждому экземпляру записывается в Redis ключом `container:{host_name}:{container_id}`. При изменении состава контейнеров на хосте устаревшие ключи удаляются (логика согласования «список в Redis» со «списком из Docker»), чтобы кэш не содержал исчезнувших контейнеров.

Смысл фонового контура заключается в том, чтобы разгрузить оператора от постоянного нажатия «обновить» и держать в Redis свежее представление хостов и контейнеров, в то время как тяжёлые операции онбординга (генерация конфигов, MinIO, PostgreSQL) по-прежнему привязаны к явным действиям или отдельным сценариям после классификации. Работа контура продемонстрирована на рисунке 12.

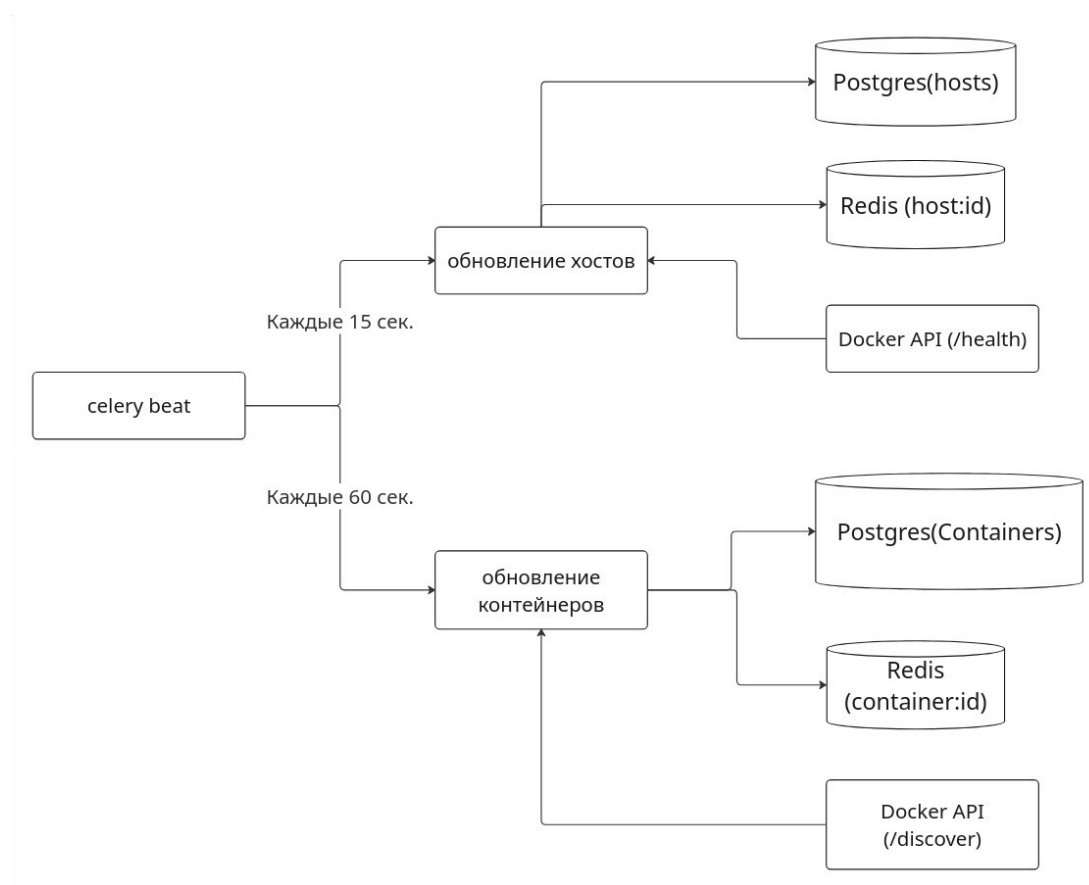


Рисунок 10 – Работа фоновых задач Celery

2.7 Проектирование базы данных PostgreSQL

Проектирование выполнено в три этапа: концептуальная → логическая → физическая модель.

2.7.1 Концептуальная модель

Концептуальная модель базы данных представлена на рисунке 13.

Сущности предметной области:

- Целевой хост – узел с Docker API (name, host, port).
- Контейнер – объект наблюдения (образ, статус, стек, результаты классификации, docker_info).
- Конфигурация Prometheus – настройка сбора метрик для контейнера (экспортёр, job, ссылка на MinIO).
- Импортированный дашборд Grafana – учётная запись после импорта (uid, url, связь с конфигом).
- Пользователь – учётная запись для входа (роль, хеш пароля).

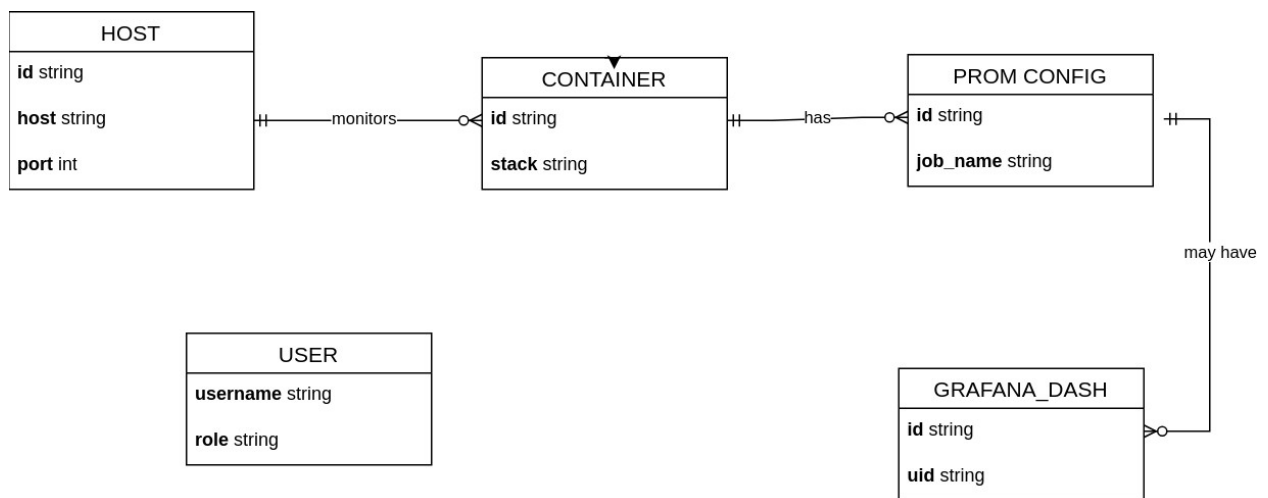


Рисунок 11 – Концептуальная модель

Примечание: связь Host–Container на концептуальном уровне логическая (через Redis `host_name`); в PostgreSQL отдельного FK «контейнер → хост» нет.

2.7.2 Логическая модель (таблицы и связи)

Для организации надежного и структурированного хранения данных в реляционной СУБД была спроектирована логическая схема базы данных. Она определяет структуру таблиц, типы полей, ограничения целостности, а также логические взаимосвязи между сущностями, обеспечивающими работу контура инвентаризации, классификации и генерации артефактов мониторинга.

В основе логической модели лежат следующие ключевые сущности:

- **hosts (Хосты)** - содержит информацию о целевых серверах, на которых развернут Docker. Хранит сетевые реквизиты доступа (`host`, `port`) и понятное имя хоста (`name`), выступая отправной точкой для инвентаризации.
- **containers (Контейнеры)** - аккумулирует метаданные о запущенных в инфраструктуре контейнерах. Для гибкости и масштабируемости системы сложные динамические

структуры данных, такие как сырая информация инспекции движка Docker и промежуточные веса классификатора, вынесены в специализированные поля `docker_info` и `classification_score` с типом данных `json`.

- `prometheus_configs` (Конфигурации Prometheus) - центральная сущность процесса автоматизации, хранящая сгенерированные параметры для агентов сбора метрик. Она напрямую связана с таблицей контейнеров отношением «один ко многим» по внешнему ключу `container_id`. Для обеспечения ссылочной целостности базы данных на эту связь наложено ограничение `ON DELETE CASCADE` (каскадное удаление): при удалении записи о контейнере все связанные с ним файлы конфигураций и метаданные зачищаются автоматически. Таблица также содержит пути к физическим файлам в объектном хранилище (`minio_bucket`, `minio_file_path`) и версионирование.
- `grafana_imported_dashboards` (Импортированные дашборды) - отвечает за хранение метаданных аналитических панелей, развернутых в Grafana. Данная сущность связана с таблицей конфигураций Prometheus (`prometheus_config_id`), что позволяет точно определять, для какого именно набора метрик и таргета был импортирован конкретный дашборд. Уникальный идентификатор `uid` снабжен ограничением уникальности (UK) для предотвращения дублирования панелей.
- `app_users` (Пользователи приложения) - изолированная таблица, обеспечивающая функционирование подсистемы аутентификации и авторизации (RBAC). Хранит хэшированные пароли (`password_hash`), уникальные имена пользователей (`username`), а также текущие роли (`role`) для разграничения прав доступа к API на бэкенде.

Визуальное представление спроектированной структуры базы данных, включая типы данных, первичные (PK) и внешние (FK) ключи, а также характер связей между таблицами, приведено на рисунке 14.

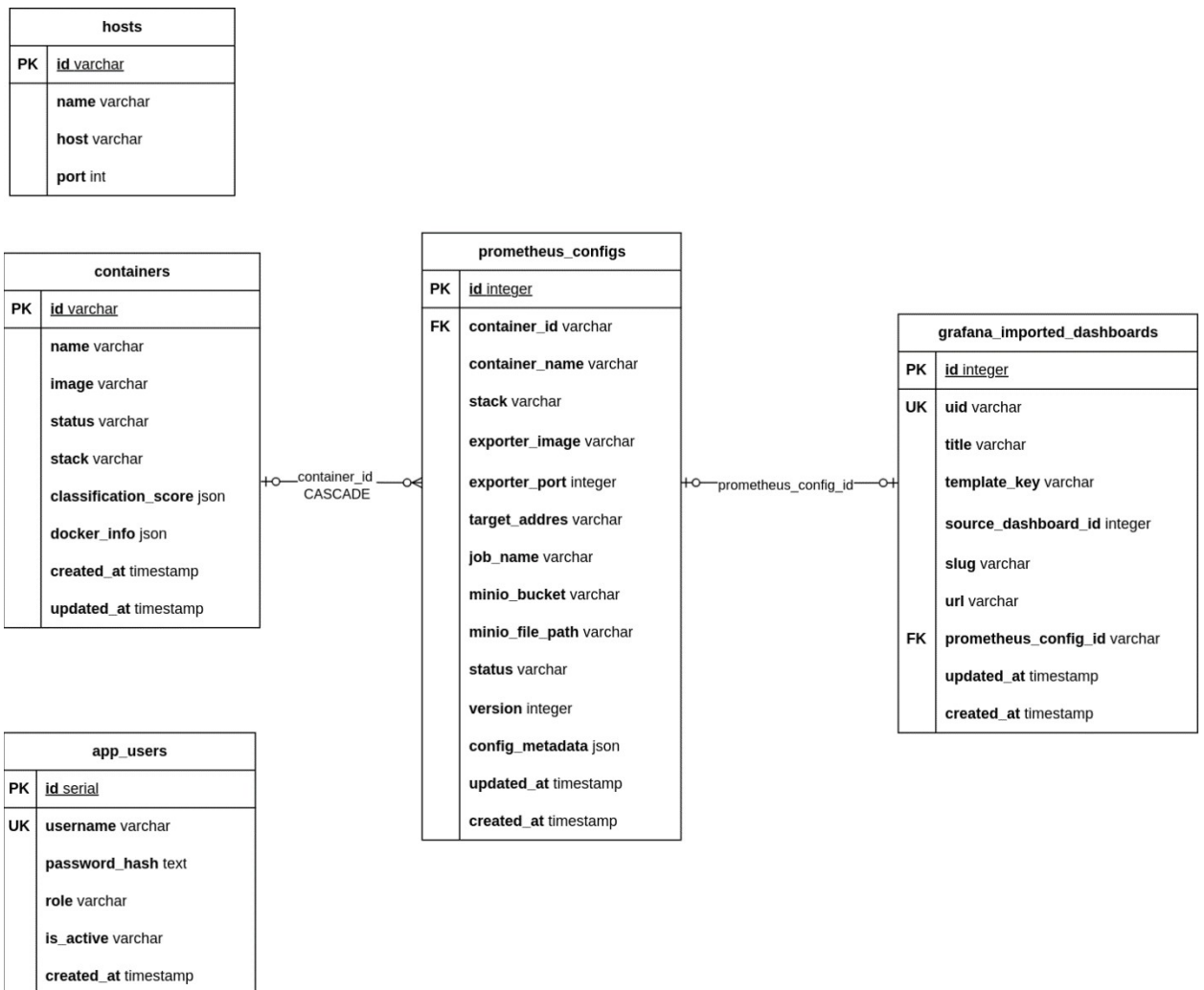


Рисунок 12 – Логическая модель бд

Ограничения целостности:

- `prometheus_configs.container_id` → `containers.id` ON DELETE CASCADE;
- `grafana_imported_dashboards.uid` – UNIQUE;
- partial unique: не более одного дашборда на `prometheus_config_id` (если NOT NULL).

2.7.3 Физическая модель (типы и индексы)

Физическая модель базы данных транслирует логическую схему в конкретные типы PostgreSQL с оптимизацией запросов через индексы. Таблица `hosts` содержит идентификатор (UUID), имя, адрес и порт подключения. Таблица `containers` хранит `name`, `image`, `status`, `stack`, а также два JSONB-поля `docker_info` и `classification_score`. Поля `name`, `status` и `stack` снабжены B-tree индексами. Таблица `prometheus_configs` связана с `containers` внешним ключом с каскадным удалением и включает версионирование `config_metadata`. Таблица `grafana_imported_dashboards` содержит `uid`, `template_key` и опциональную связь с `prometheus_config_id`. Таблица `app_users` (управляется Users API) хранит `username`, `bcrypt-`

хеш пароля, роль и флаг активности. Детальная спецификация полей, типов и индексов приведена в приложении Б (таблицы Б.1–Б.5).

2.8 Схема данных в Redis

2.8.1 Разделение logical databases

Для изоляции различных типов системного трафика и исключения взаимного влияния процессов кэширования и очередей задач используется разделение данных на уровне логических баз (Logical Databases) в рамках одного инстанса Redis. Такое решение позволяет разграничить оперативную память под нужды хранения инвентаря инфраструктуры, брокера сообщений и хранилища результатов выполнения фоновых воркеров, как показано в таблице 22.

Таблица 22 – Разделение логических баз данных Redis

Redis DB	URL (пример)	Назначение
0	redis://redis:6379/0	Кэш приложения
1	.../1	Celery broker
2	.../2	Celery result backend

2.8.2 Ключи приложения (DB 0)

Ключ `host:{host_id}` Для оперативного контроля состояния инфраструктуры в базе данных DB 0 создаются строковые ключи с префиксом `host:`, где в качестве идентификатора выступает UUID целевого хоста. Значением ключа является JSON-документ, содержащий параметры подключения к Docker API и текущий статус сетевой доступности ноды. Запись не имеет жесткого времени жизни (TTL), а её актуальность поддерживается за счет регулярного обновления каждые 15 секунд планировщиком Celery или принудительно через GET-запрос, как показано в таблице 23.

Таблица 23 – Структура JSON-объекта для ключа `host:{host_id}`

Поле JSON	Тип	Описание
name	string	Отображаемое имя
host	string	IP/hostname Docker API
port	int	Порт Docker API
status	string	HTTP-код health-check, down, timeout, local_only

Ключ `container:{host_name}:{container_id}` Информация о запущенных контейнерах кэшируется в виде составных ключей, включающих имя хоста и уникальный идентификатор контейнера. В структуре хранится как сырой вывод инспекции Docker, так и аналитические метки классификатора для последующей генерации мониторинга. Ключ сохраняется без TTL, а синхронизация данных происходит методом вычисления разности (diff): во время очередного опроса хоста бэкенд сравнивает текущий список контейнеров с кэшем и принудительно удаляет ключи тех объектов, которые были остановлены или уничтожены, как показано в таблице 24.

Таблица 24 – Структура JSON-объекта для ключа `container:{host_name}:{container_id}`

Поле JSON	Тип	Описание
info	object	Docker inspect (сырые метаданные)
classification	object	Результат docker_classification
host_id	string	UUID хоста из PostgreSQL
host_name	string	Дублируется для фильтрации

TTL: нет. При poll выполняется diff: ключи отсутствующих контейнеров удаляются.

Взаимодействие распределенной очереди задач Celery с Redis опирается на стандартные механизмы брокера. В логической базе DB 1 разворачиваются служебные структуры (списки и очереди) для диспетчеризации фоновых задач `update_containers` и `update_hosts`. Метаданные о статусе выполнения данных тасок, логи ошибок и возвращаемые значения складываются в базу DB 2, выступающую в роли выделенного backend-хранилища результатов.

2.9 Схема данных в MinIO (S3)

2.9.1 Общие параметры

Для организации надежного долговременного хранения файлов конфигураций и исключения их потери при перезапуске контейнеров бэкенда в архитектуру системы интегрировано S3-совместимое объектное хранилище MinIO. Взаимодействие с ним со стороны микросервисов осуществляется асинхронно через официальный клиентский SDK. Все генерируемые артефакты автоматически помещаются в выделенный изолированный

сектор (бакет), при этом каждый объект снабжается метаданными для контроля версий и верификации источника генерации, как показано в таблице 25.

Таблица 25 – Общие параметры объектного хранилища MinIO

Параметр	Значение
Протокол	S3 API (boto3, signature v4)
Endpoint	MINIO_ENDPOINT (напр. http://minio:9000)
Bucket	prometheus (создаётся автоматически)
Content-Type	application/x-yaml
Метаданные объекта	format=yaml, generated-by=prometheus-generation, schema-version=1.0

В MinIO не хранятся: `signatures.yml` (файл на диске, `bind-mount`), кэш JSON дашбордов Grafana (локальная папка `.dashboard_cache` в `grafana_generation`).

2.9.2 Иерархия объектов

Размещение и организация хранения файлов конфигураций внутри объектного хранилища MinIO подчиняются строгой иерархической структуре. Поскольку S3-совместимые хранилища используют плоскую структуру данных, логическое разделение на каталоги реализуется с помощью префиксов в именах ключей (объектов). Это позволяет изолировать конфигурационные артефакты разных контейнеров друг от друга и централизованно поставлять их компонентам мониторинга.

Внутри единого выделенного бакета `prometheus` для каждого классифицированного контейнера создается персональная виртуальная директория, имеющая вид `configs/{container_id}/`. Внутри этого префикса файлы распределяются по следующим целевым назначениям:

- Конфигурация сбора метрик (Scrape Configs) - в рамках отдельной виртуальной папки контейнера формируется файл `scrape_config.yml`, определяющий базовые параметры сбора. Рядом с ним генерируется файл `job_name.yml` targets array, содержащий конкретный массив статических или динамических адресов (таргетов), которые Prometheus должен опрашивать для данной задачи.
- Глобальные конфигурации инстанса - для обеспечения работы серверов мониторинга в иерархии предусмотрено создание основного управляющего файла `prometheus.yml`. Вспомогательные файлы описания конкретных работ (jobs) выносятся во вложенный подкаталог `targets/`, где они структурируются в виде отдельных манифестов формата YAML (например, `job_name.yml`).

Такой подход к организации пространства хранения позволяет агенту сбора данных (vmagent или Prometheus) точно запрашивать конфигурационные файлы конкретного контейнера по его уникальному идентификатору, не выкачивая весь массив данных из хранилища. Логическая схема и внутренняя иерархия распределения объектов внутри бакета мониторинга представлена на рисунке 15.

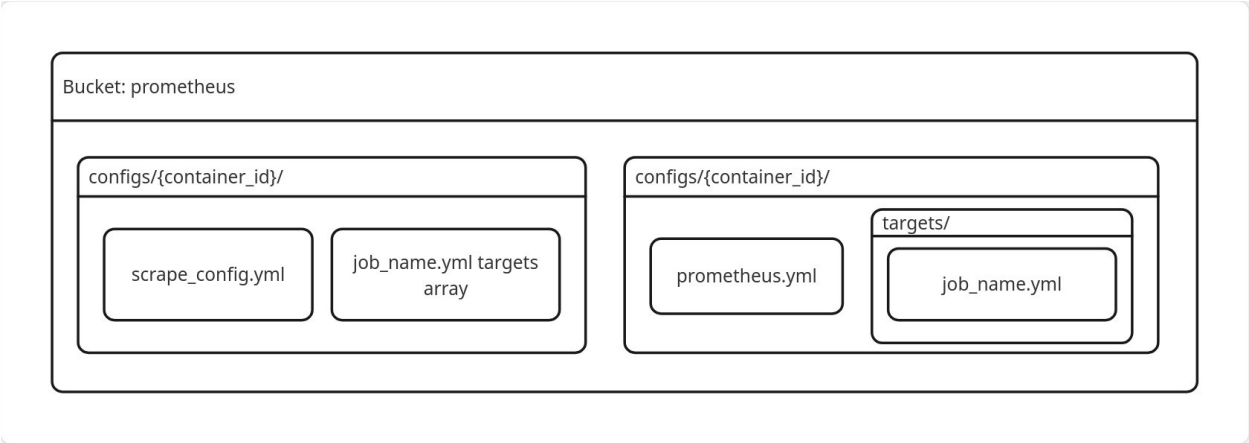


Рисунок 13 – Иерархия объектов и структура хранения конфигураций в MinIO

2.9.3 Таблица объектов

Для точного сопоставления генерируемых файлов с их местоположением в MinIO используется строгая структура ключей объектов (Object Keys). Каждый файл имеет свое функциональное назначение и привязан к конкретному событию в жизненном цикле приложения, будь то первичная конфигурация отдельного контейнера или сборка глобального файла конфигурации для центрального сервера сбора метрик. Полный перечень хранящихся объектов, их структура и триггеры создания приведены в таблице 26.

Таблица 26 – Матрица объектов и конфигурационных файлов в MinIO

Object Key	Содержимое	Когда создаётся
configs/{container_id}/ scrape_config.yml	{ scrape_configs: [{...}] }	POST /generate для контейнера
configs/{container_id}/ {job_name}.yml	YAML-массив targets	вместе со scrape_config
mainConfig/prometheus.yml	global + scrape_configs	main_config/add
mainConfig/targets/ {job_name}.yml	targets для job в main config	main_config/add

Связь с PostgreSQL: Логическая привязка физических объектов из MinIO к реляционной базе данных осуществляется через таблицу `prometheus_configs`. В ней жестко фиксируются атрибуты `minio_bucket='prometheus'` и `minio_file_path='configs/{container_id}'`. Данный путь используется микросервисами в качестве базового префикса для быстрого выполнения операций поиска (`list`) и скачивания (`get`) всех связанных файлов конфигурации конкретного контейнера.

2.10 Бизнес-логика

2.10.1 Классификация контейнеров

Процесс автоматизации мониторинга начинается с идентификации запущенного в контейнере программного стека. Специализированный внутренний сервис `docker_classification` выполняет комплексный статический анализ метаданных контейнера, собирая информацию из четырех независимых источников:

- Установленные на контейнере метки (`labels`);
- Переменные окружения (`environment variables`);
- Полное имя и тег используемого Docker-образа (`image name / tag`);
- Список открытых сетевых портов (`exposed ports`).

Полученные данные сопоставляются с эталонными правилами, описанными в конфигурационном файле `signatures.yml`, где каждому совпадению присваиваются определенные коэффициенты уверенности (веса `weight`). По результатам математического расчета сервис возвращает отсортированный по убыванию список вероятных технологических стеков с их финальными оценками. Стек, набравший максимальный балл, признается системой целевым и автоматически записывается в базу данных PostgreSQL в поля `stack` и `classification_score` карточки контейнера.

2.10.2 Генерация мониторинга

После того как контейнер успешно классифицирован, система переходит к этапу автоматического формирования инфраструктуры сбора метрик. Используя определенный на предыдущем шаге ключ `stack`, бэкенд обращается к файлу `signatures.yml` (расположенному в корне репозитория) и считывает технологическую карту для развертывания мониторинга, которая включает в себя:

- Параметры запуска совместимого экспортера (`exporter_image`, `exporter_port`, `metrics_path`);

- Набор необходимых переменных окружения или шаблонов для их подстановки (env_vars / env_template);
- Идентификатор эталонной аналитической панели (grafana_dashboard_id), который затем передается в микросервис Grafana Generation для автоматического импорта и отрисовки графиков.

2.10.3 Модель IDEF0 (контекст)

Для комплексного описания бизнес-логики и верхнеуровневого анализа процессов внутри разрабатываемой системы применяется методология функционального моделирования IDEF0. Контекстная диаграмма представляет систему в виде единого функционального блока, позволяя четко определить её границы, материальные и информационные потоки, а также управляющие регламенты и исполнительные механизмы.

Центральный блок «A0: Автоматизировать мониторинг Docker» агрегирует в себе ключевую задачу программного комплекса и взаимодействует с внешней средой через следующие типы потоков:

- Входы (Inputs) - исходные данные, детерминирующие начало работы алгоритмов. К ним относятся сетевые реквизиты и метаданные целевых хостов Docker API, а также эталонный файл конфигурации сигнатур signatures.yml, на основании которого система производит распознавание технологических стеков.
- Управление (Controls) - нормативно-технические ограничения и правила, регламентирующие выполнение функции. Верхний контур управления представлен правилами SRE (инженерными практиками обеспечения надежности) и ролевой моделью доступа на базе JWT-токенов (JWT RBAC), которая отсекает неавторизованные запросы к API.
- Механизмы (Mechanisms) - ресурсы, за счет которых физически реализуется функциональный блок. В качестве исполнителей выступают разработанные микросервисы бэкенда, а также технологический стек хранения данных, состоящий из СУБД PostgreSQL, кэша Redis и объектного S3-хранилища MinIO.
- Выходы (Outputs) - конечные материальные или информационные артефакты, получаемые на выходе из системы. Результатом автоматизации являются готовые файлы конфигурации Prometheus, автоматически импортированные интерактивные

дашборды Графана, а также актуальный инвентарь инфраструктуры, отображаемый в веб-интерфейсе пользователя (UI).

Данная функциональная структура позволяет наглядно оценить взаимосвязь между внутренними компонентами платформы и внешними сущностями. Общая контекстная диаграмма верхнего уровня для разрабатываемого решения представлена на рисунке 16.

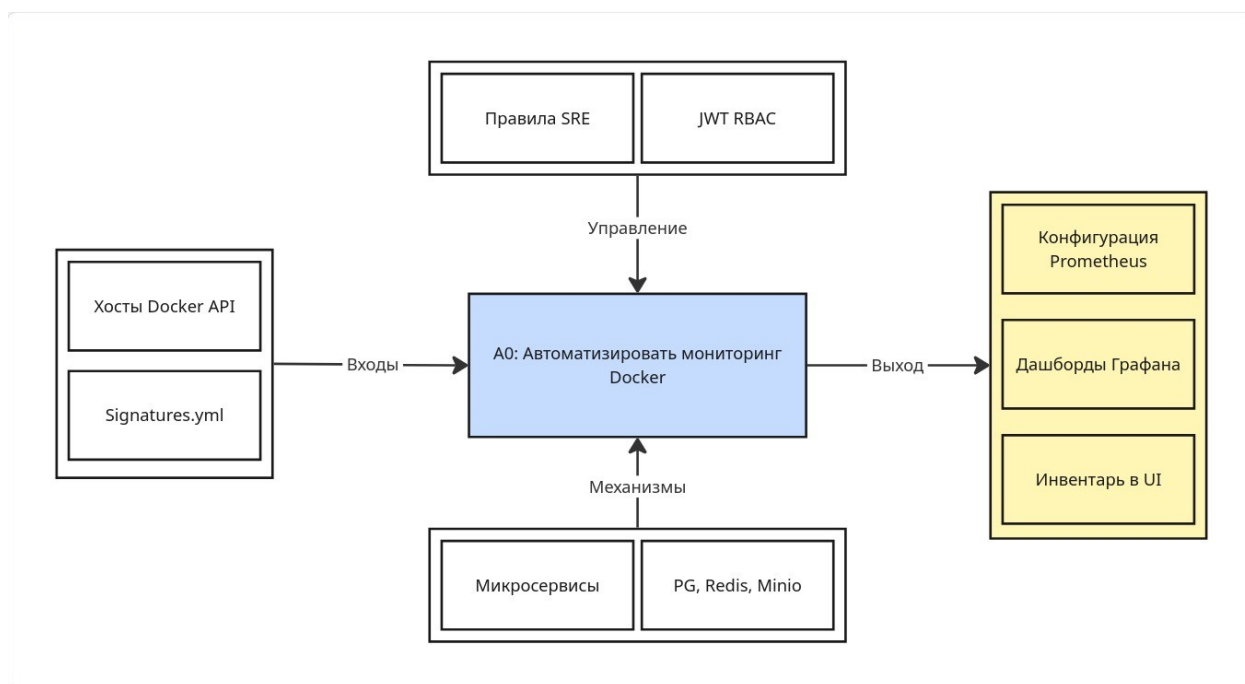


Рисунок 14 – Контекстная диаграмма IDEF0 процесса автоматизации мониторинга Docker

2.11 Пользовательский интерфейс

2.11.1 Карта экранов

Для верхнеуровневого проектирования фронтенд-части веб-приложения и настройки клиентской маршрутизации (routing) была разработана карта экранов. Она определяет логическую структуру интерфейса, вложенность страниц, а также пути навигации пользователя после прохождения процедуры авторизации.

Точкой входа в веб-интерфейс является изолированная страница авторизации /login, защищающая внутренний контур от несанкционированного доступа. После успешной валидации учетных данных и сохранения JWT-токена в браузере, пользователю открывается доступ к основному навигационному меню, которое разделено на шесть базовых разделов:

- `/hosts` (Управление хостами) - корневой раздел для регистрации, проверки статуса и просмотра списков подключенных серверов Docker API.
- `/containers` (Инвентарь контейнеров) - интерактивная таблица всех обнаруженных в инфраструктуре контейнеров. Из неё пользователь может перейти на детальную страницу конкретного объекта - `/containers/{container_id}` - для просмотра результатов классификации и сырых метаданных `docker_info`.
- `/prometheus` (Мониторинг Prometheus) - специализированный интерфейс для контроля сгенерированных конфигурационных файлов. Из этого раздела реализован прямой переход на внешнюю или встроенную веб-страницу самого Prometheus (Prometheus metrics page) для проверки статуса сбора метрик.
- `/grafana` (Визуализация) - раздел для быстрой навигации по импортированным аналитическим панелям. Обеспечивает бесшовную интеграцию и перенаправление пользователя на соответствующий интерактивный дашборд внутри Grafana (Grafana dashboard).
- `/config` (Настройки системы) - инженерный интерфейс для редактирования и кастомизации файла сигнатур `signatures.yml`, управляющего весами классификатора.
- `/users` (Управление доступом) - административная панель, доступная суперадминистратору (Maintainer) для создания, блокировки и изменения ролей пользователей.

Спроектированная архитектура путей, логика переходов и связи между внутренними страницами веб-приложения и внешними observability-компонентами наглядно представлены на рисунке 17.

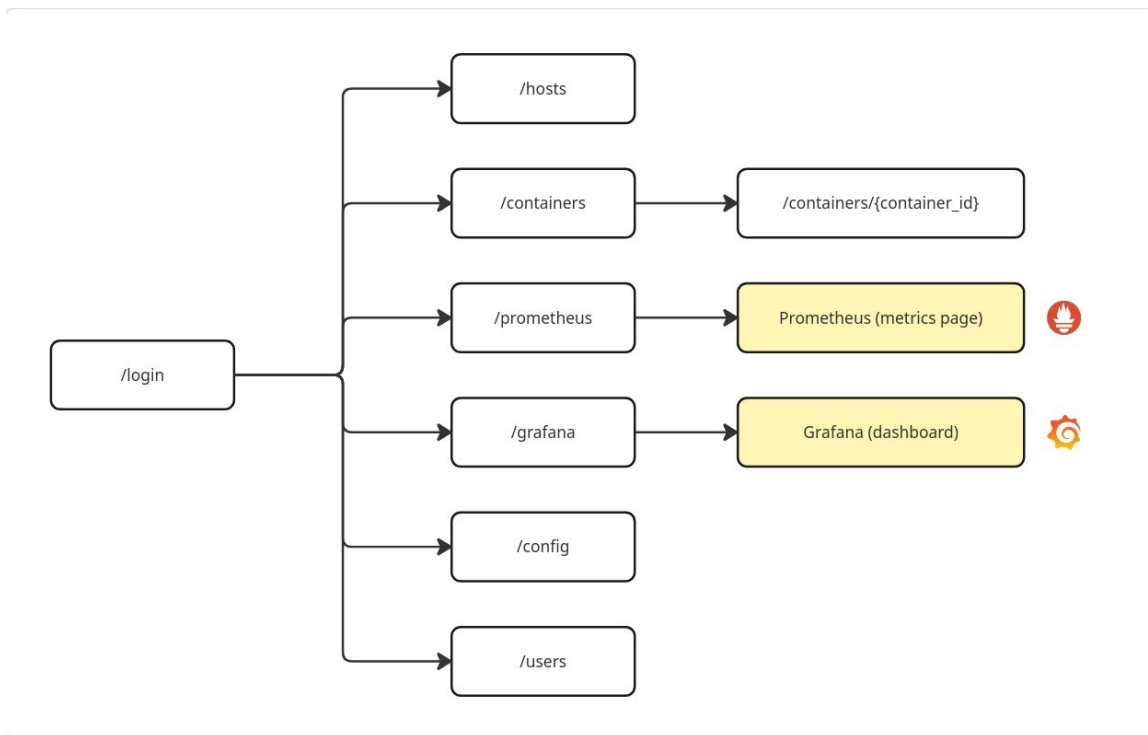


Рисунок 15 – Карта экранов и схема маршрутизации клиентского интерфейса

На стороне клиентского одностраничного приложения (SPA) навигация и отображение страниц контролируются модулем маршрутизации. Каждому URL-адресу соответствует определенный изолированный Vue-компонент, инкапсулирующий в себе собственную логику отображения, отправку запросов к REST API и обработку состояний. Это обеспечивает модульность фронтенд-архитектуры и позволяет бесшовно обновлять интерфейс без перезагрузки страницы. Полная карта соответствия путей, задействованных компонентов бэкенда интерфейса и их функционального назначения приведена в таблице 27.

Таблица 27 – Спецификация маршрутов и компонентов клиентского приложения

Маршрут	Компонент	Назначение
/login	LoginView	Аутентификация
/users	UsersView	CRUD пользователей (maintainer)
/hosts	HostsView	Управление хостами
/containers	ContainersView	Список контейнеров
/container/:id	ContainerDetailView	Детали, действия
/prometheus	PrometheusView	Конфиги, manager
/grafana	GrafanaView	Шаблоны, импорт, manager
/config	ConfigView	Общие настройки, signatures

2.11.2 Защита маршрутов

Для предотвращения несанкционированного доступа к служебным интерфейсам и реализации ролевой модели на стороне фронтенда внедрен механизм защиты маршрутов (Navigation Guards). Проверка прав осуществляется динамически в момент совершения пользователем перехода на любой из внутренних URL-адресов приложения.

Выводы по разделу 2

В данном разделе спроектирована архитектура разрабатываемого комплекса Auto_Observability. Определены три ключевых принципа построения системы: единая точка входа через API Gateway, разделение ответственности между микросервисами и дифференцированное хранение данных в зависимости от их роли. Обоснован выбор технологического стека: Python 3.11+ и FastAPI для бэкенда, Vue 3 с TypeScript для фронтенда, PostgreSQL 15, Redis 7, MinIO и Celery 5 для фоновых задач. Такой набор обеспечивает высокую производительность, типобезопасность, воспроизводимость окружения через Docker Compose и совместимость с индустриальным стандартом observability. Разработана ролевая модель с разграничением прав доступа на уровне API через JWT и RBAC. Построены диаграммы вариантов использования для каждой роли, описаны сквозные потоки данных. Спроектирована физическая модель базы данных PostgreSQL, включающая таблицы hosts, containers, prometheus_configs, grafana_imported_dashboards и app_users с необходимыми индексами и ограничениями целостности. Определена схема хранения в Redis с разделением на логические базы данных. Разработана иерархия объектов в MinIO для версионизируемого хранения YAML-артефактов. Описана бизнес-логика ключевых модулей: алгоритм взвешенной классификации контейнеров по сигнатурам, механизм генерации конфигураций Prometheus на основе классифицированного стека. Построена контекстная диаграмма IDEF0. Завершает раздел карта экранов пользовательского интерфейса с маршрутизацией и защитой маршрутов на основе ролей.

3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

Раздел 3 описывает конкретную реализацию программного комплекса Auto_Observability: файловую структуру, классы, алгоритмы, обработку ошибок, граничные случаи и связь кода с требованиями FR/NFR из раздела 1. Логическая архитектура, потоки данных и модели хранилищ приведены в разделе 2.

3.1 Структура репозитория и средства развёртывания

3.1.1 Организация исходного кода

Разработка программного комплекса осуществлена в рамках концепции монорепозитория, что позволяет централизованно управлять кодовой базой и версионированием всех компонентов. Проект разделен на изолированные, функционально независимые единицы (deployable units). Каждая такая единица инкапсулирует собственную бизнес-логику, содержит изолированный конфигурационный файл зависимостей requirements.txt и собственный Dockerfile для сборки неизменяемого docker-образа. В корневом каталоге проекта размещены глобальные конфигурационные файлы: docker-compose.yml, координирующий развертывание физического стенда, шаблон конфигурации окружения .env.example, и автоматизированный скрипт инициализации тестового окружения scripts/run_demo_observability_targets.sh. Там же находится конфигурационный файл signatures.yml, выступающий единым источником правды для модулей классификации и генерации. Архитектурное назначение ключевых директорий репозитория приведено в таблице 28.

Таблица 28 – Структура каталогов монорепозитория

Каталог	Функциональное назначение в кодовой базе
api_aggregator/	Центральное ядро системы: маршрутизация, Celery-воркеры, интеграция с СУБД и Redis
users_api/	Изолированный сервис аутентификации и управления пользователями (таблица app_users)
docker_api/	Микросервис-обертка над Docker SDK для низкоуровневого управления контейнерами

Каталог	Функциональное назначение в кодовой базе
docker_classification/	Аналитический модуль, реализующий алгоритм класса WeightedDiscovery
prometheus_generation/	Генератор конфигураций на базе PrometheusConfigGenerator и MinioService
prometheus_manager/	Управление жизненным циклом и перезапуском инстанса Prometheus, синхронизация FS
grafana_generation/	Модуль динамического импорта и кастомизации JSON-моделей дашбордов
grafana_manager/	Управление жизненным циклом и runtime-параметрами инстанса Grafana
frontend/	Одностраничное клиентское веб-приложение на фреймворке Vue 3
tests/	Набор сквозных (end-to-end) и интеграционных тестов на базе фреймворка pytest

3.1.2 Docker Compose (реализация физического стенда)

Для развертывания готового программного комплекса в целевом окружении разработан конфигурационный файл docker-compose.yml, жестко фиксирующий параметры изоляции и взаимодействия компонентов. Все контейнеры объединены в изолированную виртуальную сеть app_network типа bridge. Долговременное хранение состояний баз данных и файлов вынесено в именованные тома (volumes) postgres_data, redis_data и minio_data.

3.2 API Aggregator - детальная реализация

Точкой входа центрального агрегатора является модуль app/main.py, инициализирующий экземпляр FastAPI. Управление жизненным циклом завязано на

асинхронный контекстный менеджер `lifespan`, который при старте выполняет накат патчей реляционной схемы. Подключены роутеры с префиксами `/api/v1/hosts`, `/containers`, `/prometheus` и `/grafana`. Настроены CORS (на этапе прототипа `allow_origins=["*"]`) и кастомная JWT-авторизация. Эндпоинты `/health` и `/docs` исключены из проверки токена.

`JwtAuthMiddleware` пропускает `OPTIONS`-запросы и пути из списка исключений, извлекает токен из заголовка `Authorization`, декодирует JWT с помощью `pyjwt` (общий секрет с `Users API`) и проверяет права через матрицу RBAC. Матрица реализована в модуле `permissions.py`: роли `maintainer` и `admin` имеют полный доступ, `user` — только GET-запросы, `dev` — GET разрешены, но запрещены `PATCH /containers/update_containers`, `POST /prometheus/generate_config`, `up_exporter`, удаление дашбордов и другие критические операции.

`SQLAlchemy` модели используют декларативный стиль. Поля `docker_info` и `classification_score` имеют тип `JSONB`. Связь `Container` → `PrometheusConfig` настроена с каскадным удалением. Для работы с `Redis` реализованы классы-обёртки `Hosts` и `DockerContainers`, управляющие ключами `host:{id}` и `container:{host_name}:{container_id}`. Операции выполняются через пайплайны, `TTL` не используется — актуальность обеспечивается периодической перезаписью.

`APIGateway` — единый класс для межсервисных вызовов. Таймаут по умолчанию — 5 секунд, для тяжёлых операций (сборка общего конфига) — 15 секунд. Ошибки преобразуются: `Timeout` → 504, `ConnectionError` → 503.

`HostsService` синхронизирует `PostgreSQL` и `Redis`. Метод `upload_hosts` читает хосты из БД, выполняет `health-check` каждого через `GET /health` и обновляет `Redis`. Для локальных адресов внутри контейнера (`localhost`, `127.0.0.1`) адрес подменяется на `host.docker.internal` с пометкой `local_only`.

`UpdateContainers` реализует фоновое обновление инвентаря. Для каждого хоста вызывает `docker_api discover`, затем для каждого контейнера отправляет метаданные в `docker_classification` и сохраняет результат в `Redis`. Защита от сбоев: если все хосты вернули пустой список контейнеров, старые ключи в `Redis` не удаляются. Celery-задача `update_hosts` запускается каждые 15 секунд, `update_containers` — каждую минуту.

Роутер `hosts` предоставляет эндпоинты: `GET /get` (чтение из `Redis`), `POST /add` (создание в БД и `Redis`), `PUT /update`, `DELETE /delete`, `GET /update_hosts` (ручная

синхронизация). Роутер containers включает GET /containers (чтение из Redis с обогащением статусами Prometheus и Grafana), PATCH /update_containers, а также POST /{action} для управления жизненным циклом через Docker API.

Эндпоинт POST /prometheus/generate_config реализует сквозной онбординг контейнера. Система проверяет существование хоста, читает данные контейнера из Redis, определяет стек из классификации, вычисляет сетевой адрес таргета (приоритет — опубликованный порт экспортера, затем дефолтный из signatures.yml), вызывает prometheus_generation через APIGateway, сохраняет контейнер в PostgreSQL (stack, docker_info, classification_score) и фиксирует конфигурацию в таблице prometheus_configs с инкрементом version. Клиенту возвращаются config_id и сгенерированный YAML.

Эндпоинт POST /prometheus/up_exporter запускает sidecar-экспортер. При отсутствии активной конфигурации автоматически вызывается generate_config. Затем из config_metadata извлекаются параметры экспортера (образ, переменные окружения, сеть), формируется имя контейнера по шаблону {container_name}-exporter, и отправляется запрос в docker_api на pull_and_run. Экспортер запускается в той же сети, что и целевой контейнер.

3.3 Users API - реализация

Микросервис users_api управляет таблицей app_users. Эндпоинт POST /login проверяет пароль через bcrypt и при успехе генерирует JWT с помощью rujwt (payload содержит sub, role, iat, exp, время жизни — 480 минут). Роутер users_mgmt.py (CRUD пользователей) доступен только для роли maintainer. Пароли хешируются перед сохранением.

3.4 Docker API - реализация

DockerManager инициализирует клиент через docker.from_env(), подхватывая смонтированный сокет /var/run/docker.sock. Метод discover_containers возвращает полный inspect всех контейнеров. Метод pull_and_run_container проверяет наличие образа, при необходимости скачивает его, удаляет существующий контейнер с тем же именем (если образ изменился), затем запускает новый контейнер в указанной сети.

3.5 Docker Classification - реализация алгоритма

3.5.1 WeightedDiscovery

Алгоритмическая основа подсистемы автоматического определения типов развернутого программного обеспечения сосредоточена в классе WeightedDiscovery

(модуль `docker_clasification.py`). При инициализации конструктора класса (`__init__`) происходит чтение и парсинг конфигурационного файла `signatures.yml`, в результате чего в память загружаются четыре изолированных словаря правил: `ports`, `env`, `images` и `labels`. Также на уровне класса выставляется жесткий порог отсека слабых кандидатов `self.threshold = 50`, что минимизирует вероятность ложноположительных срабатываний классификатора.

Основной расчетный метод `classify_container` принимает на вход словарь десериализованных метаданных инспекции Docker и выполняет вычисление весов по следующему пошаговому алгоритму:

- Инициализируется пустой словарь счетчиков с дефолтными целочисленными значениями через `scores = defaultdict(int)`.
- Анализ сетевых портов: Скрипт обходит массив открытых портов контейнера. Если числовое значение обнаруженного порта присутствует в конфигурационном словаре правил `ports`, счетчику соответствующей технологии прибавляется заданный правилом вес: `scores[tech] += weight`.
- Анализ переменных окружения: Запускается цикл сканирования переменных среды контейнера. Проверка осуществляется методом поиска подстроки: если эталонный ключ из правил содержится в виде подстроки в любой реальной строке окружения `KEY=value`, весовой коэффициент технологии инкрементируется.
- Анализ имени образа: Выполняется аналогичная проверка на совпадение подстроки для поля используемого Docker-образа. Наличие ключевых слов (например, `postgres`, `redis`, `rabbitmq`) в названии или теге образа добавляет баллы в копилку соответствующего стека.
- Анализ метаданных `labels`: Система сканирует пользовательские метки контейнера. Если в конфигурации сигнатур для метки выставлен режим автоматического определения `data['tech'] == 'auto'`, то текстовое значение, содержащееся в этой метке контейнера, напрямую признается целевой технологией и получает максимальный приоритет.
- Фильтрация и сортировка результатов: Из результирующего словаря `scores` исключаются все технологии, чей суммарный балл оказался ниже установленного порога: `score < threshold`. Оставшиеся кандидаты преобразуются в список кортежей и

сортируются по убыванию набранного веса: `sorted(final.items(), key=lambda x: x[1], reverse=True)`).

Роутер микросервиса перехватывает возвращенный список и упаковывает его в стандартизированный JSON-ответ вида `{"result": [[tech, score], ...]}` для последующей обработки центральным агрегатором.

3.5.2 Отличие от `signatures.yml` корня

Важной деталью программной реализации является различие контекстов использования конфигурационных файлов, имеющих одинаковое имя `signatures.yml`. Внутри контейнера микросервиса `docker_classification` задействуется локальная копия или изолированная секция файла сигнатур, структура которой жестко ориентирована на правила статического анализа: она содержит исключительно перечни портов, подстрок окружения, имен образов и меток, ассоциированных с весовыми коэффициентами.

В то же время в корневом каталоге всего монорепозитария размещен расширенный конфигурационный файл, чья структура ориентирована на процесс генерации и развертывания инфраструктуры мониторинга. Корневой файл содержит корневые секции конкретных технологических стеков (например, `postgresql`:, `redis`:, `rabbitmq`:), внутри которых описаны параметры запуска `sidcar`-контейнеров, команды инициализации и ссылки на официальные идентификаторы дашбордов Grafana. Таким образом, это два принципиально разных формата данных, сосуществующих под одним именем: при изменении или добавлении новых правил распознавания инженеру необходимо обновить файл внутри образа классификатора либо переопределить его через независимый механизм монтирования томов.

3.6 Prometheus Generation - реализация

3.6.1 Роутер `generate`

Генерация конфигурационных файлов для агентов сбора метрик координируется роутером `prometheus_generation/app/routers/generate.py`. На вход эндпоинта передается валидированная Pydantic-модель `ContainerData`, агрегирующая в себе сырой `inspect`-объект, структуру весов классификации и опциональный параметр `prometheus_scrape_port`. Также роутер принимает `query`-параметры `host` и `prometheus_scrape_port`, которые обладают наивысшим приоритетом над данными из тела запроса.

Логика обработки запроса эндпоинтом строится по следующей схеме:

- Инициализируется экземпляр класса генератора, который вызывает метод `PrometheusConfigGenerator().generate_config(container_dict, host)`.

- Если в процессе разбора данных генератор возвращает пустое значение (`config is None`), роутер прерывает выполнение и генерирует исключение HTTP 400 Bad Request. При этом в тело ответа автоматически инжектируется список всех доступных в системе технологических стеков, извлеченных из ключей конфигурации сигнатур, что позволяет фронтенду подсказать пользователю возможные варианты для ручного выбора технологии.
- При успешной генерации вызывается метод `MinioService.upload_config()`, который асинхронно загружает сформированный текстовый файл конфигурации и сопутствующий YAML-массив таргетов в объектное хранилище MinIO.
- Дополнительно запускается фоновый алгоритм проверки существования глобального конфигурационного файла: если в бакете по пути `mainConfig/targets/{job_name}.yaml` уже присутствует старая версия файла целевых адресов, система выполняет её автоматическую синхронизацию и перезапись, предотвращая рассинхронизацию центрального сервера сбора метрик.

3.6.2 PrometheusConfigGenerator.generate_config (логика)

Внутренний аналитический модуль генерации `PrometheusConfigGenerator` представляет собой комплексный класс объемом более 300 строк кода. На первом этапе метод `_get_stack_from_classification` считывает строковое имя технологии, имеющей максимальный балл в массиве результатов классификатора, после чего функция `_normalize_stack_name` приводит строку к нижнему регистру и заменяет пробелы на символы подчеркивания для обеспечения совместимости с синтаксисом путей. Далее из загруженного файла сигнатур считывается целевая конфигурационная секция `exporter_configs`.

На основе полученной технологической карты запускается процесс формирования блока `scrape_config`. Система генерирует уникальное имя задачи `job_name`, прописывает относительный путь сбора метрик `metrics_path` и интервал опроса `scrape_interval`. Внутри блока `static_configs` формируется массив `targets`, куда вносится строковый адрес назначения в формате `f"{host}:{port}"`. При этом порт вычисляется по иерархическому принципу: приоритет имеет переданный `query`-параметр, при его отсутствии берется опубликованный порт из `inspect`-данных контейнера, а в качестве `fallback`-значения используется дефолтный `exporter_port` из файла сигнатур.

Для формирования параметров среды `sidecar`-контейнера вызывается модуль `ExporterEnvGenerator`. Он осуществляет синтаксический разбор шаблона `env_template`,

выполняя подстановку значений переменных {host} и {port}, а также автоматически извлекает учетные данные и пароли доступа из исходных переменных окружения самого защищаемого контейнера. Критически важной частью класса является метод `_apply_sidecar_runtime`, реализующий специфичные ветвления алгоритма в зависимости от выбранного технологического стека:

- **RabbitMQ:** Если в поле сигнатур присутствует флаг `native_prometheus_scrape`, система полностью отказывается от генерации sidecar-контейнера. Сбор метрик настраивается напрямую с встроенного в RabbitMQ плагина мониторинга.
- **MySQL / MariaDB:** В блок запуска экспортёра внедряется переопределенная командная строка с явным указанием флага адреса СУБД `--mysqld.address`, а параметры авторизации бесшовно инжектируются в переменные среды контейнера.
- **Elasticsearch / Opensearch:** Алгоритм автоматически генерирует специфичный стартовый флаг конфигурации `--es.uri=`.
- **ClickHouse / Kafka / NATS:** Система подставляет уникальные, присущие только этим технологиям флаги командной строки для корректной инициализации соответствующих официальных экспортёров.
- **InfluxDB:** Реализована кастомная логика для старых версий API (InfluxDB 1.x). Поскольку классический экспортёр не поддерживает обратную совместимость с этой веткой, система генерирует параметры для развертывания легковесного агента Telegraf. Конфигурация sidecar формируется в виде сложного однострочного shell-скрипта (`entrypoint shell one-liner`), который на лету создает inline-конфигурацию Telegraf внутри контейнера и запускает сбор данных.

На финальном шаге метод определяет имя виртуальной сети: выбирается первая доступная сеть контейнера нагрузки, не являющаяся системным host-интерфейсом, с откатом на стандартный bridge. Метод возвращает вызывающему роутеру словарь, содержащий сгенерированные блоки файлов и параметры запуска экспортёра.

3.6.3 MinioService (generation)

Компонент MinioService внутри модуля генерации, в отличие от аналогичного сервиса в агрегаторе, функционирует в режиме полных прав на запись и модификацию данных. Загрузка текстовых артефактов в распределенное объектное хранилище осуществляется через низкоуровневый метод `put_object` библиотеки boto3. Текстовые строки конфигураций в обязательном порядке преобразуются в бинарный поток с

помощью кодирования `.encode('utf-8')`, а в заголовках запроса передается сформированный словарь кастомных метаданных `Metadata`.

Логика распределения файлов в MinIO жестко привязана к двум типам путей. При генерации индивидуального мониторинга файлы сохраняются с использованием уникального префикса контейнера: по пути `configs/{container_id}/scrape_config.yml` размещается объект, содержащий обертку изолированной задачи сбора, а файл `configs/{container_id}/{job_name}.yml` принимает плоский YAML-массив, описывающий конечный target-адрес.

При сборке общего конфигурационного контура вызывается специализированный метод `upload_main`, который осуществляет запись глобального файла манифеста и сопутствующих файлов описания работ непосредственно в центральную директорию `mainConfig/`. Класс также предоставляет вспомогательные методы `get_yaml_file`, `list_files` и `delete_file`, которые активно задействуются менеджером конфигураций и автоматизированными тестами для верификации хранилища.

3.7 Prometheus Manager и Grafana

Prometheus Manager синхронизирует конфиги из MinIO на диск хоста: скачивает `mainConfig/prometheus.yml` и все файлы из `mainConfig/targets/`, после чего отправляет сигнал `SIGHUP` контейнеру Prometheus для горячей перезагрузки. При ошибке выполняется перезапуск контейнера.

Grafana Generation реализует импорт дашбордов: скачивает JSON-шаблон с grafana.com, подменяет `datasource` на глобальный UID Prometheus, отправляет `POST /api/dashboards/db` в Grafana. Для ускорения используется кэш JSON на диске (`.dashboard_cache`). Grafana Manager управляет контейнером Grafana (`start/stop/restart`) аналогично Prometheus Manager.

3.8 Frontend - реализация клиента

Клиентское приложение на Vue 3 + TypeScript собирается Vite. Конфигурация прокси перенаправляет `/api/v1/auth/*` и `/api/v1/users` в Users API, остальные `/api/*` — в API Aggregator. JWT хранится в `localStorage`, роутер проверяет наличие токена и роль для доступа к `/users` (только `maintainer`). Перехватчик `axios` автоматически добавляет заголовок `Authorization: Bearer`. Компоненты скрывают кнопки мутаций через `usePermissions.ts` в зависимости от роли.

3.9 Nginx

Маршрутизация и раздача статического контента в production-окружении обеспечивается веб-сервером Nginx. Конфигурационный файл содержит жестко прописанные правила разделения входящего трафика на уровне upstream-модулей. Проксирование вызовов подсистемы авторизации реализовано через директивы `/api/v1/auth/` и `/api/v1/users` без использования замыкающих слэшей (trailing slash). Это является критически важным архитектурным решением, так как строгое совпадение строк гарантирует корректную маршрутизацию POST-запросов создания пользователей и исключает их ошибочное преобразование в GET-методы из-за внутренних редиректов веб-сервера.

3.10 Файл `signatures.yml` - реализация конфигурации платформы

Размещенный в корневом каталоге монорепоzitория конфигурационный файл `signatures.yml` представляет собой декларативное описание параметров платформы. Несмотря на то, что данный файл является набором статичных данных формата YAML, а не исполняемым кодом, на его структуру завязана логика работы трех ключевых микросервисов системы. Внутри манифеста объявляются изолированные блоки для каждого поддерживаемого технологического стека, пример реализации секции для СУБД PostgreSQL приведен ниже:

Листинг 1.

```
# Реализация конфигурационной секции стека в signatures.yml
postgresql:
  job_name_suffix: "_postgres"
  exporter_port: 9187
  exporter_image: "prometheuscommunity/postgres-exporter:latest"
  grafana_dashboard_id: 9628
  env_template:
    "postgresql://{user}:{password}@{host}:{port}/{database}?
    sslmode=disable"
```

Данные параметры динамически считываются и парсятся программными компонентами системы в следующих целевых точках:

- Метод PrometheusConfigGenerator._load_signatures обращается к файлу для определения суффикса задачи и базового порта экспортера при формировании структуры scrape_configs.
- Модуль ExporterEnvGenerator использует строку шаблона env_template для генерации валидных строк подключения sidecar-контейнера к защищаемой базе данных.
- Сервис GrafanaTemplateLoader считывает числовое значение grafana_dashboard_id для автоматического скачивания соответствующей схемы визуализации с официального сайта.

Помимо базовых ключей, структура файла поддерживает установку расширенных параметров: scrape_timeout для ограничения времени ожидания ответа от экспортера, exporter_command для переопределения базовой точки входа образа и логический флаг native_prometheus_scrape (используемый для RabbitMQ), сигнализирующий системе о возможности прямого сбора метрик без развертывания дополнительных sidecar-контейнеров. Редактирование файла осуществляется через инженерную панель веб-интерфейса посредством метода PUT templates.yml, который перезаписывает файл на физическом диске контейнера grafana_generation. Благодаря использованию общего механизма Docker Volumes (bind-mount), измененный файл мгновенно становится доступен и микросервису prometheus_generation, обеспечивая синхронизацию правил генерации во всей системе.

Выводы по разделу 3

В третьем разделе представлена программная реализация разработанного комплекса Auto_Observability. Описана организация монорепозитория, где каждый микросервис (api_aggregator, users_api, docker_api, docker_classification, prometheus_generation, prometheus_manager, grafana_generation, grafana_manager, frontend) изолирован и обладает собственной бизнес-логикой. Развёртывание комплекса автоматизировано через Docker Compose с использованием именованных томов для персистентного хранения данных и единой виртуальной сети app_network.

В центральном агрегаторе реализована JWT-аутентификация через middleware JwtAuthMiddleware и ролевая модель доступа (maintainer, admin, dev, user) на базе модуля permissions.py. Для высокопроизводительного кэширования инвентаря разработаны классы-обёртки Hosts и DockerContainers, взаимодействующие с Redis через пайплайны. Межсервисные вызовы централизованы в классе APIGateway с обработкой таймаутов и сетевых ошибок. Фоновое обновление состояния хостов и контейнеров реализовано через

Celery с защитой от потери данных при сетевых сбоях: если все хосты недоступны, старые ключи в Redis сохраняются.

Алгоритм классификации `WeightedDiscovery` анализирует четыре категории метаданных контейнера (порты, переменные окружения, образы, метки) и возвращает ранжированный список технологических стеков с весовыми коэффициентами, отсекая кандидатов ниже порога 50. Генератор конфигураций `PrometheusConfigGenerator` (более 300 строк кода) на основе классифицированного стека формирует `scrape_config`, подставляет переменные окружения через `ExporterEnvGenerator` и реализует специфичные ветвления для RabbitMQ (прямой сбор метрик), InfluxDB 1.x (развертывание Telegraf), MySQL, Elasticsearch и других технологий. Сгенерированные YAML-артефакты сохраняются в MinIO.

Реализованы сквозные сценарии онбординга: эндпоинт `generate_config` выполняет полный цикл, а `up_exporter` запускает `sidecar`-экспортёр в сети целевого контейнера через `DockerManager`. `Users API` обеспечивает выдачу JWT и CRUD пользователей, `Prometheus Manager` синхронизирует конфиги на диск и отправляет сигнал `SIGHUP`, `Grafana Generation` импортирует дашборды с локальным кэшированием.

Клиентское приложение на Vue 3 с TypeScript использует Vite для сборки, axios-перехватчик для автоматической подстановки токена и роутер с защитой маршрутов. Nginx в production-сборке проксирует API-запросы, разделяя трафик между `Users API` и `API Aggregator`. Единый конфигурационный файл `signatures.yml` в корне репозитория служит источником правил для классификации, генерации и импорта дашбордов, а его редактирование через веб-интерфейс мгновенно синхронизируется между сервисами через `bind-mount`.

Таким образом, программная реализация полностью соответствует функциональным требованиям FR-01–FR-19, заложенным в разделе 1, и создаёт основу для тестирования и апробации системы, описанных в следующем разделе.

4. ТЕСТИРОВАНИЕ И АПРОБАЦИЯ СИСТЕМЫ

4.1 Цели и стратегия тестирования

Тестирование преследовало следующие цели: подтвердить корректность ключевых модулей (агрегатор, классификация, генерация Prometheus, Redis, MinIO, JWT/RBAC), обеспечить регрессионную защиту при изменении кода и выполнить апробацию на воспроизводимом стенде Docker Compose. Стратегия сочетает три уровня: модульное тестирование отдельных классов с подменой зависимостей (pytest, unittest.mock), компонентное тестирование HTTP-эндпоинтов через TestClient FastAPI и интеграционную апробацию на полном стеке с реальными Docker, Prometheus и Grafana.

4.2 Средства и окружение тестирования

Основным фреймворком выбран pytest. Для изоляции компонентов используются unittest.mock, а для эмуляции HTTP-запросов к ASGI-приложению — FastAPI TestClient. Статический анализ кода выполняется с помощью Pylint, интегрированного в GitHub Actions.

4.3 Автоматизированное тестирование

Разработан набор автотестов, покрывающих критическую бизнес-логику. Тестируется валидация JWT и разграничение прав доступа (user не может выполнять POST-запросы, dev не может вызывать update_containers и generate_config). Проверяются эндпоинты роутеров hosts и containers, кэширование в Redis (пайплайны, загрузка, чтение, удаление ключей), а также работа MinioService и APIGateway (трансформация таймаутов в 504, ConnectionError в 503).

Алгоритм классификации WeightedDiscovery проверяется на корректность подсчёта весов для различных комбинаций портов, переменных окружения, образов и меток. Генератор конфигураций PrometheusConfigGenerator тестируется на правильность формирования scrape_config для PostgreSQL, Redis, MySQL и InfluxDB. Отдельные тесты проверяют работу DockerManager (discover, pull_and_run), Prometheus Manager (синхронизация конфигов, отправка SIGHUP) и импорт дашбордов Grafana с подменой datasource. В общей сложности разработано более 200 тестовых методов, что подтверждает выполнение требования NFR-16.

4.4 Ручная апробация и интеграционные сценарии

Для сквозной проверки связей на реальном стенде выполняется развёртывание через `docker compose up`. Базовый сценарий апробации включает следующие шаги: аутентификация под учётной записью `maintainer`, добавление целевого хоста, ожидание фонового обновления инвентаря (или ручной вызов `update_containers`), появление контейнеров в таблице с заполненным полем `stack`, переход в карточку контейнера, генерация конфигурации Prometheus, запуск `sidecar`-экспортёра, синхронизация конфига через Prometheus Manager и импорт дашборда в Grafana. Дополнительно проверяется ролевая модель: пользователь с ролью `user` не видит управляющих кнопок и не может выполнить POST-запросы, а разработчик (`dev`) имеет доступ к просмотру, но не может выполнять критические операции. Для автоматизации вспомогательных проверок разработаны скрипты `tests/test_container_monitoring.py` (полный цикл через HTTP-запросы к агрегатору) и `scripts/run_demo_observability_targets.sh` (развёртывание тестовых контейнеров).

4.5 Проверка нефункциональных требований

В ходе апробации подтверждено выполнение ключевых нефункциональных требований. Простота развёртывания (NFR-01) проверена запуском `docker compose up` на чистой виртуальной машине. Производительность API (NFR-04) оценивалась замером времени ответа `GET /containers` при заполненном кэше Redis — целевое значение менее 200 мс достигнуто. Отказоустойчивость (NFR-05) обеспечена запуском двух параллельных Celery-воркеров. Безопасность (NFR-10–NFR-13) подтверждена попытками передачи поддельного, просроченного или пустого JWT, а также проверкой изоляции ролей. Сценарий безагентного сбора данных (NFR-15) реализован: система обнаруживает контейнеры без предварительной установки SSH-ключей или агентов на целевой хост.

4.6 Ограничения тестирования

При анализе результатов необходимо учитывать следующие ограничения. Большинство автотестов изолированы с помощью заглушек — они проверяют корректность логики функций, но не гарантируют интеграцию со сторонними сервисами при изменении сетевых политик. Отсутствует CI-шаг для прогона `pytest` на удалённом сервере (в пайплайне активен только Pylint). Компонент управления пользователями (Users API) практически не покрыт модульными тестами — его стабильность подтверждена через middleware агрегатора и ручную проверку. Клиентская часть не имеет сквозных e2e-тестов (Playwright/Cypress) — верификация интерфейса выполнялась

ручным клик-тестированием. Нагрузочное тестирование и специализированный аудит безопасности в рамках диплома не проводились.

Выводы по разделу 4

В разделе описана комплексная стратегия верификации программного комплекса Auto_Observability. Создана инфраструктура автоматизированного тестирования на базе pytest, включающая более 200 тестовых методов, покрывающих логику JWT/RBAC, маршрутизацию API Aggregator, кэширование в Redis, сохранение артефактов в MinIO, диспетчеризацию фоновых задач Celery, алгоритмы классификации и генерации конфигураций. Для проверки интеграционного взаимодействия развёрнут демонстрационный стенд на Docker Compose, на котором успешно проведена апробация сквозного сценария онбординга инфраструктурного хоста — от первичного обнаружения контейнеров до вывода метрик в Prometheus и построения дашбордов в Grafana. Результаты испытаний подтверждают работоспособность системы, её стабильность к регрессионным изменениям и соответствие ключевым функциональным требованиям. Зафиксированные ограничения (отсутствие e2e-тестов, нагрузочного тестирования, неполное покрытие Users API) определяют вектор дальнейшего развития системы до уровня промышленного решения.

5. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Раздел описывает порядок работы с программным комплексом Auto_Observability с точки зрения конечного пользователя: инженера эксплуатации (SRE/DevOps), администратора мониторинга или разработчика. Руководство охватывает подготовку рабочего места, вход в систему, типовые сценарии от регистрации Docker-хоста до получения метрик в Prometheus и дашборда в Grafana, а также ограничения, накладываемые ролевой моделью доступа.

Программная реализация интерфейса и API описана в разделе 3; архитектура и потоки данных — в разделе 2. Настоящий раздел ориентирован на практическое использование без необходимости редактировать конфигурационные файлы вручную на хосте и без доступа по SSH (требование NFR-15).

5.1 Назначение и область применения

Auto_Observability — веб-приложение для централизованного управления онбордингом Docker-контейнеров в контур наблюдаемости. Пользователь выполняет следующие задачи через браузер: регистрация целевых машин с доступным Docker API; просмотр инвентаря контейнеров с автоматической классификацией технологического стека; генерация конфигураций Prometheus и запуск sidecar-экспортёров метрик; включение job в общий prometheus.yml, перезапуск Prometheus; импорт готовых дашбордов Grafana по шаблонам из signatures.yml; редактирование правил классификации и параметров менеджеров (для ролей с соответствующими правами); управление учётными записями (только роль maintainer). Система не заменяет Prometheus и Grafana, а автоматизирует подготовку их конфигурации. Пользователь по-прежнему может открыть нативный UI Prometheus и Grafana по ссылкам из интерфейса

5.2 Требования к рабочему месту пользователя

5.2.1 Программные требования.

Для работы с системой необходим современный браузер с поддержкой ES6 и WebSocket (Chrome 90+, Firefox 90+, Edge 90+). Разрешение экрана должно быть не менее 1280×720, для удобной работы с таблицей контейнеров рекомендуется 1920×1080. Требуется доступ к хосту, на котором развёрнут frontend (по умолчанию порт 3000).

5.2.2 Предварительные условия на стороне инфраструктуры.

Перед началом работы оператор (или администратор стенда) должен обеспечить запущенный стек docker compose up -d, доступность Docker API на целевых хостах (для локального стенда — сервис docker_api с пробросом /var/run/docker.sock), а также заполненный файл .env в корне репозитория (копия из .env.example), включающий JWT_SECRET (общий секрет для Users API и Aggregator), MAINTAINER_USERNAME и MAINTAINER_PASSWORD (учётная запись первого администратора), PROMETHEUS_CONFIG_HOST_PATH (путь на хосте к каталогу конфигов Prometheus).

5.3 Обзор интерфейса и навигация

После успешного входа пользователь видит шапку приложения с логотипом Auto Observability, горизонтальным меню и блоком учётной записи (имя пользователя, роль, кнопка «Выход»). Как выглядит главная навигация, показано на рисунке 16.

Пункты меню:

Пункт	Маршрут	Назначение
Hosts	/hosts	Учёт Docker-нод
Containers	/containers	Инвентарь контейнеров (стартовая страница после входа)
Prometheus	/prometheus	Конфиги, main config, Prometheus Manager
Grafana	/grafana	Импорт и список дашбордов, Grafana Manager
Configuration	/config	Редактор signatures.yml и настроек Prometheus Manager
Пользователи	/users	Только для maintainer

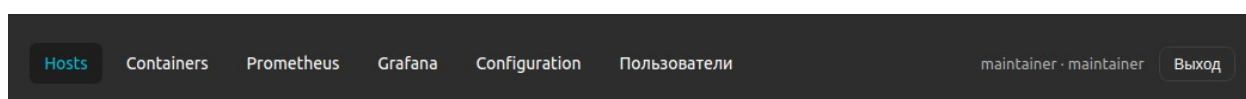


Рисунок 16 – Главная навигация веб-интерфейса Auto_Observability

5.4 Вход в систему и завершение сеанса

5.4.1 Первый вход (учётная запись maintainer)

При первом запуске Users API автоматически создаёт пользователя с ролью maintainer из переменных окружения MAINTAINER_USERNAME и MAINTAINER_PASSWORD (по умолчанию в .env.example: maintainer / changeme — обязательно смените на стенде). Для входа откройте в браузере, введите имя пользователя и пароль, затем нажмите «Войти». Экран входа в систему представлен на рисунке 17.

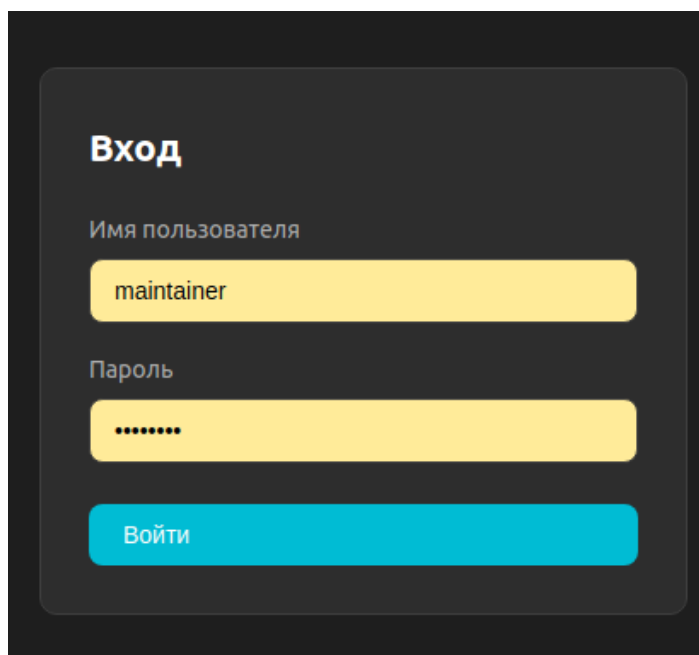


Рисунок 17 – Экран входа в систему

Для выхода из системы нажмите «Выход» в правой части шапки. Токен удаляется из браузера, выполняется переход на страницу входа.

5.5 Работа с хостами (раздел Hosts)

Раздел предназначен для регистрации машин, с которых система снимает инвентарь Docker-контейнеров (FR-03, FR-04).

5.5.1 Просмотр списка хостов

Перейдите Hosts. Таблица содержит колонки:

Колонка	Описание
Name	Понятное имя (необязательно)
Host	IP-адрес или hostname Docker API
Port	Порт API (для docker_api на стенде — 8004 на хосте)

Колонка	Описание
Status	up (зелёный) — health-check успешен; down — хост недоступен
Actions	Edit / Delete (если роль позволяет)

Статус обновляется автоматически каждые 15 секунд (Celery) и при нажатии Refresh. Список зарегистрированных хостов можно увидеть на рисунке 18.

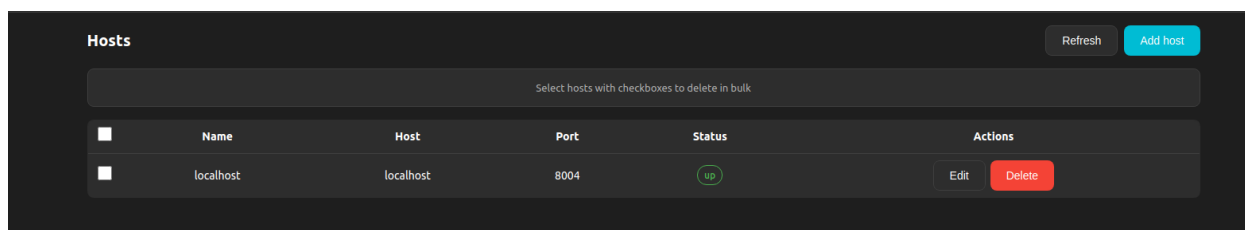


Рисунок 18 – Список зарегистрированных хостов

5.5.2 Добавление хоста

Нажмите Add host, затем заполните модальное окно: Name (например, local-docker), Host (для стенда на одной машине — host.docker.internal или IP хоста, для удалённой ноды — её адрес), Port (порт сервиса Docker API, на учебном стенде — 8004). После сохранения агрегатор выполняет health-check и записывает хост в Redis, после чего запись появляется в таблице. Диалог добавления хоста показан на рисунке 19.

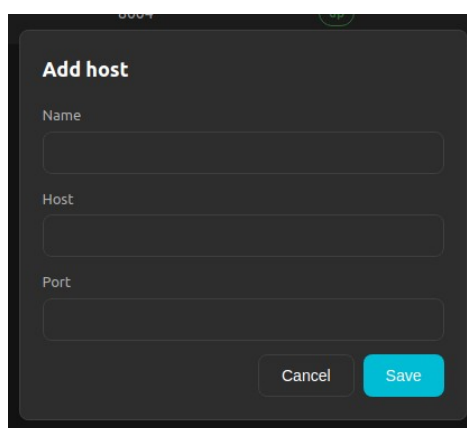


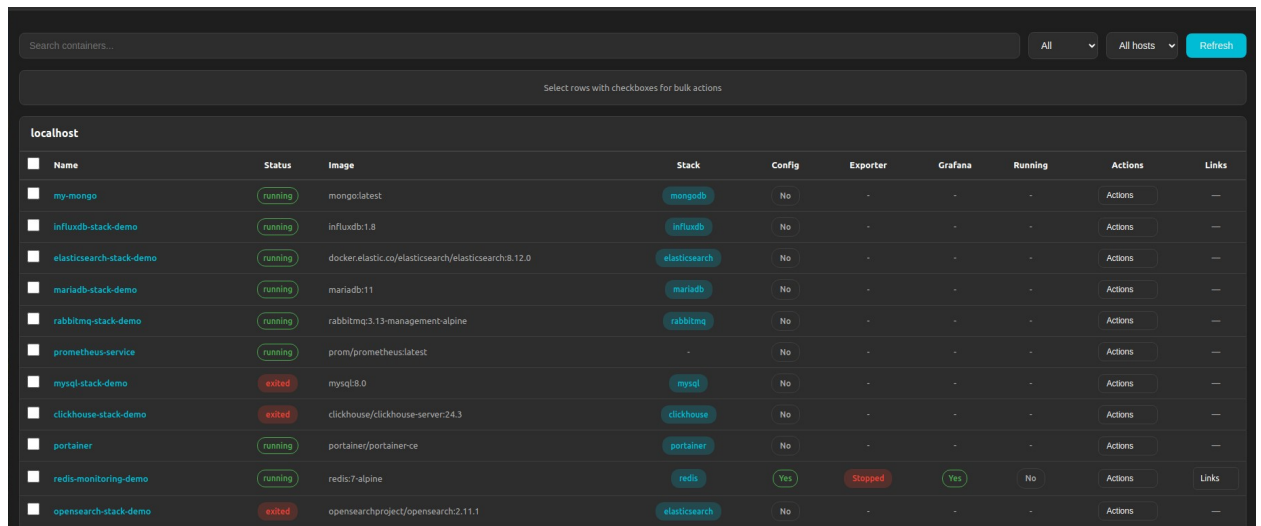
Рисунок 19 – Диалог добавления хоста

5.6 Работа с контейнерами (раздел Containers)

Центральный раздел системы (FR-05, FR-06, FR-07). Отображает оперативный снимок из Redis, обогащённый данными Prometheus и Grafana.

5.6.1 Панель фильтрации и обновления

Панель фильтрации и обновления. В верхней части страницы расположены: поле Search containers для поиска по имени контейнера, фильтр All / Running / Stopped для отбора по состоянию, выпадающий список All hosts для фильтра по хосту, а также кнопка Refresh для принудительного опроса Docker API и переклассификации (PATCH update_containers). Фоновое обновление без участия пользователя выполняется каждую минуту (FR-17). Панель фильтров и группировка по хостам показаны на рисунке 20.



Name	Status	Image	Stack	Config	Exporter	Grafana	Running	Actions	Links
my-mongo	running	mongo:latest	mongodb	No	-	-	-	Actions	—
influxdb-stack-demo	running	influxdb:1.8	influxdb	No	-	-	-	Actions	—
elasticsearch-stack-demo	running	docker.elastic.co/elasticsearch/elasticsearch:8.12.0	elasticsearch	No	-	-	-	Actions	—
mysql-stack-demo	exited	mysql:8.0	mysql	No	-	-	-	Actions	—
clickhouse-stack-demo	exited	clickhouse/clickhouse-server:24.3	clickhouse	No	-	-	-	Actions	—
portainer	running	portainer/portainerce	portainer	No	-	-	-	Actions	—
redis-monitoring-demo	running	redis:7-alpine	redis	Yes	Stopped	Yes	No	Actions	Links
opensearch-stack-demo	exited	opensearchproject/opensearch:2.11.1	elasticsearch	No	-	-	-	Actions	—

Рисунок 20 – Раздел Containers: панель фильтров и группировка по хостам

5.6.2 Столбцы таблицы контейнеров

Колонка	Значение для пользователя
Name	Имя контейнера (клик — переход к деталям)
Status	running, exited и т.д.
Image	Docker-образ
Stack	Результат классификации: postgresql, redis, nginx...
Config	Yes — конфиг Prometheus создан; No — ещё не онбордили
Exporter	Running / Stopped / Missing — состояние sidecar
Grafana	Yes — дашборд импортирован
Running	Контейнер нагрузки запущен
Actions	Меню операций
Links	Быстрые ссылки (Prometheus, Grafana)

Если колонка Stack пуста (-), дождитесь фонового обновления или нажмите Refresh; при необходимости проверьте правила в signatures.yml.

5.6.3. Меню Actions (операции над контейнером)

Откройте выпадающее меню в колонке Actions. Доступные пункты зависят от роли и состояния контейнера:

Пункт	Кто может	Действие
Start / Stop / Remove	maintainer, admin	Управление жизненным циклом через Docker API
Generate config	maintainer, admin, dev	Генерация YAML в MinIO + запись в БД
Start exporter	maintainer, admin, dev	Запуск sidecar-экспортёра
Start all	maintainer, admin, dev	Сквозной пайплайн (см. п. 5.8)
Remove exporter	maintainer, admin	Остановка и удаление sidecar
View details	все	Карточка контейнера

Меню Actions для контейнера представлено на рисунке 21.

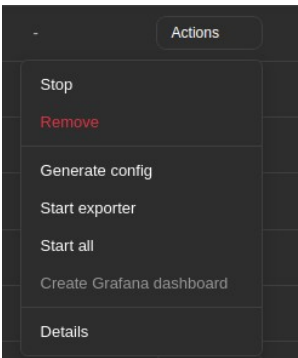


Рисунок 21 – Меню Actions для контейнера

5.6.5 Карточка контейнера (детали)

Клик по имени или View details → маршрут /container/{id}.

На странице отображаются:

- имя, ID, статус, образ, Stack;
- блок Prometheus: статус конфига, экспортёр;
- Networks, Ports, Environment, Labels (сырые метаданные Docker inspect);
- блок Grafana: наличие дашборда;
- кнопки Start / Stop / Remove (admin/maintainer);

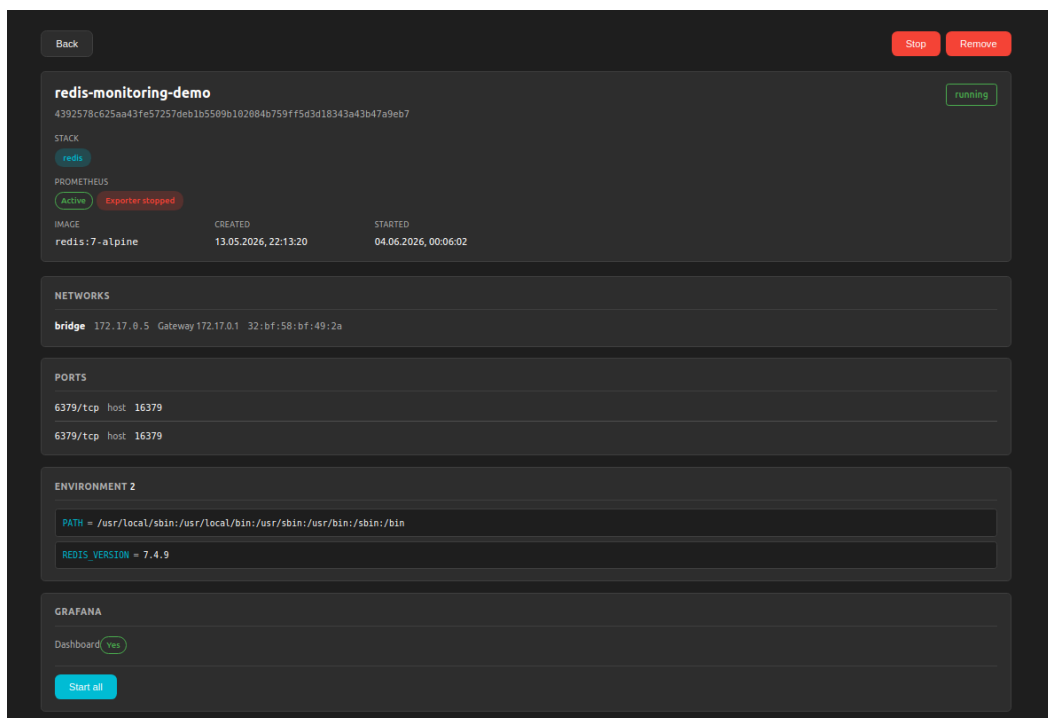


Рисунок 22 – Детальная карточка контейнера

5.7 Сквозной сценарий онбординга («мониторинг по кнопке»)

Рекомендуемый сценарий для подключения нового контейнера к мониторингу. Соответствует UC-05 и требованию NFR-15.

5.7.1 Вариант А: «Start all» (одна команда)

Это наиболее простой путь для пользователя. Предусловия: хост зарегистрирован и в статусе up, контейнер running, в Stack определён технологический стек. На странице Containers найдите контейнер, в меню Actions выберите Start all. В диалоге укажите порт экспортёра (по умолчанию предлагается 9100, для PostgreSQL часто 9187 — берётся из signatures.yml). Дождитесь завершения (может занять до 2 минут). В правом нижнем углу появится уведомление «Start all completed». Успешный онбординг (все колонки Config / Exporter / Grafana = Yes) показан на рисунке 23.



Рисунок 23 – Успешный онбординг: все колонки Config / Exporter / Grafana = Yes

5.7.2 Вариант В: пошаговый онбординг

Для контроля каждого этапа или при отладке.

Шаг 1. Generate config (или страница /container/{id}/generate-exporter).

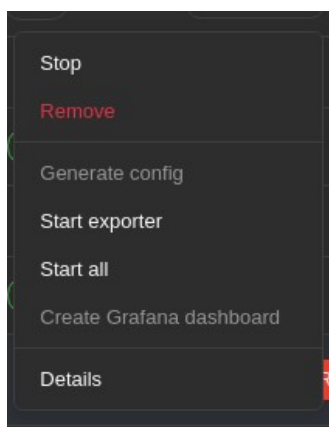


Рисунок 25 – Страница «Generate & Start Exporter», Step 1

Шаг 2. Start exporter — указать порт, нажать запуск.

Ввод данных на сайте localhost

Enter exporter port (default: 9100):

Продолжить

Отмена

Рисунок 26 – Step 2: Start Exporter с полем Exporter Port

Шаг 3. Prometheus → в таблице Services найти контейнер → Add to main config (если не добавлен автоматически).

Шаг 4. Prometheus Manager → Restart Prometheus (если интерфейс показывает «To apply changes, Prometheus needs to be restarted»).

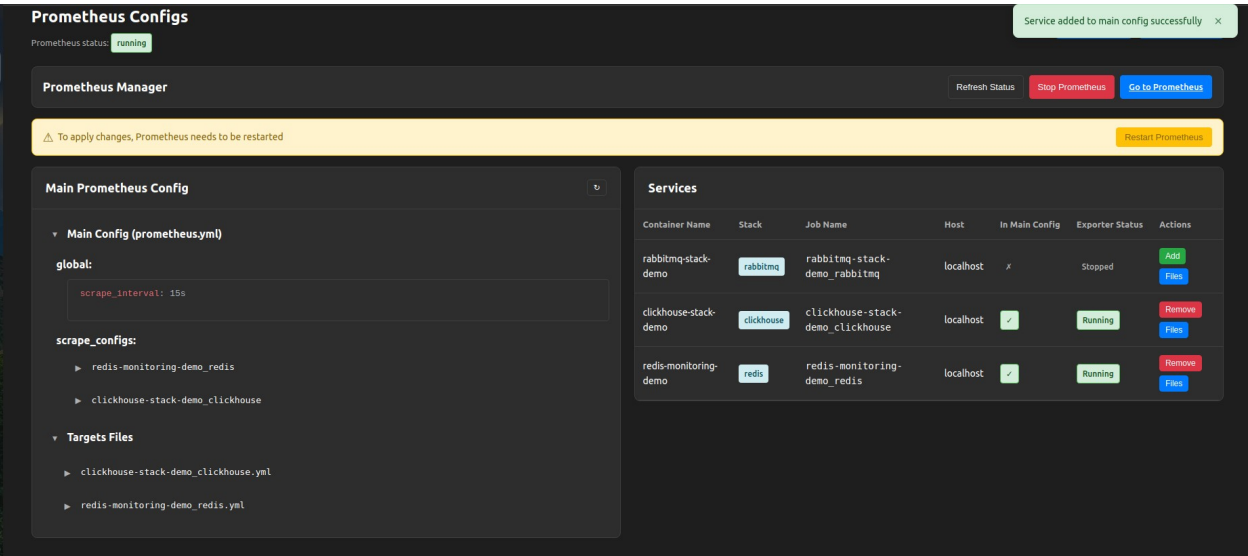


Рисунок 27 – Раздел Prometheus: предупреждение о необходимости перезапуска

Шаг 5. Откройте Go to Prometheus → Status → Targets — target в состоянии UP.

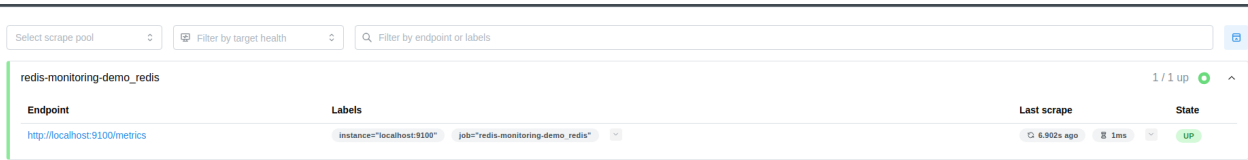


Рисунок 28 – Prometheus UI: target в состоянии UP

Шаг 6. Grafana → таблица Available Containers → Create dashboard (кнопка активна при Ready = Ready).

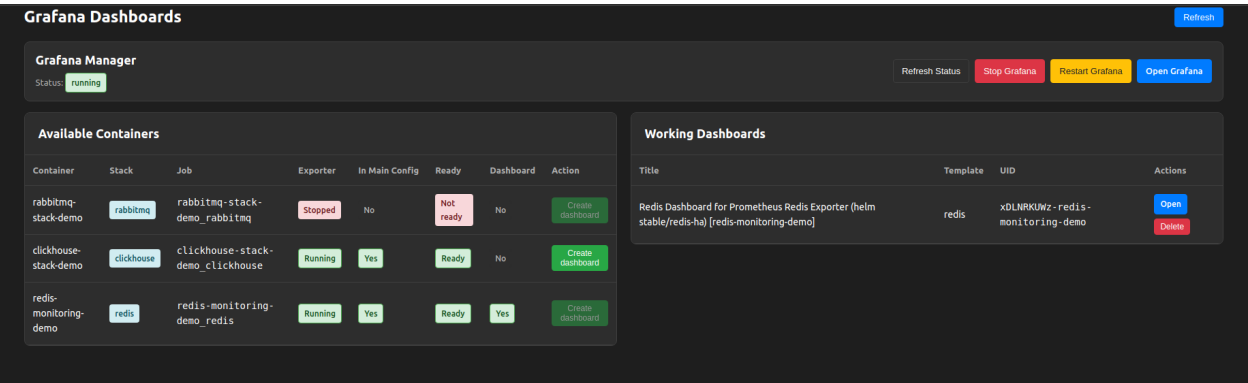


Рисунок 29 – Импорт дашборда: кнопка Create dashboard

Шаг 7. Open в таблице Working Dashboards или Open Grafana.

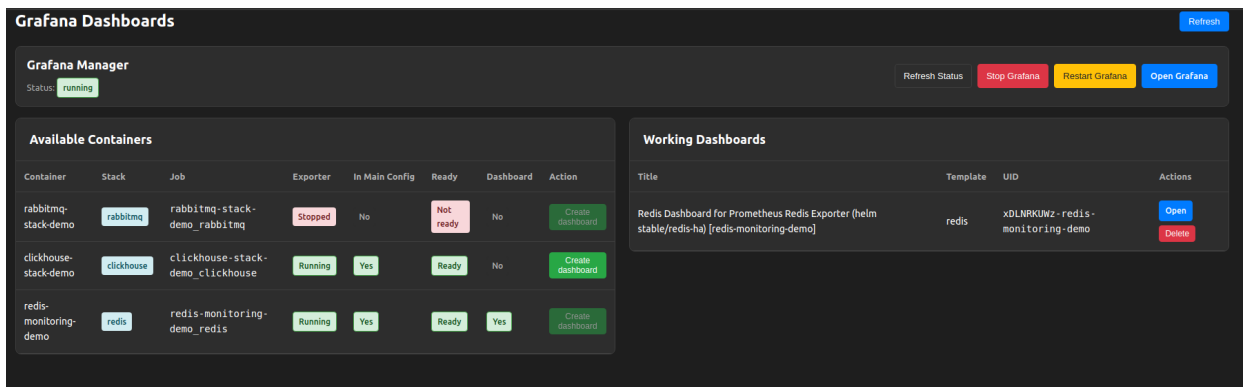


Рисунок 30 – Импортированный дашборд Grafana (PostgreSQL / Redis и т. д.)

5.8 Раздел Prometheus

Раздел Prometheus (маршрут /prometheus) реализует требования FR-11, FR-12, FR-13.

5.8.1 Блок Prometheus Manager

Содержит кнопки: Refresh Status (текущее состояние контейнера Prometheus на хосте), Start Prometheus / Stop Prometheus (жизненный цикл через prometheus_manager), Go to Prometheus (внешняя ссылка на UI Prometheus). Панель Prometheus Manager со статусом running показана на рисунке 31.



Рисунок 31 – Панель Prometheus Manager со статусом running

5.8.2 Main Prometheus Config

Представлен в левой колонке в виде дерева: Main Config (prometheus.yml) — global, список scrape_configs; Targets Files — YAML-файлы targets для каждого job. Кнопка ↻ перезагружает содержимое из MinIO.

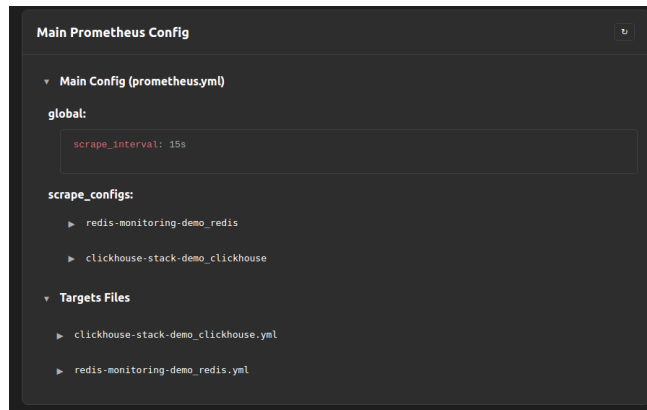


Рисунок 32 — Prometheus конфиг

5.8.3 Таблица Services

Расположена в правой колонке и содержит все сгенерированные конфигурации. Колонки: In Main Config (job включён в общий prometheus.yml), Exporter Status (sidecar запущен или нет), Actions (View files, Add/Remove from main, Delete config). Внешний вид main config и таблицы Services можно увидеть на рисунке 33.

Services						
Container Name	Stack	Job Name	Host	In Main Config	Exporter Status	Actions
rabbitmq-stack-demo	rabbitmq	rabbitmq-stack-demo_rabbitmq	localhost	✗	Stopped	Add Files
clickhouse-stack-demo	clickhouse	clickhouse-stack-demo_clickhouse	localhost	✓	Running	Remove Files
redis-monitoring-demo	redis	redis-monitoring-demo_redis	localhost	✓	Running	Remove Files

Рисунок 33 – Main config и таблица Services

5.9 Раздел Grafana

Раздел Grafana (маршрут /grafana) реализует требования FR-14, FR-15, FR-16.

5.9.1 Grafana Manager

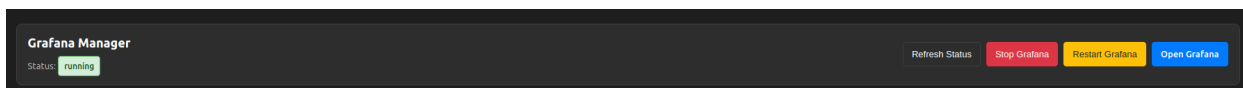


Рисунок 34 – Grafana Manager

Аналогично Prometheus: Start / Stop / Restart Grafana, Open Grafana.

5.9.2 Available Containers

Список контейнеров, для которых уже есть конфиг Prometheus. Колонка Ready:

- Ready — экспортёр работает и job в main config; можно импортировать дашборд;
- Not ready — выполните шаги из п. 5.8.

Кнопка Create dashboard неактивна, если дашборд уже создан (Dashboard: Yes).

5.9.3 Working Dashboards

Список импортированных панелей с колонками Title, Template, UID, Open, Delete. Раздел Grafana (Available Containers и Working Dashboards) показан на рисунке 35.

Working Dashboards			
Title	Template	UID	Actions
Redis Dashboard for Prometheus Redis Exporter (helm stable/redis-ha) [redis-monitoring-demo]	redis	xDLNRKUWz-redis-monitoring-demo	<button>Open</button> <button>Delete</button>

Рисунок 35 – Работающие дашборды

5.10 Раздел Configuration

Раздел Configuration (маршрут /config) реализует требование FR-10. Доступен на чтение всем; кнопка Save доступна только maintainer и admin.

Observability services (signatures.yml)- это единый файл правил, содержащий веса классификации (ports, env, image, labels), параметры экспортёров (exporter_image, exporter_port, env_template) и grafana_dashboard_id для автоматического импорта. Редактор

CodeMirror с подсветкой YAML. Перед сохранением выполняется локальная проверка синтаксиса. Редактор `signatures.yml` показан на рисунке 36.

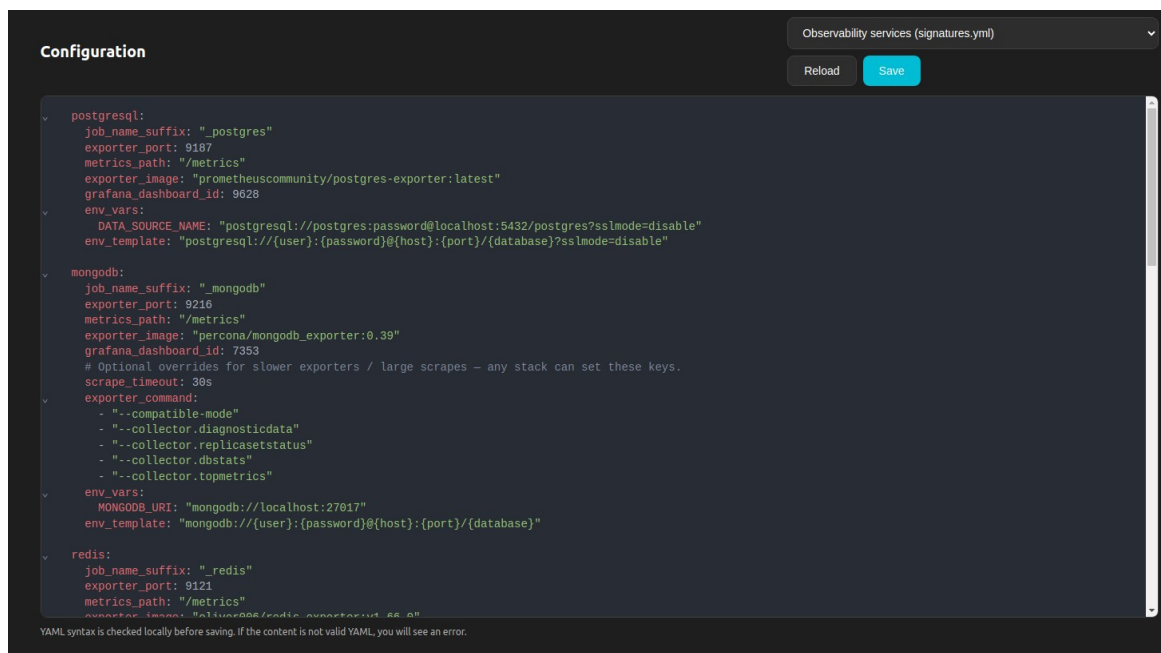


Рисунок 36 – Редактор `signatures.yml` в разделе Configuration

5.10.2 Prometheus manager settings

Второй пункт выпадающего списка, содержит настройки образа и путей Prometheus Manager (для опытных администраторов). Внимание: ошибка в YAML может нарушить классификацию и генерацию конфигов для всех новых контейнеров. Перед сохранением рекомендуется сделать резервную копию файла.

5.11 Управление пользователями (только maintainer)

Раздел «Пользователи» (маршрут `/users`) виден в меню только при роли `maintainer` (FR-02). Внешний вид раздела представлен на рисунке 36.

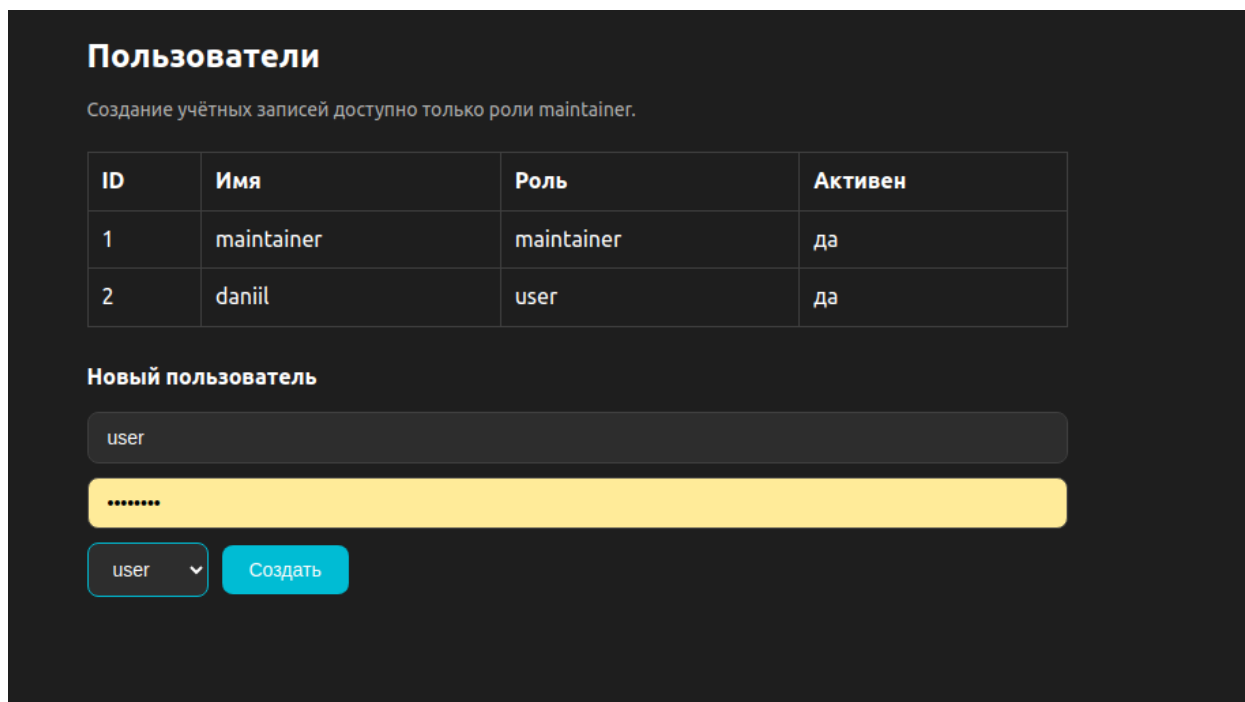


Рисунок 37 – Раздел «Пользователи»

Для создания пользователя введите логин и пароль, выберите роль (admin, dev или user — роль maintainer через UI не создаётся), затем нажмите «Создать». Новый пользователь может сразу войти через /login.

Выводы по разделу 5

В разделе приведено руководство пользователя для работы с Auto_Observability. Описаны назначение системы, требования к рабочему месту и инфраструктуре, процедуры входа и выхода. Рассмотрены основные разделы интерфейса: Hosts (регистрация Docker-хостов), Containers (центральный раздел с таблицей контейнеров, фильтрацией и меню операций), Prometheus и Grafana (управление конфигурациями и импорт дашбордов), Configuration (редактирование signatures.yml), а также управление пользователями для роли maintainer. Детально описан сквозной сценарий онбординга контейнера («Start all» и пошаговый вариант). Ключевые элементы интерфейса проиллюстрированы рисунками. Руководство полностью покрывает функциональные требования FR-01–FR-16, FR-18 и подтверждает выполнение NFR-15.

ЗАКЛЮЧЕНИЕ

В условиях повсеместного внедрения контейнеризации и микросервисной архитектуры задача обеспечения наблюдаемости (observability) инфраструктуры приобретает первостепенное значение. Рост числа короткоживущих Docker-контейнеров и распределение нагрузки по множеству хостов делают ручную настройку систем мониторинга - написание конфигураций Prometheus, развёртывание экспортеров метрик, подбор дашбордов Grafana - трудоёмкой и подверженной ошибкам рутиной, которая противоречит принципам Site Reliability Engineering о сокращении операционной нагрузки (toil) на инженеров эксплуатации.

В рамках выпускной квалификационной работы была поставлена и решена задача разработки программного комплекса Auto_Observability, реализующего концепцию автоматизированного онбординга контейнеров в контур мониторинга: от обнаружения и классификации технологического стека до генерации конфигурационных артефактов, их хранения и применения на хосте с визуализацией в Grafana. Проведённый анализ существующих решений (VictoriaMetrics/vmagent, Grafana Alloy, Netdata, Checkmk) показал, что зрелые платформы либо требуют глубокой предварительной настройки и знания предметной области, либо не обеспечивают полного цикла «определение стека → конфигурация Prometheus → экспортер → дашборд» в едином управляемом интерфейсе для централизованного парка Docker-хостов.

По результатам исследования и разработки достигнуты следующие основные результаты.

- Сформулированы и реализованы требования к системе. Определены функциональные требования (FR-01–FR-19), включая учёт хостов, синхронизацию инвентаря контейнеров, сигнатурную классификацию, генерацию конфигураций Prometheus, запуск экспортеров, управление менеджерами Prometheus и Grafana, JWT-аутентификацию и ролевую модель доступа (maintainer, admin, dev, user). Нефункциональные требования согласованы с моделью качества ISO/IEC 25010 и реализованы в виде воспроизводимого стенда на Docker Compose, кэширования в Redis, хранения YAML в MinIO и автоматизированного тестирования на pytest.
- Спроектирована и реализована микросервисная архитектура. Система построена по принципу единой точки входа (API Aggregator на FastAPI) с выделенными сервисами: Users API, Docker API, Docker Classification, Prometheus Generation, Prometheus Manager, Grafana Generation, Grafana Manager, веб-клиент на Vue 3. Обеспечено

разделение данных: PostgreSQL - метаданные и учёт настроенного мониторинга; Redis - оперативный снимок хостов и контейнеров и очереди Celery; MinIO - объектное хранилище конфигурационных YAML. Фоновые задачи Celery поддерживают актуальность инвентаря без постоянного участия оператора.

- Реализован механизм автоматической классификации и генерации мониторинга. Взвешенная система правил по меткам, образу, портам и переменным окружения контейнера определяет технологический стек (PostgreSQL, Redis, Kafka и др.). На основе единого файла `signatures.yml` формируются scrape-конфигурации и targets для Prometheus, переменные окружения экспортёров и идентификаторы дашбордов Grafana. Учтены особенности отдельных стеков (в том числе нестандартные сценарии вроде InfluxDB с Telegraf).
- Обеспечены безопасность и управляемость доступа. Реализована выдача JWT через Users API, проверка токенов и матрица RBAC на API Aggregator, разграничение прав для ролей наблюдателя и разработчика. Веб-интерфейс отражает ограничения ролей; критичные операции дублируются проверкой на сервере.
- Проведено тестирование и апробация. Разработан набор автотестов (pytest) с покрытием JWT/RBAC, роутеров агрегатора, Redis, MinIO, классификации, генерации Prometheus, Docker API и health-check микросервисов. На стенде Docker Compose выполнена ручная апробация сквозного сценария: регистрация хоста → обнаружение контейнеров → генерация конфигурации → запуск экспортёра → применение конфигурации Prometheus → импорт дашборда Grafana.

Таким образом, цель работы достигнута: создан программный комплекс, позволяющий существенно сократить ручной труд при подключении Docker-контейнеров к мониторингу и обеспечить воспроизводимый, документированный процесс онбординга через веб-интерфейс и REST API. Направления дальнейшего развития системы могут включать: внедрение refresh-токенов и интеграцию с корпоративным SSO (LDAP/OAuth2); ограничение CORS и усиление политик безопасности для production; вынос CI-прогона pytest в pipeline; поддержку оркестраторов Kubernetes; автоматическое обнаружение через service discovery без привязки только к Docker API; интеграцию с Alertmanager и распределённым хранилищем метрик; e2e-тестирование frontend; расширение библиотеки сигнатур и шаблонов дашбордов.

Практическая значимость работы заключается в том, что разработанный прототип может использоваться в учебных и исследовательских стендах, а также как основа для

промышленного развития платформы автоматизации observability в контейнерных средах предприятий и центров обработки данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бернс, Б. Проектирование распределённых систем: паттерны и парадигмы для масштабируемого программного обеспечения / Б. Бернс ; пер. с англ. - СПб. : Питер, 2020. - 280 с.
2. Бейер, Д. Site Reliability Engineering: как Google управляет надёжностью production-систем / Д. Бейер [и др.] ; пер. с англ. - СПб. : Питер, 2018. - 550 с.
3. Бейер, Д. The Site Reliability Workbook: практическое применение SRE / Д. Бейер [и др.] ; пер. с англ. - СПб. : Питер, 2019. - 520 с.
4. Клеппман, М. Высоконагруженные приложения: проектирование, разработка, адаптация / М. Клеппман ; пер. с англ. - СПб. : Питер, 2022. - 640 с.
5. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер ; пер. с англ. - М. : Вильямс, 2019. - 544 с.
6. Ричардсон, К. Микросервисы: паттерны разработки и рефакторинга / К. Ричардсон ; пер. с англ. - СПб. : Питер, 2022. - 624 с.
7. Тёрнбулл, Д. Prometheus: инфраструктура и приложения мониторинга / Д. Тёрнбулл ; пер. с англ. - СПб. : Питер, 2020. - 280 с.
8. Шарма, А. Grafana: визуализация операционных данных / А. Шарма ; пер. с англ. - СПб. : БХВ-Петербург, 2023. - 304 с.
9. Карлсон, Дж. Redis на практике / Дж. Карлсон ; пер. с англ. - М. : ДМК Пресс, 2015. - 280 с.
10. Регис, Э. PostgreSQL 15. Основы языка SQL и администрирования / Э. Регис ; пер. с англ. - СПб. : БХВ-Петербург, 2023. - 784 с.
11. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл ; пер. с англ. - М. : Вильямс, 2020. - 736 с.
12. Docker Documentation [Электронный ресурс]. - URL: <https://docs.docker.com/> (дата обращения: 20.05.2026).
13. FastAPI Documentation [Электронный ресурс]. - URL: <https://fastapi.tiangolo.com/> (дата обращения: 20.05.2026).
14. Vue.js 3 Documentation [Электронный ресурс]. - URL: <https://vuejs.org/guide/> (дата обращения: 20.05.2026).
15. MinIO Object Storage Documentation [Электронный ресурс]. - URL: <https://min.io/docs/minio/linux/index.html> (дата обращения: 20.05.2026).
16. Celery - Distributed Task Queue Documentation [Электронный ресурс]. - URL: <https://docs.celeryq.dev/> (дата обращения: 20.05.2026).

17. pytest Documentation [Электронный ресурс]. - URL: <https://docs.pytest.org/> (дата обращения: 20.05.2026).
18. Jones, M. JSON Web Token (JWT) : RFC 7519 [Электронный ресурс] / М. Jones [и др.]. - 2015. - URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 20.05.2026).
19. ISO/IEC 25010:2023. Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model [Электронный ресурс]. - URL: <https://www.iso.org/standard/35733.html> (дата обращения: 20.05.2026).

ПРИЛОЖЕНИЕ А

Листинги программного кода системы Auto_Observability

Листинг А.1 – JwtAuthMiddleware

(api_agregator/app/auth/middleware.py)

```
"""HTTP middleware: JWT + role checks for /api/v1/* on api_agregator."""

import logging
import os

from fastapi import Request, status
from fastapi.responses import JSONResponse
from starlette.middleware.base import BaseHTTPMiddleware, RequestResponseEndpoint

from app.auth.jwt_decode import decode_access_token
from app.auth.permissions import is_request_allowed

logger = logging.getLogger(__name__)

EXEMPT_PATHS = frozenset({"/", "/health", "/favicon.ico"})
EXEMPT_PREFIXES = ("/docs", "/openapi.json", "/redoc")

class JwtAuthMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next: RequestResponseEndpoint):
        path = request.url.path
        if request.method == "OPTIONS":
            return await call_next(request)
        if path in EXEMPT_PATHS:
            return await call_next(request)
        if path.startswith(EXEMPT_PREFIXES):
            return await call_next(request)
        if not path.startswith("/api/v1"):
            return await call_next(request)
```

```

if os.getenv("SKIP_JWT_AUTH", "").lower() in ("1", "true", "yes"):
    return await call_next(request)

auth = request.headers.get("authorization")
if not auth or not auth.lower().startswith("bearer "):
    return JSONResponse(
        status_code=status.HTTP_401_UNAUTHORIZED,
        content={"detail": "Not authenticated"},
        headers={"WWW-Authenticate": "Bearer"},
    )
token = auth.split(" ", 1)[1].strip()
if not token:
    return JSONResponse(
        status_code=status.HTTP_401_UNAUTHORIZED,
        content={"detail": "Not authenticated"},
        headers={"WWW-Authenticate": "Bearer"},
    )
try:
    payload = decode_access_token(token)
except Exception as exc:
    logger.debug("JWT decode failed: %s", exc)
    return JSONResponse(
        status_code=status.HTTP_401_UNAUTHORIZED,
        content={"detail": "Invalid or expired token"},
        headers={"WWW-Authenticate": "Bearer"},
    )

role = payload.get("role")
if not isinstance(role, str):
    return JSONResponse(
        status_code=status.HTTP_401_UNAUTHORIZED,
        content={"detail": "Invalid token"},
    )

```



```

if not is_request_allowed(request.method, path, role):
    return JsonResponse(
        status_code=status.HTTP_403_FORBIDDEN,
        content={"detail": "Insufficient permissions"},
    )

request.state.jwt_payload = payload
return await call_next(request)

```

Листинг А.2 – Матрица прав RBAC **(api_agregator/app/auth/permissions.py)**

```

"""Role-based access for api_agregator routes."""

from __future__ import annotations

VALID_ROLES = frozenset({"maintainer", "admin", "dev", "user"})

# (method, path) — path must match request.url.path (no query string)
_DEV_EXACT_DENY: list[tuple[str, str]] = [
    ("PATCH", "/api/v1/containers/update_containers"),
    ("POST", "/api/v1/containers/container/start"),
    ("POST", "/api/v1/containers/container/stop"),
    ("DELETE", "/api/v1/containers/container/remove"),
    ("POST", "/api/v1/prometheus/generate_config"),
    ("POST", "/api/v1/prometheus/up_exporter"),
    ("PATCH", "/api/v1/prometheus/update_signature"),
    ("POST", "/api/v1/prometheus/main_config/add"),
    ("DELETE", "/api/v1/prometheus/main_config/remove"),
    ("POST", "/api/v1/prometheus/manager/start"),
    ("POST", "/api/v1/prometheus/manager/stop"),
    ("POST", "/api/v1/prometheus/manager/settings"),
    ("POST", "/api/v1/prometheus/manager/config/update"),
    ("PUT", "/api/v1/grafana/templates.yml"),
    ("POST", "/api/v1/grafana/import_dashboard"),

```

```

("POST", "/api/v1/grafana/manager/start"),
("POST", "/api/v1/grafana/manager/stop"),
("POST", "/api/v1/grafana/manager/restart"),
]

```

```

def _dev_denied(method: str, path: str) -> bool:
    m = method.upper()
    pair = (m, path)
    if pair in _DEV_EXACT_DENY:
        return True
    if m == "DELETE" and path.startswith("/api/v1/grafana/imported_dashboards/"):
        return True
    return False

```

```

def is_request_allowed(method: str, path: str, role: str) -> bool:
    if role not in VALID_ROLES:
        return False
    if role in ("maintainer", "admin"):
        return True
    m = method.upper()
    if role == "user":
        return m == "GET"
    if role == "dev":
        if m == "GET":
            return True
        return not _dev_denied(m, path)
    return False

```

Листинг А.3 – WeightedDiscovery — классификация
(docker_classification/app/services/docker_classification.py)

```

import os
import yaml

```

```
from collections import defaultdict
```

```
class WeightedDiscovery:
```

```
    """
```

```
    Класс для классификации Docker контейнеров по технологиям.
```

```
    Использует взвешенную систему правил для определения технологий  
    на основе портов, переменных окружения, образов и меток.
```

```
    """
```

```
    def __init__(self, rules_path: str = None) -> None:
```

```
        """
```

```
        Инициализация WeightedDiscovery.
```

```
        Args:
```

```
            rules_path: Путь к файлу с правилами классификации.
```

```
            Если None, используется signatures.yml в текущей директории.
```

```
        """
```

```
        if rules_path is None:
```

```
            current_dir = os.path.dirname(os.path.abspath(__file__))
```

```
            rules_path = os.path.join(current_dir, 'signatures.yml')
```

```
        with open(rules_path, 'r', encoding='utf-8') as f:
```

```
            self.rules = yaml.safe_load(f)
```

```
        self.threshold = 50
```

```
    def classify_container(self, labels: dict, envs: list, image: str, ports: list) -> list:
```

```
        """
```

```
        Классификация контейнера по технологиям.
```

```
        Args:
```

```
            labels: Словарь с метками контейнера
```

```
            envs: Список переменных окружения
```

```
            image: Имя образа Docker
```

```
            ports: Список портов контейнера
```

Returns:

```
list: Отсортированный список кортежей (технология, балл)
"""
scores = defaultdict(int)

for port, data in self.rules['ports'].items():
    if port in ports:
        scores[data['tech']] += data['weight']

for env_key, data in self.rules['env'].items():
    if any(env_key in env_var for env_var in envs):
        scores[data['tech']] += data['weight']

for img_part, data in self.rules['images'].items():
    if img_part in image:
        scores[data['tech']] += data['weight']

for lb_key, data in self.rules['labels'].items():
    if lb_key in labels:
        tech = labels[lb_key] if data['tech'] == "auto" else data['tech']
        scores[tech] += data['weight']

final_decision = {tech: score for tech, score in scores.items() if score >= self.threshold}

return sorted(final_decision.items(), key=lambda x: x[1], reverse=True)
```

Листинг А.4 – PrometheusConfigGenerator — sidecar и YAML
(prometheus_generation/app/services/prometheus_config_generator.py)

```
def _apply_sidecar_runtime(
    self,
    stack_key: str,
    container_info: Dict[str, Any],
```

```

network_name: Optional[str],
exporter_config: Dict[str, Any],
generated_env_vars: Dict[str, str],
) -> None:

```

```

"""

```

Правит command/env/entrypoint sidecar-экспортера под реальные образы (mysqld 0.15, ES exporter, Telegraf и т.д.).

```

"""

```

```

from urllib.parse import quote

```

```

eg = self.env_generator

```

```

cname = eg._get_container_name(container_info)

```

```

env_list = container_info.get("Config", {}).get("Env", [])

```

```

env_dict = eg._parse_env_vars(env_list)

```

```

target = eg._get_container_target(container_info, cname, network_name)

```

```

if stack_key == "rabbitmq" and exporter_config.get("native_prometheus_scrape"):

```

```

    exporter_config["env_vars"] = {}

```

```

    exporter_config["exporter_command"] = None

```

```

    exporter_config.pop("exporter_entrypoint", None)

```

```

    return

```

```

if stack_key in ("mysql", "mariadb"):

```

```

    cport = eg._extract_port_from_exposed(container_info, eg._get_default_port(stack_key))

```

```

    creds = eg._get_credentials(env_dict, stack_key)

```

```

    user = creds.get("user") or "root"

```

```

    pw = creds.get("password") or ""

```

```

    exporter_config["exporter_command"] = [

```

```

        f"--mysqld.address={target}:{cport}",

```

```

        f"--mysqld.username={user}",

```

```

    ]

```

```

    exporter_config["env_vars"] = {"MYSQLD_EXPORTER_PASSWORD": pw}

```

```

    exporter_config.pop("exporter_entrypoint", None)

```

```

    return

```

```

if stack_key == "elasticsearch":
    cport = eg._extract_port_from_exposed(container_info, eg._get_default_port(stack_key))
    uri = f"http://{target}:{cport}"
    exporter_config["exporter_command"] = [
        f"--es.uri={uri}",
        "--es.ssl-skip-verify",
    ]
    exporter_config["env_vars"] = {}
    exporter_config.pop("exporter_entrypoint", None)
    return

```

```

if stack_key == "opensearch":
    cport = eg._extract_port_from_exposed(container_info, eg._get_default_port(stack_key))
    creds = eg._get_credentials(env_dict, stack_key)
    user = creds.get("user") or "admin"
    pw = creds.get("password") or ""
    if pw:
        uq = quote(str(user), safe='')
        pq = quote(str(pw), safe='')
        uri = f"http://{uq}:{pq}@{target}:{cport}"
    else:
        uri = f"http://{target}:{cport}"
    exporter_config["exporter_command"] = [
        f"--es.uri={uri}",
        "--es.ssl-skip-verify",
    ]
    exporter_config["env_vars"] = {}
    exporter_config.pop("exporter_entrypoint", None)
    return

```

```

if stack_key == "clickhouse":
    cport = eg._extract_port_from_exposed(container_info, eg._get_default_port(stack_key))
    exporter_config["exporter_command"] = [
        f"-scrape_uri=http://default@{target}:{cport}/",
    ]

```

```

exporter_config["env_vars"] = generated_env_vars or {}
exporter_config.pop("exporter_entrypoint", None)
return

if stack_key == "kafka":
    cport = eg._extract_port_from_exposed(container_info, eg._get_default_port(stack_key))
    exporter_config["exporter_command"] = [
        f"--kafka.server={target}:{cport}",
    ]
    exporter_config["env_vars"] = generated_env_vars or {}
    exporter_config.pop("exporter_entrypoint", None)
    return

if stack_key == "nats":
    uri = f"http://{target}:8222"
    exporter_config["exporter_command"] = ["-varz", uri]
    exporter_config["env_vars"] = generated_env_vars or {}
    exporter_config.pop("exporter_entrypoint", None)
    return

if stack_key == "influxdb":
    cport = eg._extract_port_from_exposed(container_info, eg._get_default_port(stack_key))
    influx_uri = f"http://{target}:{cport}"
    script = (
        f'printf "%s\n" "[[inputs.influxdb]]" " urls = [\\"{influx_uri}\\""' "" '
        "[[outputs.prometheus_client]]" " listen = \":9122\\"" " path = \"/metrics\\"" '
        "> /tmp/tg.conf && exec telegraf --config /tmp/tg.conf"
    )
    exporter_config["exporter_entrypoint"] = ["/bin/sh", "-c", script]
    exporter_config["exporter_command"] = None
    exporter_config["env_vars"] = {}
    return

exporter_config["env_vars"] = generated_env_vars or {}
exporter_config.pop("exporter_entrypoint", None)

```

```
# ... _get_exporter_host_port ...
```

```
def _build_prometheus_config(  
    self,  
    container_name: str,  
    container_info: Dict[str, Any],  
    exporter_config: Dict[str, Any],  
    target_address: str
```

```
) -> Dict[str, Any]:
```

```
    """
```

Строит конфигурацию Prometheus.

Args:

container_name: Имя контейнера

container_info: Информация о контейнере

exporter_config: Конфигурация экспортера

target_address: Адрес целевого контейнера

Returns:

Dict[str, Any]: Конфигурация Prometheus

```
    """
```

```
labels = container_info.get('Config', {}).get('Labels', {})
```

```
job_name = f"{container_name}{exporter_config.get('job_name_suffix', '')}"
```

```
scrape_interval = self._scraping_timing(  
    exporter_config.get('scrape_interval'), '15s'  
)
```

```
scrape_timeout = self._scraping_timing(  
    exporter_config.get('scrape_timeout'), '30s'  
)
```

```
scrape_config = {  
    'job_name': job_name,
```



```

'scrape_interval': scrape_interval,
'scrape_timeout': scrape_timeout,
'file_sd_configs': [
    {
        'files': [f'targets/{job_name}.yaml']
    }
]
}

```

```

scrape_port = exporter_config.get("prometheus_scrape_port")
if scrape_port is None:
    scrape_port = exporter_config.get("exporter_port", "9187")
scrape_port = str(scrape_port).strip()

```

```

target_yaml = {
    'targets': [f'{target_address}:{scrape_port}'],
    'labels': labels
}

```

```

prometheus_config = {
    'scrape_config': scrape_config,
    f'target': target_yaml
}

```

```

return prometheus_config

```

```

# ... get_container_network ...

```

```

def generate_config(self, container_data: Dict[str, Any], target_address: str) ->
Optional[Dict[str, Any]]:

```

```

"""

```

Генерирует конфигурацию Prometheus на основе данных Docker контейнера.

Args:

container_data: Данные о контейнере (info и classification)

target_address: Адрес целевого контейнера

Returns:

Optional[Dict[str, Any]]: Конфигурация Prometheus или None при ошибке

"""

```
container_info = container_data.get('info', {})
```

```
classification = container_data.get('classification', {})
```

```
container_name = self._get_container_name(container_info)
```

```
stack = self._get_stack_from_classification(classification)
```

```
if not stack:
```

```
    return None
```

```
stack_key = self._normalize_stack_name(stack)
```

```
if stack_key not in self.exporter_configs:
```

```
    return None
```

```
exporter_config = dict(self.exporter_configs[stack_key])
```

```
for _k in ("grafana_dashboard_id", "dashboard_id"):
```

```
    exporter_config.pop(_k, None)
```

```
scrape_override = container_data.get("prometheus_scrape_port")
```

```
if scrape_override is not None:
```

```
    exporter_config["prometheus_scrape_port"] = int(scrape_override)
```

```
network_name = self.get_container_network(container_info)
```

```
if stack_key == "rabbitmq" and exporter_config.get("native_prometheus_scrape"):
```

```
    generated_env_vars: dict[str, str] = {}
```

```
else:
```

```
    generated_env_vars = self.env_generator.generate_env_vars(
```

```
        container_info=container_info,
```

```
        stack_key=stack_key,
```

```
        network_name=network_name,
```

```

    )

    self._apply_sidecar_runtime(
        stack_key, container_info, network_name, exporter_config, generated_env_vars
    )

    if exporter_config.get("native_prometheus_scrape"):
        int_port = exporter_config.get("exporter_port")
        try:
            pint = int(int_port) if int_port is not None else None
        except (TypeError, ValueError):
            pint = None
        pub = self._published_host_port(container_info, pint) if pint is not None else None

```

Листинг A.5 – generate_config — онбординг Prometheus
(api_aggregator/app/routers/prometheus.py)

```

async def generate_config(
    container_id: str,
    host_id: str,
    db: Session = Depends(get_db)
) -> dict[str, Any]:
    """

```

Генерация конфига Prometheus и сохранение в БД.

Если sidecar-экспортёр ещё не запущен, конфиг всё равно генерируется (хост scrape порта берётся из БД или внутреннего порта экспортёра).

Args:

container_id: Идентификатор контейнера
 host_id: Идентификатор хоста
 db: Сессия базы данных

Returns:

dict[str, Any]: Словарь с данными о созданной конфигурации

Raises:

HTTPException: Если хост не найден или контейнер не найден

"""

```
hosts_service = HostsService(db)
```

```
host_dto = hosts_service.get_host_by_id(host_id)
```

```
if not host_dto:
```

```
    raise HTTPException(status_code=404, detail="Host not found")
```

```
host = host_dto.host
```

```
docker_containers = DockerContainers()
```

```
host_candidates = list(
```

```
    dict.fromkeys(
```

```
        x for x in (str(host_id).strip(), str(host_dto.id).strip()) if x
```

```
    )
```

```
)
```

```
container_data: dict[str, Any] = {}
```

```
redis_host_key = host_candidates[0] if host_candidates else str(host_id).strip()
```

```
for hk in host_candidates:
```

```
    cd = docker_containers.get_container(container_id, hk)
```

```
    if cd:
```

```
        container_data = cd
```

```
        redis_host_key = hk
```

```
        break
```

```
if not container_data:
```

```
    return {"error": "Container not found"}
```

```
info = container_data.get("info", {})
```

```
classification = container_data.get("classification", {})
```

```
container_name = (
```

```
    info.get("Name", "").lstrip("/") or
```

```
    info.get("Config", {}).get("Hostname", "unknown")
```

```
)
```

```

existing_cfg = (
    db.query(PrometheusConfig)
    .filter(
        PrometheusConfig.container_id == container_id,
        PrometheusConfig.status == "active",
    )
    .order_by(PrometheusConfig.created_at.desc())
    .first()
)

# Use container-reachable address for Prometheus targets.
network_settings = info.get("NetworkSettings", {})
networks = network_settings.get("Networks", {}) or {}
labels = info.get("Config", {}).get("Labels", {}) or {}

target_address = host or "localhost"
if target_address in ("127.0.0.1",):
    target_address = "localhost"
if not target_address:
    if labels.get("com.docker.compose.service"):
        target_address = labels["com.docker.compose.service"]
    elif container_name:
        target_address = container_name

all_containers_data = docker_containers.get_containers()
exporter_found, exporter_running, _, _ = _resolve_exporter_state(
    all_containers_data,
    host_name=redis_host_key,
    container_name=container_name,
)
scrape_port: int | None = None
if exporter_found and exporter_running:
    pref_internal = existing_cfg.exporter_port if existing_cfg else None
    scrape_port = resolve_scrape_host_port_for_workload(

```

```

        config_info=(existing_cfg.config_metadata or {}).get("info") if existing_cfg else None,
        exporter_internal_port=pref_internal,
        workload_container_name=container_name,
        redis_host_key=redis_host_key,
        all_containers=all_containers_data,
    )
else:
    logger.warning(
        "Exporter for %s not running or missing; generating config without live scrape port",
        container_name,
    )
if existing_cfg:
    infop = (existing_cfg.config_metadata or {}).get("info") or {}
    raw_sp = infop.get("scrape_host_port")
    if raw_sp is not None:
        try:
            scrape_port = int(raw_sp)
        except (TypeError, ValueError):
            scrape_port = None
    if scrape_port is None and existing_cfg.exporter_port is not None:
        scrape_port = int(existing_cfg.exporter_port)

stack = None
if classification.get("result"):
    stack = classification["result"][0][0] if classification["result"] else None

    logger.info("Proceeding with config generation for container %s exporter_found=%s
exporter_running=%s",
                container_id, exporter_found, exporter_running)

payload: dict[str, Any] = copy.deepcopy(container_data)
if scrape_port is not None:
    payload["prometheus_scrape_port"] = scrape_port
    logger.info(
        "Using prometheus_scrape_port=%s for container %s (host scrape)",

```

```

        scrape_port,
        container_name,
    )

api_gateway = APIGateway(prometheus_generation_url)
gen_params: dict[str, Any] = {'host': target_address}
if scrape_port is not None:
    gen_params['prometheus_scrape_port'] = int(scrape_port)
config_data = api_gateway.make_request(
    method='POST',
    endpoint='/api/v1/generate/',
    json_data=payload,
    params=gen_params,
)

exporter_config = config_data.get("info", {})
config_file = config_data.get("config", {})

container = db.query(Container).filter(Container.id == container_id).first()

if not container:
    classification_score = None
    if classification.get("result"):
        classification_score = classification.get("result")

container = Container(
    id=container_id,
    name=container_name,
    image=info.get("Config", {}).get("Image", ""),
    status=info.get("State", {}).get("Status", "unknown"),
    stack=stack,
    classification_score=classification_score,
    docker_info=info
)
db.add(container)

```

```

else:
    container.status = info.get("State", {}).get("Status", container.status)
    container.docker_info = info
    if stack:
        container.stack = stack
        container.classification_score = classification.get("result")

network_name = exporter_config.get("network")
exporter_image = exporter_config.get("exporter_image")
exporter_port = exporter_config.get("exporter_port")
job_name_suffix = exporter_config.get("job_name_suffix", "")

job_name = f"{container_name}{job_name_suffix}"

info_block: dict[str, Any] = {
    'container_name': container_name,
    'stack': stack,
    'exporter_image': exporter_image,
    'exporter_port': exporter_port,
    'target_address': target_address,
    'job_name': job_name,
    'network': network_name,
    'exporter_env_vars': exporter_config.get("env_vars", {}),
    'exporter_command': exporter_config.get("exporter_command"),
    'exporter_entrypoint': exporter_config.get("exporter_entrypoint"),
}

if "native_prometheus_scrape" in exporter_config:
    info_block["native_prometheus_scrape"] = bool(
        exporter_config["native_prometheus_scrape"]
    )

if scrape_port is not None:
    info_block["scrape_host_port"] = scrape_port
elif existing_cfg:
    prev_hp = (existing_cfg.config_metadata or {}).get("info", {}).get("scrape_host_port")
    if prev_hp is not None:

```



```

        info_block["scrape_host_port"] = prev_hp

config_metadata = {
    'host_name': redis_host_key,
    'config': config_file,
    'info': info_block,
}

existing_config = db.query(PrometheusConfig).filter(
    PrometheusConfig.container_id == container_id,
    PrometheusConfig.status == "active"
).order_by(PrometheusConfig.created_at.desc()).first()

if existing_config:
    existing_config.container_name = container_name
    existing_config.stack = stack
    existing_config.exporter_image = exporter_image
    existing_config.exporter_port = exporter_port
    existing_config.target_address = target_address
    existing_config.job_name = job_name
    existing_config.minio_bucket = config_file.get("bucket")
    existing_config.minio_file_path = config_file.get("file")
    existing_config.config_metadata = config_metadata
    existing_config.version += 1

    prometheus_config = existing_config
else:
    prometheus_config = PrometheusConfig(
        container_id=container_id,
        container_name=container_name,
        stack=stack,
        exporter_image=exporter_image,
        exporter_port=exporter_port,
        target_address=target_address,
        job_name=job_name,

```

```

        minio_bucket=config_file.get("bucket"),
        minio_file_path=config_file.get("file"),
        status="active",
        config_metadata=config_metadata
    )
    db.add(prometheus_config)
db.commit()
db.refresh(prometheus_config)

return {
    "config_id": prometheus_config.id,
    "container_id": container_id,
    "config": config_data
}

```

**Листинг А.6 – up_exporter — запуск sidecar
(api_agregator/app/routers/prometheus.py)**

```

@router.post("/up_exporter", status_code=status.HTTP_200_OK)
async def up_exporter(container_id: str, port: int, db: Session = Depends(get_db)) -> dict[str,
Any]:

```

```

    """

```

Запуск образа экспортера для контейнера.

Использует данные из сохраненной конфигурации (network и exporter_env_vars).

Args:

container_id: Идентификатор контейнера

port: Порт для экспортера

db: Сессия базы данных

Returns:

dict[str, Any]: Результат запуска экспортера или ошибка

```

    """

```

```

config = db.query(PrometheusConfig).filter(

```

```

PrometheusConfig.container_id == container_id,
PrometheusConfig.status == "active"
).order_by(PrometheusConfig.created_at.desc()).first()

docker_containers = DockerContainers()
container_data: dict[str, Any] | None = None

if not config:
    all_containers = docker_containers.get_containers()
    container_data = all_containers.get(container_id)
    if not container_data:
        logger.error("No active config and container %s not in Redis", container_id)
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=(
                "No active Prometheus config for this container. "
                "Container not found in Redis — run update containers first."
            ),
        )
    hosts_service_bootstrap = HostsService(db)
    host_dto_bootstrap = hosts_service_bootstrap.resolve_host_for_container(
        None, container_data
    )
    if not host_dto_bootstrap:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=(
                "No Docker host registered. Add a host in settings "
                "or set DOCKER_API_URL."
            ),
        )
    redis_host_key = host_dto_bootstrap.id
    logger.info(
        "No active config for %s; bootstrapping from classification before exporter start",
        container_id,

```

```

    )
    config = _bootstrap_active_prometheus_config(
        db, container_id, container_data, redis_host_key
    )

if container_data is None:
    container_data = docker_containers.get_containers().get(container_id)

config_metadata = config.config_metadata or {}
config_host_key = config_metadata.get("host_name")

# Получаем адрес хоста для обращения к docker_api
hosts_service = HostsService(db)
host_dto = hosts_service.resolve_host_for_container(config_host_key, container_data)
if not host_dto:
    logger.error("Host %s not found (container host_id=%s)", config_host_key, (container_data
or {}).get("host_id"))
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=(
            f"Host {config_host_key!r} not found. "
            "Register the Docker host or regenerate the Prometheus config."
        ),
    )
)
host_id = host_dto.id

# Преобразуем адрес хоста для Docker (localhost -> host.docker.internal)
host_address = hosts_service._resolve_host_for_docker(host_dto.host)
docker_api_host_url = f"http://{host_address}:{host_dto.port}"

if container_data is None:
    container_data = docker_containers.get_container(container_id, host_id)

# Если не найден с host_id, пробуем найти без фильтра по хосту
if not container_data:

```

```

        logger.warning("Container %s not found with host_id %s, trying to find without host filter",
container_id, host_id)
    all_containers = docker_containers.get_containers()
    container_data = all_containers.get(container_id)

    if container_data:
        logger.info("Container found without host filter, using it")
    else:
        logger.error("Target container %s not found in Redis at all", container_id)
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Target container not found in Redis. Please update containers first.",
        )

    container_info = container_data.get("info", {})
    if not container_info:
        logger.error("Container info is empty for %s", container_id)
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Container info is empty",
        )

    config_metadata = config.config_metadata or {}
    exporter_info = dict(config_metadata.get("info") or {})

    network_name = exporter_info.get("network")
    if not network_name:
        network_settings = container_info.get("NetworkSettings", {})
        networks = network_settings.get("Networks", {})
        if networks:
            network_names = list(networks.keys())
            network_name = (
                "bridge"
                if "bridge" in network_names
                else network_names[0] if network_names else None

```

```

    )

target_address = (
    exporter_info.get("target_address")
    or config.target_address
    or _resolve_target_address(container_info)
)
pref_scrape = exporter_info.get("scrape_host_port")
if pref_scrape is not None:
    try:
        pref_scrape = int(pref_scrape)
    except (TypeError, ValueError):
        pref_scrape = None

if not _exporter_sidecar_ready(exporter_info) or not network_name:
    logger.info(
        "Refreshing exporter sidecar metadata from prometheus_generation for %s",
        container_id,
    )
    _refresh_exporter_metadata_from_generation(
        db,
        config,
        container_data,
        target_address,
        prometheus_scrape_port=pref_scrape,
    )
    config_metadata = config.config_metadata or {}
    exporter_info = dict(config_metadata.get("info") or {})
    network_name = exporter_info.get("network") or network_name

if not network_name:
    network_settings = container_info.get("NetworkSettings", {})
    networks = network_settings.get("Networks", {})
    if networks:
        network_names = list(networks.keys())

```

```

network_name = (
    "bridge"
    if "bridge" in network_names
    else network_names[0] if network_names else None
)

if not network_name:
    logger.error("Target container %s has no networks. Container must be running.",
container_id)
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail="Target container has no networks. Container must be running.",
    )

if not _exporter_sidecar_ready(exporter_info):
    raise HTTPException(
        status_code=status.HTTP_400_BAD_REQUEST,
        detail=(
            "Could not resolve exporter env/command/entrypoint for this stack "
            "after refresh. Check container classification and signatures.yml."
        ),
    )

if exporter_info.get("native_prometheus_scrape"):
    try:
        iport = int(config.exporter_port) if config.exporter_port is not None else None
    except (TypeError, ValueError):
        iport = None
    hp_live = (
        host_port_from_redis_inspect({"info": container_info}, iport)
        if iport is not None
        else None
    )
    scrape_stored = int(hp_live) if hp_live is not None else int(port)
    md = dict(config.config_metadata or {})

```

```

inf = dict(md.get("info") or {})
inf["scrape_host_port"] = scrape_stored
md["info"] = inf
config.config_metadata = md
flag_modified(config, "config_metadata")
db.commit()
db.refresh(config)
try:
    await generate_config(
        container_id=config.container_id, host_id=str(host_dto.id), db=db
    )
except Exception as sync_exc:
    logger.warning(
        "Regenerate Prometheus config after native scrape sync failed for %s: %s",
        container_id,
        sync_exc,
        exc_info=True,
    )
try:
    mgr = get_prometheus_manager_gateway()
    mgr.make_request("POST", "/api/v1/manage/config/update", timeout=120.0)
except Exception as mgr_exc:
    logger.warning(
        "Prometheus manager config pull after native scrape failed: %s",
        mgr_exc,
        exc_info=True,
    )
return {
    "message": "RabbitMQ native Prometheus plugin scrape (no exporter sidecar)",
    "native_prometheus_scrape": True,
    "scrape_host_port": scrape_stored,
    "stack": config.stack,
}

```

```

exporter_env_vars = exporter_info.get("exporter_env_vars") or {}

```



```

if not isinstance(exporter_env_vars, dict):
    exporter_env_vars = {}

json_data = {
    "image_name": config.exporter_image,
    "name": f"{config.container_name}-exporter",
    "ports": {f"{config.exporter_port}/tcp": port},
    "detach": True,
    "network": network_name
}

if exporter_env_vars:
    json_data["environment"] = exporter_env_vars
    logger.info("Using env vars from config: %s", exporter_env_vars)
else:
    logger.info("No env vars for exporter %s (command/entrypoint only)", container_id)

cmd = exporter_info.get("exporter_command")
if not cmd:
    cmd = _signature_exporter_command(config.stack)
if not cmd:
    cmd = _generation_exporter_command(config.stack)
if isinstance(cmd, list):
    cmds = [str(x) for x in cmd if str(x).strip()]
    if cmds:
        json_data["command"] = cmds
elif isinstance(cmd, str) and cmd.strip():
    json_data["command"] = cmd.strip()

ep = exporter_info.get("exporter_entrypoint")
if ep:
    if isinstance(ep, list):
        ep_list = [str(x) for x in ep if str(x).strip()]
        if ep_list:
            json_data["entrypoint"] = ep_list

```

```

elif isinstance(ep, str) and ep.strip():
    json_data["entrypoint"] = ep.strip()

try:
    # Используем адрес хоста вместо глобального DOCKER_API_URL
    api_gateway = APIGateway(docker_api_host_url)
    logger.info("Using docker_api at %s for host %s", docker_api_host_url, host_id)
    start_exporter = api_gateway.make_request(
        method='POST',
        endpoint='/api/v1/manage/container/pull_and_run',
        json_data=json_data,
        timeout=300,
    )

    inner = (
        start_exporter.get("result")
        if isinstance(start_exporter, dict)
        else None
    )

    if isinstance(inner, dict) and inner.get("error"):
        logger.error(
            "docker_api pull_and_run returned error for %s: %s",
            container_id,
            inner["error"],
        )
        raise HTTPException(
            status_code=status.HTTP_502_BAD_GATEWAY,
            detail=str(inner["error"]),
        )

    if not isinstance(inner, dict) or not inner.get("container_id"):
        logger.error(
            "docker_api pull_and_run missing container_id for %s, raw=%s",
            container_id,
            start_exporter,

```

```

    )
    raise HTTPException(
        status_code=status.HTTP_502_BAD_GATEWAY,
        detail=(
            "docker_api returned success without container_id; "
            "exporter was not started reliably."
        ),
    )

logger.info("Exporter started successfully: %s", start_exporter)
all_containers = docker_containers.get_containers()
hp_live = resolve_scrape_host_port_for_workload(
    config_info=(config.config_metadata or {}).get("info"),
    exporter_internal_port=int(config.exporter_port) if config.exporter_port is not None else
None,
    workload_container_name=config.container_name.lstrip("/"),
    redis_host_key=str(config_metadata.get("host_name") or "localhost"),
    all_containers=all_containers,
)
scrape_stored = int(hp_live) if hp_live is not None else int(port)
md = dict(config.config_metadata or {})
inf = dict(md.get("info") or {})
inf["scrape_host_port"] = scrape_stored
md["info"] = inf
config.config_metadata = md
flag_modified(config, "config_metadata")
db.commit()
db.refresh(config)

try:
    await generate_config(container_id=config.container_id, host_id=str(host_dto.id),
db=db)
except Exception as sync_exc:
    logger.warning(
        "Regenerate Prometheus config after exporter start failed for %s: %s",

```

```

        container_id,
        sync_exc,
        exc_info=True,
    )
try:
    mgr = get_prometheus_manager_gateway()
    mgr.make_request("POST", "/api/v1/manage/config/update", timeout=120.0)
except Exception as mgr_exc:
    logger.warning(
        "Prometheus manager config pull after exporter start failed: %s",
        mgr_exc,
        exc_info=True,
    )

return {
    "message": "Exporter started successfully",
    "exporter": start_exporter,
    "network": network_name,
    "environment": exporter_env_vars,
    "stack": config.stack,
    "scrape_host_port": scrape_stored,
}
except HTTPException:
    raise
except Exception as e:
    logger.error("Failed to start exporter: %s", e, exc_info=True)
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail=f"Failed to start exporter: {str(e)}",
    ) from e

@router.get("/get_signature", status_code=200)
async def get_signature() -> Dict[str, str]:
    """

```

Получение файла с настройками генерации Prometheus Config.

Returns:

```
Dict[str, str]: Файл с настройками
"""

api_gateway = APIGateway(prometheus_generation_url)
result = api_gateway.make_request(
```

Листинг A.7 – UpdateConfig — MinIO → Prometheus (prometheus_manager/app/services/update_config.py)

```
def _fix_target_for_host_network(self, target: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """
```

Исправляет адрес target для работы с network_mode: host.

Заменяет IP хоста или host.docker.internal на localhost и использует внешний порт экспортера,

так как Prometheus использует network_mode: host и может обращаться к хосту напрямую.

Args:

target: Список с конфигурацией target

Returns:

```
List[Dict[str, Any]]: Исправленная конфигурация target
"""
```

```
if not target:
```

```
    return target
```

```
for target_item in target:
```

```
    if not isinstance(target_item, dict) or 'targets' not in target_item:
        continue
```

```
    targets_list = target_item.get('targets', [])
```

```
    if not targets_list:
```

```

        continue

    for i, target_address in enumerate(targets_list):
        if not isinstance(target_address, str):
            continue

        host_part = target_address.split(':')[0]
        port_part = target_address.split(':')[1] if ':' in target_address else '9187'

        needs_fix = (host_part == 'host.docker.internal' or
                     host_part.startswith('172.17.') or
                     host_part.startswith('172.18.') or
                     host_part.startswith('172.19.') or
                     host_part.startswith('172.20.') or
                     host_part == '127.0.0.1' or
                     host_part == 'localhost')

        if needs_fix:
            # Keep explicit port from target file and normalize host only.
            # Choosing "first exporter host port" is non-deterministic and can
            # break scrapes when multiple exporters are running.
            targets_list[i] = f"localhost:{port_part}"

    return target

def update(self) -> None:
    """
    Обновляет конфигурацию Prometheus из MinIO.

    Загружает основной конфиг и файлы targets из MinIO
    и сохраняет их в локальную директорию prometheus/.
    """
    main_config = self._get_yaml_file('mainConfig/prometheus.yml')
    if not main_config:

```

```

return

prometheus_yaml_path = os.path.join(self.prometheus_dir, 'prometheus.yml')
with open(prometheus_yaml_path, 'w', encoding='utf-8') as f:
    yaml.dump(main_config, f, allow_unicode=True, default_flow_style=False,
sort_keys=False)

target_files = self._list_files('mainConfig/targets/')

os.makedirs(self.targets_dir, exist_ok=True)

for file_path in target_files:
    target_content = self._get_yaml_file(file_path)
    if target_content:
        target_content = self._fix_target_for_host_network(target_content)
        file_name = file_path.split('/')[-1]
        target_path = os.path.join(self.targets_dir, file_name)

        if os.path.exists(target_path):
            try:
                if not os.access(target_path, os.W_OK):
                    os.remove(target_path)
            except (PermissionError, OSError) as e:
                try:
                    os.remove(target_path)
                except Exception:
                    pass

        try:
            with open(target_path, 'w', encoding='utf-8') as f:
                yaml.dump(
                    target_content,
                    f,
                    allow_unicode=True,
                    default_flow_style=False,

```

```

        sort_keys=False
    )
    os.chmod(target_path, 0o644)
except PermissionError as e:
    logger.error(f"Permission denied writing to {target_path}: {e}")
    raise

```

Листинг A.8 – GrafanaDashboardImporter

(grafana_generation/app/services/grafana_dashboard_importer.py)

```

def ensure_prometheus_datasource(self, ds_uid: str) -> None:
    """Ensure Grafana has Prometheus datasource with expected UID."""
    if not self.base_url:
        raise GrafanaHttpError(503, "GRAFANA_URL is not configured")
    get_url = f"{self.base_url}/api/datasources/uid/{ds_uid}"
    r = requests.get(get_url, auth=(self.user, self.password), timeout=30)
    if r.status_code == 200:
        return
    if r.status_code not in (404,):
        raise GrafanaHttpError(r.status_code, f"Failed to check datasource UID {ds_uid}: {r.text}")

    create_url = f"{self.base_url}/api/datasources"
    payload = {
        "name": "Prometheus",
        "type": "prometheus",
        "access": "proxy",
        "url": self._prometheus_url(),
        "uid": ds_uid,
        "isDefault": True,
        "jsonData": {
            "httpMethod": "POST",
        },
    }
    cr = requests.post(

```



```

        create_url,
        json=payload,
        auth=(self.user, self.password),
        headers={"Accept": "application/json", "Content-Type": "application/json"},
        timeout=60,
    )
    if cr.status_code >= 400:
        raise GrafanaHttpError(cr.status_code, f"Failed to create datasource UID {ds_uid}: {cr.text}")

```

```

def import_dashboard(
    self,
    dashboard_body: dict[str, Any],
    prometheus_ds_uid: str | None = None,
    *,
    overwrite: bool = True,
    folder_id: int | None = None,
    message: str | None = None,
) -> dict[str, Any]:
    """
    POST /api/dashboards/db
    dashboard_body должен содержать uid, panels, templating как в сохранении JSON.
    """
    if not self.base_url:
        raise GrafanaHttpError(503, "GRAFANA_URL is not configured")
    if prometheus_ds_uid:
        self.ensure_prometheus_datasource(prometheus_ds_uid)

    payload: dict[str, Any] = {
        "dashboard": dashboard_body,
        "overwrite": overwrite,
        "message": message or "imported via Auto Observability",
    }
    if folder_id is not None:
        payload["folderId"] = folder_id

```

```

url = f"{self.base_url}/api/dashboards/db"
logger.info("Importing Grafana dashboard %s via %s", dashboard_body.get("title"), url)
r = requests.post(
    url,
    json=payload,
    auth=(self.user, self.password),
    headers={"Accept": "application/json", "Content-Type": "application/json"},
    timeout=120,
)
try:
    data = r.json() if r.text else {}
except Exception:
    data = {"detail": r.text}

if r.status_code >= 400:
    raise GrafanaHttpError(
        r.status_code,
        data.get("message") or data.get("detail") or r.text or r.reason,
    )
return data

```

ПРИЛОЖЕНИЕ Б

Таблица Б.1 – Таблица hosts

Атрибут	Тип PostgreSQL	Ограничение
id	VARCHAR(255)	PRIMARY KEY, UUID по умолчанию
name	VARCHAR(255)	NULL
host	VARCHAR(255)	NOT NULL
port	INTEGER	NOT NULL

Таблица Б.2 – Таблица containers

Атрибут	Тип	Индекс
id	VARCHAR(255) PK	–
name	VARCHAR(255) NOT NULL	да
image	VARCHAR(500)	–
status	VARCHAR(50)	да
stack	VARCHAR(100)	да
classification_score	JSONB	–
docker_info	JSONB NOT NULL	–
created_at, updated_at	TIMESTAMP	–

Таблица Б.3 – Таблица prometheus_configs

Атрибут	Тип	Примечание
id	SERIAL PK	–
container_id	VARCHAR(255) FK	CASCADE
minio_bucket	VARCHAR(100)	обычно prometheus
minio_file_path	VARCHAR(500)	префикс configs/{container_id}
config_metadata	JSONB	host, env экспортера, пути файлов
version	INTEGER	default 1

Таблица Б.4 – Таблица grafana_imported_dashboards

Атрибут	Тип	Примечание
uid	VARCHAR(128) UNIQUE	ID в Grafana
template_key	VARCHAR(128)	ключ в signatures.yml (напр. redis)
source_dashboard_id	INTEGER	ID на grafana.com
prometheus_config_id	INTEGER FK nullable	связь с мониторингом

Таблица Б.5 – Таблица app_users

Атрибут	Тип	Примечание
username	VARCHAR(128) UNIQUE	логин
password_hash	TEXT	bcrypt
role	VARCHAR(32)	maintainer / admin / dev / user
is_active	BOOLEAN	блокировка учётки

